

№1 Что такое компьютерная система? Отличие информационной и управляющей системы? Почему большинство современных компьютерных систем считаются системами с преобладающей программной составляющей? Примеры. Понятие информационного процессора.

Компьютерная система - любые системы, оснащенные внутренними алгоритмами управления (т.е. внутри рассматриваемого объекта есть специальная аппаратура (например, контроллер или датчик), которая либо что-либо анализирует, либо чем-то управляет). Светодиодная лампа с контроллером напряжения, часы, дверной замок... (в целом, если задуматься, то практически все, что нас окружает - это компьютерные системы)

Информационные системы, их задача - работа с данными (получить, преобразовать, обработать, передать, изменить вид и т.д.)

- Приоритет - производительность
- Спекулятивные вычисления (предварительное выполнение задачи, для возможной минимизации задержки в будущем)
- Параллелизм (повышение производительности, расширение доступа новым пользователям)
- Кластерные и облачные вычисления (исполнение для широкого доступа и высокой надежности)

Примеры:

- Браузер
- Поисковые системы
- Старые мейнфреймы

Управляющие системы, их задача - взаимодействие с физическими объектами или явлениями для контроля и управления

- Встроенность (ограничение ресурсов (потребление энергии), спец. аппаратура, интеграция в реальный мир (под дождем, под солнцем))
- Real time (предсказуемость поведения, выполнение в заданное время)
- Автономность эксплуатации (отсутствие обслуживания, мобильность)

Примеры:

- Умные лампочки
- Вояджер

С точки зрения информации, эти классы систем очень похожи, так как в любом случае они преобразуют информацию, но с технической точки зрения они имеют **отличия**, из-за которых подходы к их разработке и проектированию значительно отличаются. К ключевым отличиям можно отнести **два следующих пункта**:

- управляющие системы имеют специальное аппаратное исполнение (набор портов ввода-вывода адаптирован под то окружение, в котором система будет работать; корпус компьютера адаптирован под место установки; процессоры подобраны с учетом используемых алгоритмов и оптимизации энергопотребления; есть дополнительное экранирование и т.п.);

- требования реального времени, связанные с тем, что компьютер должен работать не в своем "компьютерном времени", а в физическом, а значит, у нас есть конкретные требования о том, что и когда должно происходить.

Рассмотрим пример, когда одна и та же программа может выступать как в роли информационной системы, так и управляющей: видеокодек. Если он используется для преобразования видеофайла между форматами для публикации в интернете, то это типичный пример информационной системы, для которой время выполнения не принципиально (чем быстрее, тем лучше). А если он используется для видеоплеера, то это уже система реального времени, и скорость преобразования файла в видеопоток должна соответствовать количеству кадров в секунду, так как в случае слишком быстрого, медленного или скачущего воспроизведения насладиться фильмом не получится.

Системы с преобладающей программной составляющей - системы, в которых наибольшая часть средств идет на разработку software (программное обеспечение) и его дальнейшую поддержку. Так получилось, что таких систем большинство, потому что:

- Программное обеспечение проще изменить
- Стоимость ошибки при разработке software меньше, чем в случае с hardware
- Аппаратные вещи делаются более универсальными, а программная составляющая конкретизирует, что требуется от этой железяки

Про преобладание программной части - потому что на данный момент большая часть логики, алгоритмов управления, основная часть решаемых системой задач реализуется посредством создания ПО. Именно оно фиксирует все функции, их задачи, цели и последовательность выполнения. Примеры: умные часы, персональные компьютеры, роботы-пылесосы. Мы можем брать текущие стандарты и расширять их.

Информационный процессор – это система (электрическая, механическая или биологическая), которая принимает информацию (последовательность символов или состояний) в одной форме и обрабатывает (преобразует) ее в другую форму. Например, аналоговый сигнал преобразуется в цифровой. Абстрактное устройство. Включает в себя:

- ввод
- процессор (обработчик),
- хранилище
- вывод

Части м.б. вырожденные (захардкодили инпут)

Информационный процессор

Информационный процессор

система (электрическая, механическая или биологическая), которая принимает информацию (последовательность символов или состояний) в одной форме и обрабатывает (преобразует) ее в другую форму.

Например: исходный сигнал в статистику, посредством алгоритмического процесса.



Информационный процессор
включает:

1. ввод
2. процессор
3. хранилище
4. вывод

№2 Системная инженерия. Понятие системы. Варианты рассмотрения систем. Модульность. Жизненный цикл. Операционное окружение и обеспечивающие системы. Заинтересованные стороны (stakeholders). Проблема передачи информации при разработке компьютерных систем.

Системная инженерия - дисциплина, включающая в себя подходы и средства по эффективной разработке системы, нацеленной на понимании интересов и удовлетворении стекхолдеров.

Системная инженерия (SE) - это междисциплинарный подход и средство, позволяющее реализовать успешные системы. Он фокусируется на целостном и одновременном понимании потребностей заинтересованных сторон (стейкхолдеров); изучении возможностей; документировании требований; и синтезе, проверке, приемке и разработке решений при рассмотрении всей проблемы, от исследования концепции системы до вывода системы из эксплуатации.

Проще говоря, SE ставит перед собой задачу, как создавать успешные системы, а также собрать вместе представителей разных дисциплин, чтобы сделать большую систему. Ключевая роль системного инженера: координация и структуризация команд разработки, процессов, передачи информации и т.п.

Система - совокупность подсистем или элементов, способных в совокупности выполнять или производить некоторый продукт, информацию, выполнять некоторое действие или их последовательность

Варианты рассмотрения систем:

- Совокупность частей.

Система – комбинация взаимосвязанных компонентов, объединенных для достижения определенной цели, которая не может быть достигнута без объединения компонентов. В этом определении мы смотрим на систему **с точки зрения того**, из чего она сделана

- Как часть функц. место, *позволяющее ее идентифицировать*. Определяет набор функций, которые выполняет система, а не тем, как реализована система и чем она является.

Система – набор решаемых задачи или предоставляемых услуг, ограничений и требований, интеграций с другими системами и в целом окружения, в котором она находится. Такой взгляд на систему является **взглядом стейкхолдеров** и зачастую позволяет избежать большинства проблем при реализации. Функц. место определяется стейкхолдером, заинтересованной стороной, ради которой мы эту систему рассматриваем.

Stakeholder - физическое и юридическое лицо, имеющее интерес (как могут влиять на характеристики системы) к системе, которое может накладывать ограничения на вашу систему т.е. например, пользователь, который определяет дизайн, разработчик, определяющий какие именно технологии будут использоваться, директор, заинтересованный в актуальности и спросе на рынке.

Операционное окружение - окружение, в котором развертываются системы. Оно является ключевым фактором при определении возможностей системы, желаемых результатов и выгод заинтересованных сторон, а также ограничений. Пример: мы разработали телефон → нужна вышка связи. Решает, что делает, с кем и почему.

- Система как жизненный цикл
ЖЦ - процесс развития системы, начиная от идеи (план, требования) и заканчивая ее эксплуатацией или выводом из нее. Стадии, этапы:
 - концепт, разработка (код), производство (деплой/аппаратура), утилизация(использование), поддержка, вывод из эксплуатации

Обеспечивающая система - система, которая служит дополнением к рассматриваемой системе на протяжении стадий ее жизненного цикла, но необязательно вносит непосредственный вклад в ее функционирование. (пример - системы документирования или uml-диаграммы на этапе требований)

Модульность - качество системы являться совокупностью некоторых взаимодействующих модулей, которые возможно заменить или дополнить

Модульность - выбор *гранулярности компонентов, уровня абстракции и вычислительных платформ, который имеет решающее значение при разработке компьютерной системы*. То есть вся внутренняя организация системы состоит из взаимосвязанных модулей, которые в совокупности определяют целевую систему

Этот выбор должен соответствовать компьютерной системе и требованиям бизнеса. Например, если вы делаете веб-приложение - вы должны работать на веб-фреймворке с компонентами веб-страниц;

Гранулярность – это понятие, которое относится к размеру или уровню детализации, с которым выполняются операции или осуществляется разделение чего-либо

Проблемы передачи информации при разработке систем:

- формулирование информации
- передача и сохранение информации (передача информации о проекте от человека к человеку)
- использование информации (информацию мало задокументировать, ее надо принять в себя и применить на практике)
- неполнота, неоднозначность, противоречивость, решение других проблем

Poor management - через месяц уже ничего не помнишь в своем коде, плохая/неоднозначная документация, проблемы коммуникации, у каждого свой уникальный опыт. Выбрали необычное решение (mongoDB vs Oracle), если что-то не так сделать, то тебе настучат по голове потом.

№3 Цели и задачи архитектурного проектирования компьютерных систем, его эффект. Понятие архитектуры. Различные трактовки (Гради Буч, ISO 42010 и д.р.) и их практическая значимость.

Архитектура (по Гради Бучу) - **логическая и физическая структура** компонентов системы и их взаимосвязи, сформированные всеми стратегическими и тактическими проектными решениями, применяемыми во время разработки (стягивание ЖЦ в определение архитектуры системы, тк АК не может быть определена вне команды разработчиков, вне пользователей). **Логический взгляд - концепции и модели внутри этой системы** (компоненты, модули, решения и их взаимодействия, определяющие общий дизайн системы). **Физический взгляд - программный и аппаратный состав реализации системы** (системообразующая программная часть). **Важно: обе модели об организации работы обеспечивающих систем, не о конкретной реализации.** Основная идея - связь между логической и физической составляющими, это всегда командное и комплексное решение.

Архитектура (ISO 42010) - фундаментальные концепции или свойства системы в ее среде, воплощенные в ее элементах, взаимосвязях и принципах ее проектирования и развития. В системе нам важны ее внутренние элементы, отношения (с операционным окружением), принципы разработки и эволюция (развитие в будущем). **Важно: процесс разработки выше, чем готовый результат и из чего он состоит.** Разделение на архитектуру системы (некоторая абстракция, зависящая от проекта) и описание архитектуры (конкретный документ).

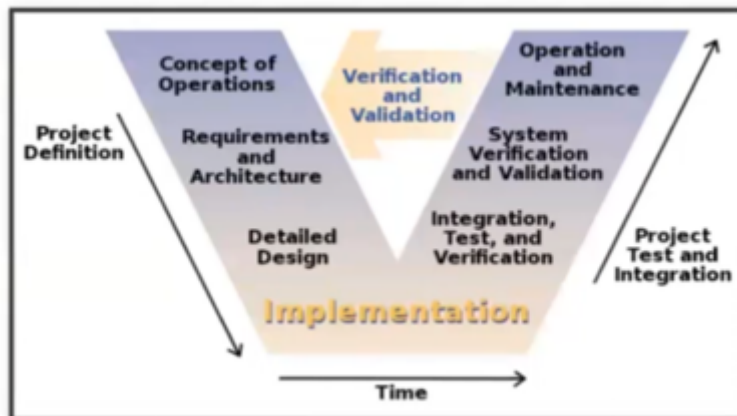
Сущностные определения:

Архитектура (интернет) - все важное.

Архитектура (Eoin Woods) - **набор проектных решений**, которые, если они будут приняты неправильно, могут привести к **отмене** вашего продукта. Есть ли разница между вариантом а и б с точки зрения успеха проекта? если нет, вопрос не архитектурный

Почему архитектура – это важно?

Чем дальше мы уходим во времени, тем больше надо исправлять и тем больше стоимость проекта (V-модель).



Архитектурное проектирование – это важно, так как оно определяет, в целом, всю нашу систему. Правильный выбор архитектуры позволяет нам как снизить затраты на саму разработку, так и существенно уменьшить риски.

Поэтому

Цели архитектурного проектирования - минимизировать риски при разработке, поддержке и изменении системы

Задачи - использовать такие концепции и принимать такие решения для конкретной создаваемой системы, которые уменьшат затраты и будут удовлетворять целям.

Эффект архитектурного проектирования - снижение рисков при разработке проекта на поздних сроках (чем раньше предотвратим, чем лучше) и вообще не уложиться в сроки.
*Усилия, потраченные на проект – это качество ПО * затраты на ПО / актуальные затраты.*

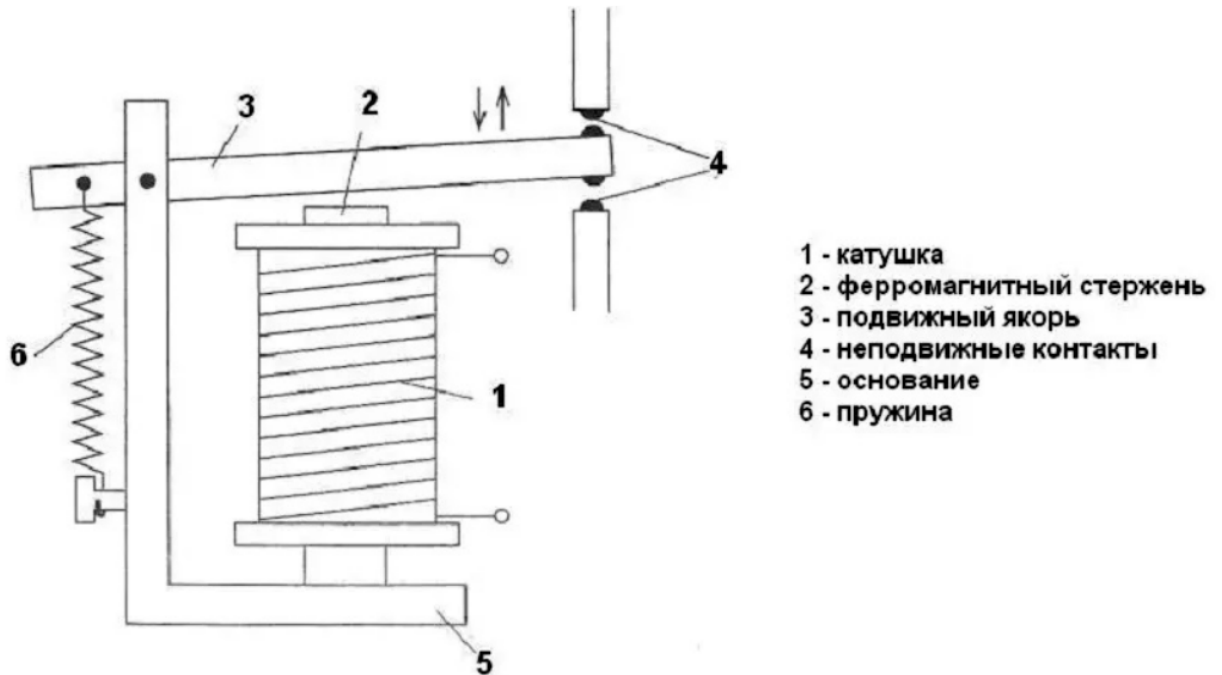
*Сроки * затраты (качество и деньги)*

№4 Реле как базис компьютерной системы. Область применения и принципы построения систем на базе реле. Примеры релейных схем и принцип их функционирования. Программируемые логические контроллеры (ПЛК). Область применения. Особенности аппаратуры и программирования по сравнению с компьютерами общего назначения.

Реле – это простая единица компьютерной техники (подобно транзистору). Т.е. реле - электромеханическое устройство, предназначенный для управления электрическими сигналами. Есть: вход и выход, между ними ключ, магнитная катушка замыкает ключ, пружинка возвращает назад. Имеет 2 состояния - 0/1 - есть питания/нет питания. Есть 2 состояния:

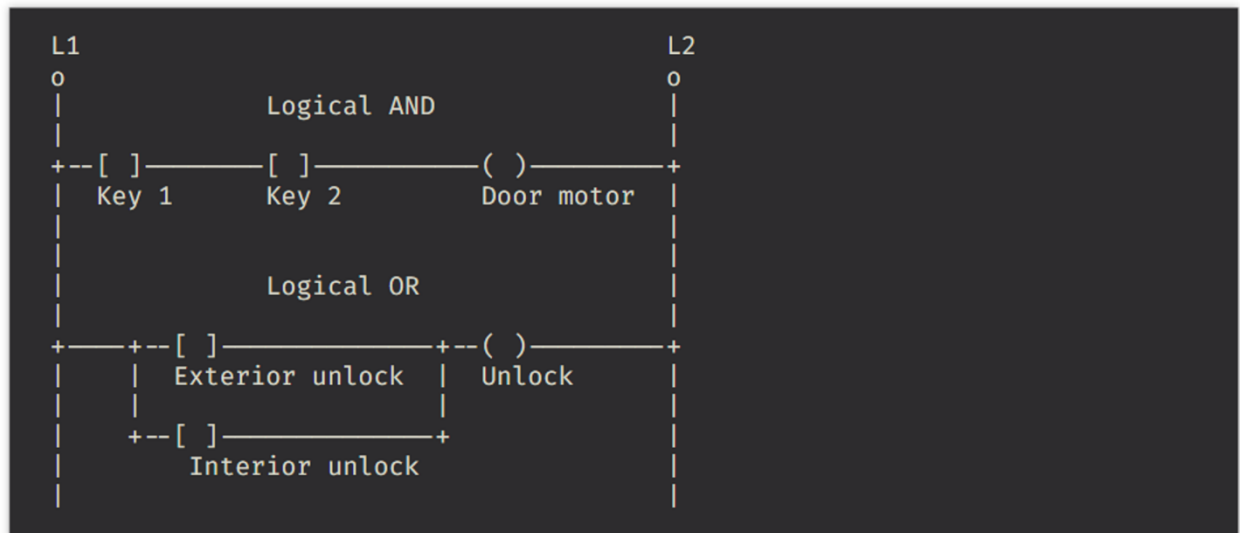
нормально разомкнутое (работает как повторитель, $1 \rightarrow 1$)

нормально замкнутое (работает как не, $1 \rightarrow 0$)

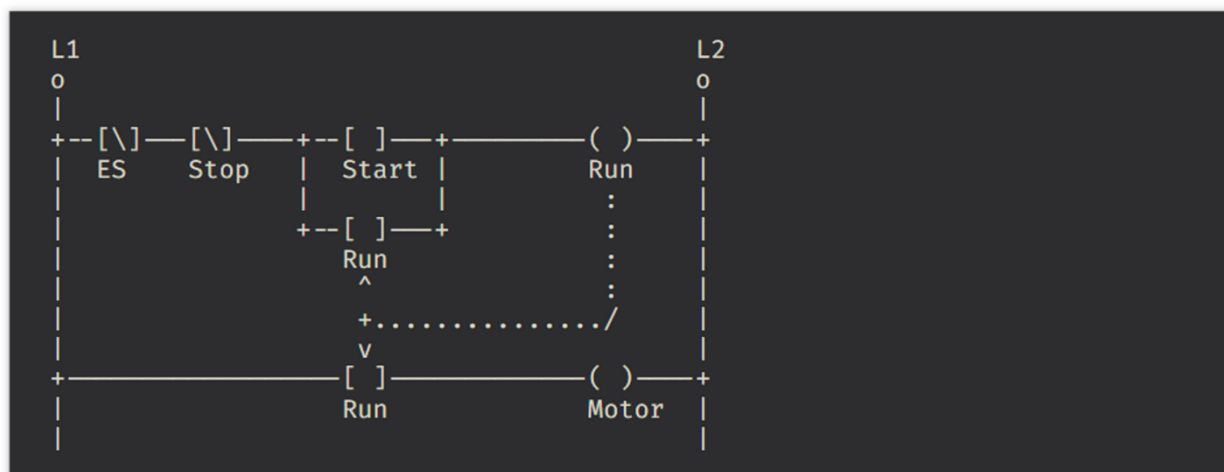


Позволяет описывать простейшие логические схемы (И, ИЛИ). Релейная диаграмма описывает включение оборудования в релейные стойки.

Пример релейной схемы:



(), () - актуаторы, [], [] – реле, L1 и L2 – линия 1 и 2. Тут примеры с дверным мотором и дверью.



Принцип построения: 2 силовые линии, между ними активаторы и реле, реле передают или нет ток, активаторы активируют реле при наличии тока. (ES – Emergency stop, нажали на кнопку старт, ток потек, ток от run - обратная связь на ИЛИ, теперь цепь будет замкнута постоянно, пока не нажмут один из стопов)

Логический базис формируется на основе 3 функций - И, ИЛИ, НЕ (либо штрих Шеффера, либо стрелка Пирса). Работают на двоичной логике. **Реле позволяют реализовать любые функции логического базиса посредством их комбинирования.**

Цифровая схемотехника. Булев базис

Элементы цифровой
схемотехники:

1. двоичная логика;
2. полный набор булевых функций:
 - И, ИЛИ, НЕ;
 - штрих Шеффера $x|y = \overline{xy}$;
 - стрелка Пирса $x \downarrow y = \overline{x \vee y}$.
3. комбинационные схемы;
4. триггер — хранение состояния.



Область применения:

Сейчас все перешли на полупроводники (они лучше), но реле применяется в:

- Раньше на их основе производили компьютеры (но они большие и не приспособлены под высокие частоты и сильную нагрузку)
- электрощитовое оборудование - силовые шкафы управления (*высокий ток*)
- особые условия эксплуатации, где электроника не работает (Венера +600), (реле бывают не только электрические, но и механические (включая пневматические), тепловые, оптические, акустические и магнитные, что позволяет применять их в тех областях, в которых электрические компоненты не могут работать;)
- Можно реализовать триггеры (например RS)
- автоматизация производственных процессов (промышленность)
 - плк (программируемые логические контроллеры)
 - *релейные диаграммы = язык реле диаграмм*

НО по сравнению с транзисторами реле:

- Потребляют больше энергии
- Больше по размеру

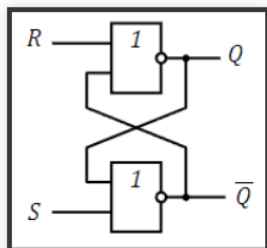
НО по сравнению с ПЛК релейные схемы:

- плохо масштабируются
- сложно настраиваются и поддерживаются
- ненадежны
- проблемно вести документацию



Реле и состояние. RS-триггер

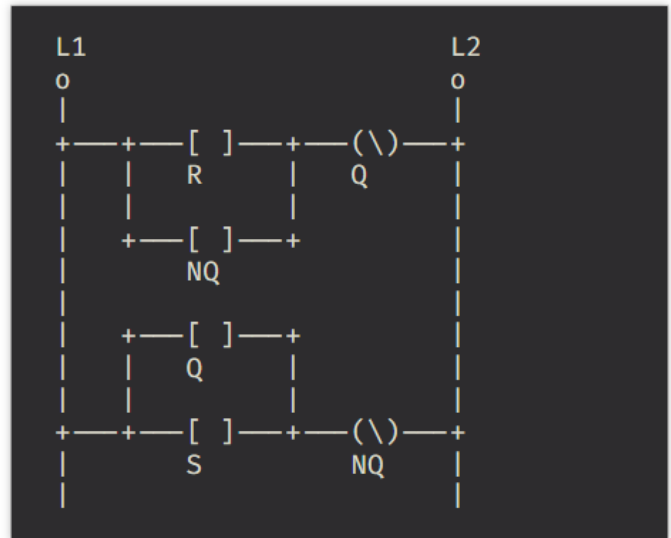
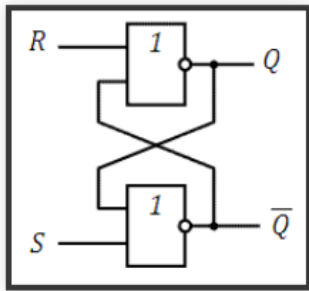
Асинхронный триггер, который сохраняет своё предыдущее состояние при неактивном состоянии обоих входов и изменяет своё состояние при подаче на один из его входов активного уровня.



S	R	Q(t)	NQ(t)
0	1	0	1
1	0	1	0
0	0	Q(t-1)	NQ(t-1)
1	1	0	0

Reset и Set

RS-триггер на реле



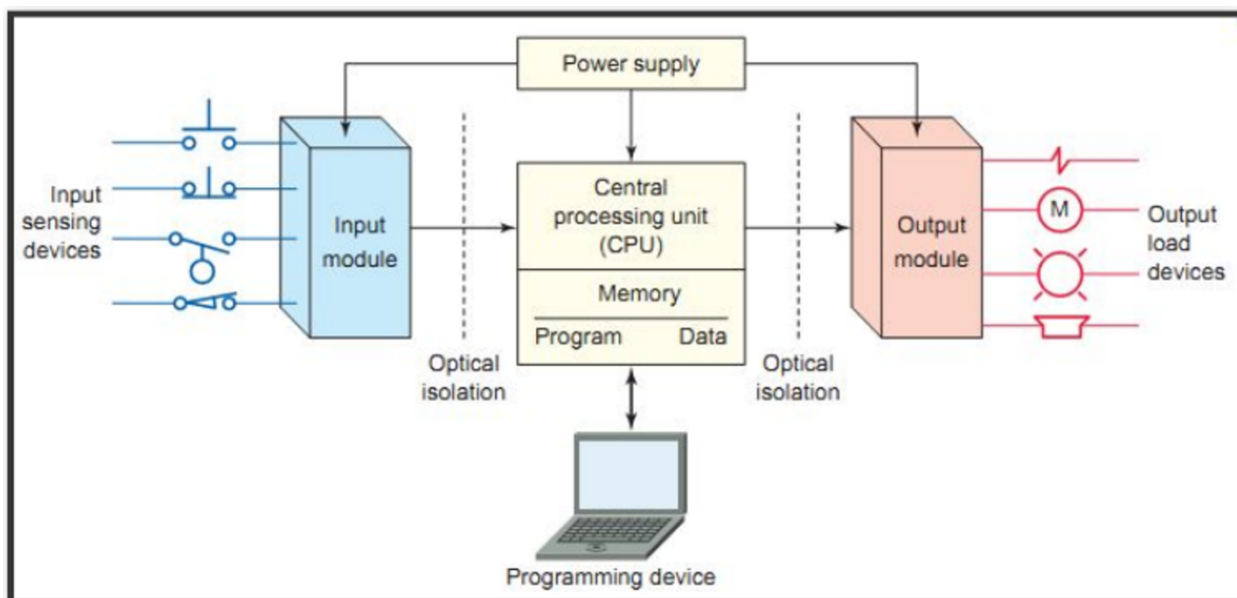
ПЛК (Программируемые логические контроллеры) - специальная разновидность электронной вычислительной машины для производства, которая пришла на замену реле. Это небольшие ящички, которые мы конфигурируем либо аппаратным (при помощи тумблеров), либо программным путями.

Особенности:

- ориентированы на управляющие системы (система в реальном времени);
- законченные устройства для встроенного использования (*поставили, подключили - работает*);
- специализация ввода-вывода, модульность (специальные коды ввода/вывода для того, чтобы получать аналоговые сигналы, контроллеры управления, то есть специализированные вещи для производственной эксплуатации);
- выделение инструментальной составляющей (разделение между инструментальными вещами и устройством, которое эксплуатируется);
- эксплуатация в тяжёлых условиях (защищенные вводы/выводы, устойчивость к статическому напряжению и тд).

Устройство

- Central control unit, как правило сейчас это обычный компьютер, к нему подключается Programming device (инструментальный компьютер), через который можно зашить функцию управления
- Есть модули ввода и вывода, получить данные из внешнего мира, отправить назад.
- Optical isolation – способ как защитить девайс от внешнего мира (не пережечь)



Стандарт IEC 61131, который содержит языки для АСУТП:

- Ladder Diagram (Релейно-контактные схемы, схемы с реле – воспроизведение того, что было в шкафу до программирования в виде документации в программной системе, которая потом работает)
- Function Block Diagram (Функциональные блочные диаграммы – позволяет коммутировать элементы со сложным управлением в формате дата флоу, в формате взаимодействующих блоков)
- Sequential Function Chart (Последовательностные функциональные диаграммы – конечные автоматы)
- Structured Text (Структурированный текст – классическое программирование на уровне бейсика)

№5 Принципы кодирования информации в компьютерных системах.

Двоичный код, достоинства и недостатки. Машинное слово и адресация. Код грея, BCD, Base64, Base58.

Цифровая схемотехника. Булев базис

Элементы цифровой
схемотехники:

1. двоичная логика;
2. полный набор булевых функций:
 - И, ИЛИ, НЕ;
 - штрих Шеффера $x|y = \overline{xy}$;
 - стрелка Пирса $x \downarrow y = \overline{x \vee y}$.
3. комбинационные схемы;
4. триггер — хранение состояния.



Чтобы реализовать двоичную логику нам нужны булевы функции (достаточно штриха Шеффера и стрелки Пирса), из минимального набора булевых функций мы можем сделать комбинационную схему и триггеры (элемент, который позволяет нам хранить состояния)

Двоичное кодирование

Информация кодируется посредством 0 или 1 и их комбинации. 0 соответствует низкий сигнал (но не его отсутствие, тк непонятно это нулевой сигнал или обрыв линии), 1 - высокий. Между ними существует запретная зона, в которой сигнал может интерпретироваться как случайный, что уменьшает ошибки при передаче от поставщика к приемщику по линии передачи.

Достоинства:

- Надежно и помехоустойчиво
- Простая схемотехника для того, чтобы реализовать простые арифметические операции
- Диапазоны и точность наращиваются разрядностью (число разрядов определяет точность)

- Погрешности by-design (означает, что погрешность определяется на этапе проектирования, а не на этапе производства схем) (погрешности, заложенные на уровне архитектуры. Нет погрешности от аппаратуры, либо работает везде и у всех, либо не работает нигде) *Погрешность “by design”, а не “by implementation” (в первом случае мы знаем про нашу погрешность и область определения, во втором, как в случае с аналоговыми машинами, значение будет дрейфовать)*

Недостатки:

- Нечитаемое представление (для человека)
- Простые десятичные дроби записываются в виде бесконечных двоичных дробей.
- Дискретное кодирование сигналов (влияет на точность)

Машинное слово - единица данных при обработке вычислителем (процессором, обычно 64 бита). Пример: сложение, пересылка, и т.п.

Машинное слово - это минимальная единица данных, с которой оперирует процессор. Размер машинного слова определяет максимальное количество информации, которое процессор может обрабатывать за одну операцию.

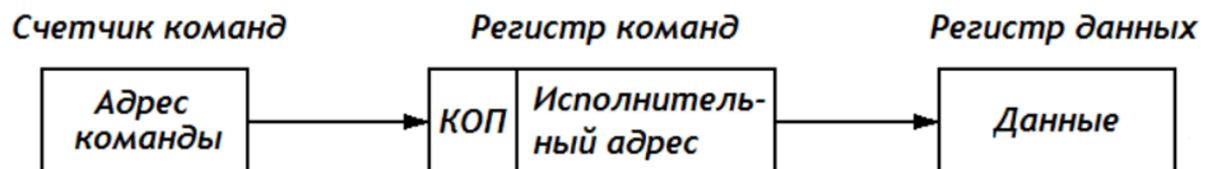
Представления данных:

- Целые числа:
 - Позиционные системы счисления.
 - Дополнительный код - для отриц. чисел
 - Big & Little endian. Big пишем как есть, little - наоборот каждые 8 бит.
- Дроби:
 - С фиксированной точкой.
 - С плавающей точкой.
- Перечисления:
 - Двоично-десятичное кодирование.
 - Символы (старые кодировки).
- Тегированные данные. Теги - проверка типов в Runtime

Адресация:

- абсолютная (В адресной части команды может быть непосредственно указан исполнительный адрес.)
- относительная (В качестве одной из компонент при формировании адреса используется счетчик команд. При этом способе адресации операнд отстоит в памяти от команды на фиксированную величину. Иными словами, операнд указывается путем задания его смещения (displacement) в памяти относительно обращаемой к операнду команды.)
- косвенная (что в ячейке) (Адресный код команды в этом случае указывает адрес ячейки памяти, в которой находится адрес операнда или команды)
- подразумеваемая (опкод) (В команде может не содержаться явных указаний об адресе участвующего в операции операнда или адресе, по которому должен быть размещен результат операции, но этот адрес подразумевается.)
- непосредственная (хардкод) (В команде содержится не адрес операнда, а непосредственно сам операнд. При непосредственной адресации не требуется обращения к памяти для выборки операнда и ячейки памяти для его хранения. Непосредственная адресация удобна для хранения различного рода констант.)

- укороченная (в команде задаются только младшие разряды адресов, а старшие разряды при этом подразумеваются нулевыми. Такая адресация позволяет использовать только небольшую часть фиксированных ячеек в начале всей адресуемой области памяти, и поэтому применяется лишь совместно с другими способами адресации)
- адресация слов переменной длины (обычно реализуется путем указания в команде местоположения в памяти начала слова и его длины.)
- стековая (Стек представляет собой группу последовательно пронумерованных регистров или ячеек памяти, снабженных указателем стека, в котором автоматически при записи и считывании устанавливается номер (адрес) последней занятой ячейки стека (вершины стека). При операции записи заносимое в стек слово помещается в следующую по порядку свободную ячейку стека, а при считывании из стека извлекается последнее поступившее в него слово.)
- автоинкремент автодекремент (Поскольку регистровая косвенная адресация требует предварительной загрузки регистра косвенным адресом из оперативной памяти, что связано с потерей времени, такой тип адресации особенно эффективен при обработке массива данных, если имеется механизм автоматического приращения или уменьшения содержимого регистра при каждом обращении к нему. Такой механизм называется соответственно автоинкрементной и автодекрементной адресацией. В этом случае достаточно один раз загрузить в регистр адрес первого обрабатываемого элемента массива, а затем при каждом обращении к регистру в нем будет формироваться адрес следующего элемента массива. При автоинкрементной адресации сначала содержимое регистра используется как адрес операнда, а затем получает приращение, равное числу байт в элементе массива. При автодекрементной адресации сначала содержимое указанного в команде регистра уменьшается на число байт в элементе массива, а затем используется как адрес операнда.)



КОП – код операции. Команда состоит из двух частей: кода операции (КОП) и адреса.

Код Грея - система кодирования помехоустойчивого кода, в котором две соседние кодовые комбинации различаются только в 1 двоичном разряде. Из-за чего помехи являются скорее шумом, чем большим скачком

Source	Text (ASCII)	M								a								n							
	Octets	77 (0x4d)								97 (0x61)								110 (0x6e)							
Bits		0	1	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0	1	1	0	1	1	1	0
Base64 encoded	Sextets	19								22								5							
	Character	T								W								F							
	Octets	84 (0x54)								87 (0x57)								70 (0x46)							

Base58 - модификация Base64, который придумали для биткоина изначально, оптимизирован для того, чтобы с ним могли работать люди, для исключения не буквенно-цифровых символов и внешне похожих символов (0 и o, 1 и i). Он by-design – в конце записи всегда имеет хэш сумму.

№6 Что такое комбинационная схема? Построение через таблицу истинности и алгоритмизацию. Состояние и параллелизм, переходные процессы. Состояние x и z. Особенности поведения комбинационных схем по сравнению с программами. Реализация "условного оператора". Программируемые логические интегральные схемы (ПЛИС) и их устройство.

Комбинационная схема - схема, составленная из набора **логических элементов**, в совокупности реализующая **заданную таблицу истинности**. Простейшими элементами являются OR, AND, NOT, BUF (этот используется для восстановления сигнала). Реализаций функций - отображение множества A на множество B. Если они конечны по размерам, то можем записать как таблицу → как схему. Построение: через таблицу истинности (черный ящик): дали на вход данные - получили данные, как работает - не волнует или через логику работы (алгоритмически), через алгоритмизацию (белый ящик, этапность/структура расчета): схема строится на основе понимания функции.

Через таблицу истинности

Шаги:

1. Формирование таблицы истинности.
2. Запись таблицы истинности в канонической форму (ДНФ или КНФ), отображаемой в комбинационную схему.
3. Минимизация булевой функции (сокращаем размер схемы).

Шифратор (Encoder, 4-to-2)

1. Таблица истинности

Input				Output	
i_3	i_2	i_1	i_0	O_1	O_0
0	0	0	0		x
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

2. КДНФ

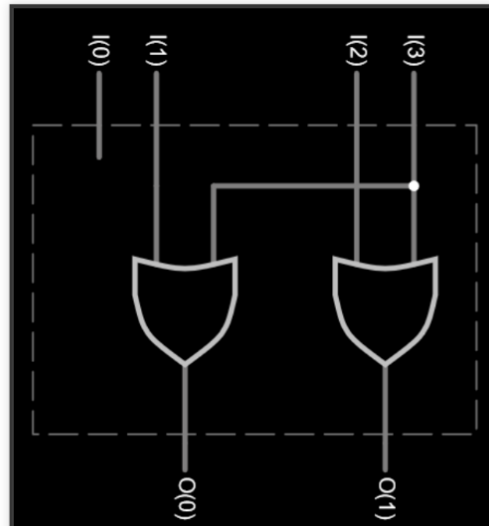
$$O_1 = \overline{i_3} \overline{i_2} \overline{i_1} i_0 \vee \overline{i_3} \overline{i_2} i_1 \overline{i_0}$$

$$O_0 = \overline{i_3} \overline{i_2} i_1 i_0 \vee \overline{i_3} i_2 \overline{i_1} i_0$$

3. Минимизированная функция

$$O_1 = i_3 \vee i_2$$

$$O_0 = i_3 \vee i_1$$

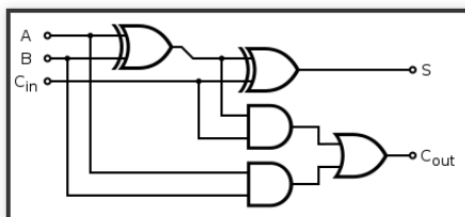


КДНФ – каноническая дизъюнктивная нормальная форма, (ККНФ – конъюнктивная)

Через алгоритмизацию

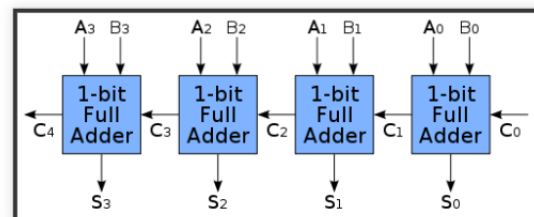
Схема строится на основе понимания функции, взаимосвязи входа и выхода. Творческий процесс. Схема может быть каскадной.

Полный сумматор (1-bit):



- A, B — биты операндов;
- S — результат;
- C_{in}, C_{out} — бит переноса.

Каскадный сумматор:



- Размер: $O(n)$.
- Для таблицы истинности: $O(n^2)$.

Полный сумматор: бит A, B и 1 бит выхода S (0 или 1), C_{out} – бит переноса, C_{in} – перенос с прошлого этапа. Если есть бит переноса, мы его перекидываем в следующий сумматор (получается каскадный сумматор). Проблема таблицы истинности: при добавлении нового столбца таблица сильно разрастается.

Свойства:

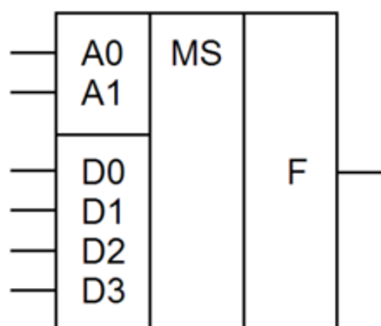
- **Стабильное состояние** при корректном вводе (стабильный вывод при стабильном входе, если нет цикла (есть значение а, которое подается на вход схеме, которая инкрементирует, дальше оно по циклу опять подается на а на вход, такая схема никогда не закончит свою работу, она все время будет находиться в переходном состоянии))
- **Задержка установления** стабильного состояния после изменения входных значений (время между изменением и новым стабильным состоянием, чем больше задержка, тем меньше частота)
- Параллелизм уровня бит (параллельная работа узлов) - узлы с параллельным включением работают параллельно, что ускоряет работу схемы
- **Накопление ошибки** в физическом процессе (возможно при прохождении через схему), что может привести к ошибке на логическом уровне (буферы) - если бы не было задержек, то мы могли бы делать КС бесконечно длинной
- Уровень линии, а не сигнал (это связано с тем, что сигнал всегда выходит)

Состояния:

- 0 и 1
- Состояние **z** - **отключено**, схема не подключена ни к чему, бесконечное сопротивление (помогает отличать отсутствие сигнала от логического нуля, тк 0 обычно не кодируют нулевым уровнем сигнала).
- Состояние **x** - **неизвестно**, произвольное значение из-за неполного определения таблицы истинности. ($\sqrt{-1}$, $x/0$)

Реализация условий:

- *через регистры*
- Условный оператор на уровне схемотехники реализуется за счет мультиплексоров комбинационная схема, которая имеет: несколько входов (+ управляющий сигнал - селектор) и один выход. Сигналы с управляющих входов показывают, какой из сигналов с сигнальных входов передать на выход. *Сокращение числа сигнальных линий и сохранение качества без потери сигнала. Демультимплексор - ровно наоборот*



Для **хранения состояния** (например, зафиксировать состояние сигнала для счетчика или ограничение распространения сигнала по схеме) в комбинационной схеме используются **триггеры** - электронные устройства, способные длительное время находиться в одном из 2 устойчивых состояний и чередовать их под воздействием внешних сигналов.

Триггер позволяет:

1. Ограничить распространение сигнала по схеме.

- Разорвать цикл (мы не можем в комб. схеме поставить цикл обратной связи, потому что она никогда не придет в стабильное состояние, поэтому мы можем вклинить триггер, который будет защелкивать состояние).
 - Сократить задержку в комбинационной схеме (это позволяет увеличить частоту и делать это конвейерно).
2. Зафиксировать состояние линии на длительное время.

Виды:

- условия изменения состояния (по уровню - хранит данные на выходе при 0 входе, при высоком сигнале повторяет входное напряжение или по фронту - переключение одновременно нескольких триггеров (D))
- Интерфейс (D (Delay) – *срабатывает по фронту синхронного сигнала и обычно на позитивном фронте он защелкивает то значение, которое было подано на вход и спустя какую-то задержку начинает его транслировать наружу, т.е. запоминает состояние на входе и выдает его на выход*, RS (Reset/Set) - асинхронный, хранит предыдущее состояние при неактивных входах и изменяет состояние при подаче на один из его входов сигнала)

Примеры многотактовых схем:

- *Переиспользование с запоминанием состояния (пространственные и временные вычисления) (сложение 64 битных чисел на 16 битном сумматоре)*
- *Конвейерное исполнение (позволяет параллельно выполнять некоторый набор задач)*
- *Итеративные расчеты (решение через n итераций)*
- *Программное управление*
- *Синхронная схемотехника (проще синхронизация, частота по самой медленной схеме, дискретизация сигналов по времени, синхронность - условна)*
- *Повышение частоты при разбиении схемы на несколько*

Особенности работы комб. схем по сравнению с программами:

1. Все процессы между регистрами всегда происходят параллельно
2. Передача сигнала - физический аналоговый процесс. Есть питание и контакт - есть передача. Сигнал не может не идти
3. Нет понятия “система остановилась”. Она всегда работает, если есть питание
4. Таблица истинности неполна - результат будет случайным, но будет всегда (и, возможно, воспроизводимым)

Интегральные схемы – это схемы изготовленные из кремниевого песка

Очень сложный этап производства, берем песок, из него выращиваем кристалл, нарезаем на ваферы, на них наносим вещество и потом через линзу пускаем свет, вырезаем мелкие интегральные схемки и корпусулируем их на одну плату. Это стоит очень дорого

dark silicon - часть чипа для отведения тепла

ПЛИС (это **FPGA (Field Programmable Gate Array** (программируемая пользователем вентиляционная матрица))) - интегральная схема для создания конфигурируемых цифровых электронных схем

- Логика работы задается не на уровне производства, а на уровне программирования, возможность при помощи программирования задать нужное ей поведение (достигается при помощи HDL - Hardware Definition Language, или HLS - High Synthesis Language)
- Получается физическое устройство с кастомной логикой функционирования посредством специального ПО и языков описания аппаратуры

ПЛИС - огромная матрица из элементов процессора (память, регистры, lookup tables, конфигурация определяет связи)

lookup table - булева функция на n бит - логические блоки - реализуется функция в зависимости от входных данных

конфигурация - соединение ячеек между собой

ПЛИС состоит из:

- Programmable Interconnect - так называемые мультиплексоры, которые связывают наши гейты
- Logic Block - маленькая логическая схема, поведение которых как раз контролируется битстримом, который задается при помощи HDL
- I/O Block
- Block RAM
- DSP Unit - блоки, которые выполняют специализированные задачи уже на аппаратном уровне (например, умножители)

CPU vs FPGA:

- Императивное программирование vs Декларативное программирование
- Временные вычисления vs Пространственные вычисления
- Динамическое планирование vs Статическое планирование
- ALU, Registers, Branch Predictors vs CLB, Switch Box, Block Memory (короче, разная микроархитектура)

СБИС (ASIC) - интегральные схемы специального назначения

Их дорого делать при малой серийности, поэтому вместо этого используют **FPGA**

Синтез - есть текстовый язык для описания схемы, он транслируется в набор булевых схем, описывающих логику (регистры и схемы), затем когда мы понимаем на какой плис с каким устройством мы должны отразить схему инструментарий это делает.

Чип - огромная матрица, конфигурация определяет связи, IO, сигналы, зашиваются таблицы по маскам

Уменьшает серийность = меньше денег тратим

Отличие от классических программ - в модели вычисления, реализуемой вычислителем и в которой мы его программируем

Высокоуровневый синтез - генерируем схему процессора программно

Высокоуровневый синтез - есть много вариантов реализации схемотехники посредством создания через языки описания архитектуры. И выбор состоит исходя из требований, выдвигаемых для создаваемого продукта (логика, число тактов и прочее). Изменение логики может повлечь препроектирование, что тормозит весь процесс, который сам по себе трудоемкий и дорогой. Из этого появилась идея высокоуровневого синтеза:

- Определяя спецификацию, определяем набор ограничений
- Система синтеза исходя из вышеуказанного создает модель схемотехники
- Затем верификация модели на соблюдение требований

Плюсы:

- Повышает скорость проектирования, ускоряет стадию прототипирования
- Позволяет соблюсти переносимость алгоритма между целевыми платформами (писами)
- Адаптируемость микроархитектуры под новые условия (при динамическом изменении ограничений)
- Автоматизация процесса оптимизации схемы

Минусы:

- Много вариантов комбинаторно. Ограниченный контроль за результатами синтеза (из-за сложности алгоритма)
- Ограничили контроль за результатом → произошла магия
- Попасть в паттерны
- Кривая обучения (тяжело стать специалистом в этом)
- Уровень зрелости технологии (зависит от инструментария)
 - о Нужна дополнительная верификация
 - о Проверка на стабильность работы

№7 2-этапное производство. Понятия Hardware и Software, их свойства.

Сравнение с понятиями программного и аппаратного обеспечения.

Проблемы, специфичные для аппаратного обеспечения: производство, эксплуатация, устаревание. Принципы совместного проектирования (HW/SW CoDesign).

Программная система (software system) - система, состоящая из программного и аппаратного обеспечений, а также данных, которые позволяют программному обеспечению работать. Software не может существовать отдельно без hardware, а также нужны дополнительные данные помимо исходного кода.

Основная идея - при появлении аппаратуры в компьютерной системе возникают вопросы рода:

- можем ли мы поменять
- а если сделали ошибку

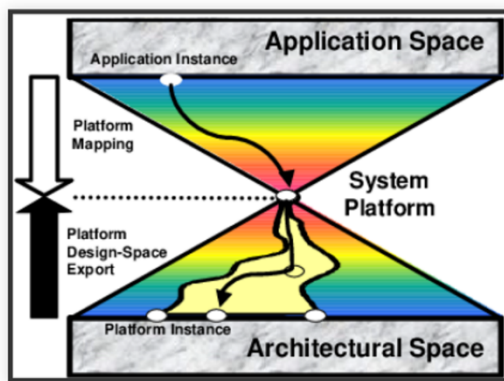
В таком случае задумались над созданием “универсальных” систем, которые создаются при помощи 2-этапного производства.

2-этапное производство - подход к построению систем:

- Производство универсальной (для класса задач) компьютерной системы (Hardware)
- Настройка прикладного поведения (Software)

Решает проблемы:

- Внесение изменений в поведение компьютера после производства
- Переносимость прикладного поведения между разным оборудованием
- Обеспечение эксплуатации системы за пределами срока жизни аппаратуры



Platform-Based Design

Уточнения для схемы:

- ↑ схема конфигурации или язык описания прикл. поведения.
- ⇕ экземпляр сконфигурированной компьютерной системы.
- ↓ экземпляр "универсальной" компьютерной системы.

Как обеспечить:

- Заложение возможности конфигурирования на этапе проектирования, также важно предусмотреть незадокументированные возможности – нецелевое использование, дыры в безопасности - незапланированная конфигурируемость)

Методы конфигурации:

- Сборка, комплектация (в зависимости от набора электроники будет вести себя по-разному)
 - Джемперы (переключатель для замыкания контактов) (Например, можно таким образом перепрошить биос и изменить стандартную загрузку системы)
 - Плата расширения (специализированные вычислители, расширение портов ввода/вывода, вынесение произвольных элементов во внешний модуль: память и т.д.) (шасси для компонентов сри, гри и т.д.)
 - Макетные платы (поверхностный монтаж) (возможность припаять что-нибудь к плате, так как элементы находятся на поверхности и не такие маленькие)
- Реконфигурация, коммутация (в зависимости от подключения)

- о Реконфигурируемые (адаптивные) архитектуры - архитектуры, обеспечивающие более быстрые и эффективные вычисления за счет перепрограммируемых интегральных схем для обработки данных и задач управления. Коммутация выч. узлов выполняется после производства.

По сути, это создание систем, которые на архитектурном уровне подразумевают возможность реконфигурации, суть в котором что **вычислитель создается из универсальных блоков и соединяете их так, чтобы они решали нужные вам задачи** (в чипе на программном уровне формируются схемы, которые вам нужны)

- Программирование, настройки, пользовательское программирование
 - о Данные определяют, какой прикладной процесс будет разворачиваться
 - о Число уровней исполнения/интерпретации данных ограничено только целесообразностью

Проблемы аппаратного обеспечения (*аппаратуры программных систем, которые состоят из SW, HW и Data*):

- Производство - при появлении аппаратной составляющей нам надо налаживать само производство (а это очень емкий процесс: логистика, склады, специалисты, производственная цепочка, тестирование, упаковка и дистрибуция), к тому же добавляется такая неприятная штука как гарантийный ремонт
- Эксплуатация - ни для кого не секрет, что любое оборудование деградирует (выводится из строя), также к нему нужен постоянный физический доступ, нужно учитывать условия эксплуатации (например, из-за этого нужно поддерживать температуру и влажность), стоит не забывать про “магические” эффекты (например, “оловянные усики”, из оловянного припоя начинают расти кристаллы, что приводит к короткому замыканию, так как кристаллы прорастают между контактными площадками). Также можно отнести такие моменты:
 - о Чем современнее технология, тем сложнее внести изменения (сравним тот же навесной монтаж и процессор)
 - о Не всегда доступен физический контакт (например, Вояджер)

Хоть это и не минус, но иногда это даже к лучшему (например, в плане безопасности)

- Устаревание - у каждой техники есть свой срок службы. Многим компаниям проще купить новое оборудование, чем обслуживать старое (чтобы не нарваться на те же самые штрафы). В плане запасных компонентов:
 - о Их дорого хранить
 - о Их число конечно

К тому же старое активно заменяется новым. Возникают сложности в нахождении устаревшего оборудования (штучное или дорогое производство, к тому же низкого спроса)

Устаревание можно разрешить разными путями:

- Полная замена элементной базы (дорого и долго)
- При помощи имитации (старый интерфейс, но новое оборудование - имитация старых систем; порой даже глушат новое оборудование, чтобы оно соответствовало старому)

- Виртуализация

Аппаратное обеспечение – это то, что существует физически, т.е. электронные и механические части вычислительного устройства, входящие в состав системы или сети, исключая программное обеспечение и данные (информацию, которую вычислительная система хранит и обрабатывает). Аппаратное обеспечение включает: компьютеры и логические устройства, внешние устройства и диагностическую аппаратуру, энергетическое оборудование, батареи и аккумуляторы.

Программное обеспечение – это то, что не имеет физического воплощения, т.е. совокупность программ, системы обработки информации и программных документов, необходимых для эксплуатации. Позволяет аппаратному обеспечению вычислительной системы выполнять вычисления или функции управления. (Программа – это данные, отчуждаемо и передаваемо, может включать документацию)

На самом деле тяжело разделить Hardware от Software и наоборот. Это не аппаратное и программное обеспечения, это граница между тем, что легко изменить и адаптировать, а что не поддается изменениям.

Software - та часть, которая приспосабливает технику к различным видам использования, можно сказать, что гибкий, flexibleware and difficultware (в плане использования)

Hardware - easyware, легко реализуемо

- Программная/аппаратная составляющая слабо связаны с SW/HW
- Наименование программной или аппаратной составляющей больше зависит от языка, чем от уровня гибкости (ПЛИС – это аппаратура, хотя программируемо)
- Разделение на SW/HW зависит в большей степени от **способа использования** элементной базы.
 - о Hardware — то, что тяжело/долго/дорого поменять
 - о Software — то, что легко/быстро/дешево поменять
- HW совпадает с аппаратной составляющей, если нет альтернатив: питание, антенны

Примеры:

- Внутри процессоров интел есть операционная система minix, которая является полноценной unix-подобной ос и software'ом

- Виртуальная машина является полностью программным элементом, но мы все равно воспринимаем ее как аппаратуру

Свойства ПО:

- Быстрый цикл разработки
- Легко заменяется у пользователя
- Пользователь как тестер
- Удаленное обновление
- Сервис - вершина владения комп. системы (полный доступ к своему сервису типа Яндекс Музыка) (мы как разработчики обладаем абсолютным доступом и абсолютной властью)
- Автоматизация процесса внедрения и создания (CI/CD)
- Высокая сложность программ

В плане проектирования системы, в которой разрабатывается как Software, так и Hardware, существуют два подхода: классический и codesign

Классический дизайн

- HW и SW решают разные задачи, разрабатываются последовательно
- Затягивает сроки, излишняя шаблонность
- Изначально разрабатываются порознь

У нас есть 2 проблемы при проектировании системы:

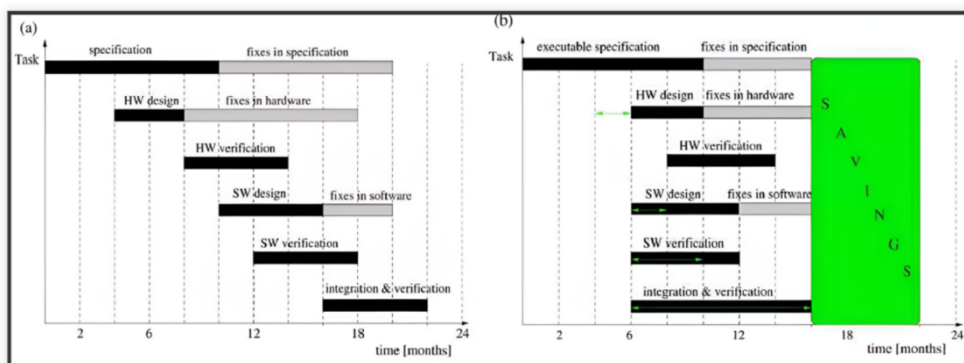
- Мы либо берем готовое шаблонное аппаратное решение, и на его основе пишем программы, что приводит к шаблонности
- Либо создаем кастомные железки с нужным нам набором команд, что затягивает процесс разработки

Hardware/Software CoDesign:

- Проектирование системы в целом
- Совместная разработка, параллельный дизайн и проектирование
- Основная идея в том, чтобы как можно дольше не разделять HW и SW (пока непонятно, что за что будет отвечать), или работаем совместно до того момента, пока не станет трудно перекидывать части между HW и SW.

Что происходит? –

1. Прописываем спецификацию (контракт, интерфейсы, определяем какие цели будет решать аппаратура, а какие программная часть)
2. Начинаем разработку параллельно
3. Тратим больше времени на спеку, но выигрываем за счет параллельной разработки



Выбор инструментов разработки SW:

- От HW и его инструментов
- От команды
- От особенностей прикладной задачи (но не так важно) (прикладная область (библиотеки, структура/принципы вычислительного процесса (модели вычислений)))

№8 Понятие модели вычислений (MoC). Сопоставление понятия парадигмы программирования и MoC. Примеры MoC (последовательные, параллельные, функциональные) и их роль в разработке компьютерных систем. Использование MoC в разных вычислительных платформах.

Всегда центральным вопросом для систем был выбор языка и спецификации, которыми мы описываем то, как система будет функционировать. Например:

- *SQL - язык для выбора данных*
- *Assembler - язык, меняющий состояние машины и приводящий ее в определенное действие*
- *Электрическая схема - как сигналы будут распространяться и изменяться*

Модель вычислений - язык для описания модели процессов/программ. Характеризует как исполняется программа: все возможные состояния вычислителя, алгоритмы в рамках этой модели в процессе вычисления. Определяет возможности вычислительной машины, как устроен и какие задачи может решать, **состояния и их последовательность, правила перехода** (пример - система команд процессора - фиксация модели вычисления). Понятие скорее про теорию (**минималистичны**), помогает проанализировать систему

- Последовательные модели - описывают системы, у которых есть состояния, которые можно менять последовательно поступающими командами. Отличия между всеми примерами последовательных моделей - число состояний (может быть бесконечным) (например, конечный автомат - машина с конечным числом состояний, находится в одном, машина Тьюринга - тот же расширенный конечный автомат, абстракция, которая катается по бесконечной ленте)
- Функциональные модели (символьные модели). *Пример - лямбда-исчисления.* Основная идея сделать вычисления высоким искусством математики: инструкция – это не пошаговое действие для какого-либо автомата, а символьное выражение, которое фиксирует текущее состояние выч. системы и все возможные будущие. Несмотря на то, что это не самый эффективный способ проводить вычисления, но дает хорошие возможности для оптимизации и предсказаний насчет того, какими свойствами будет обладать, какими не будет. К тому же при помощи таких моделей проще доказывать корректность работы программы. - *использование набора текста, который за счет простых преобразований позволяет описать произвольный вычислительный процесс комбинаторная логика (пример: лямбда-исчисления - мы описываем действия с некоторыми данными посредством специализированного набора текста. Тут также - текстом описываем вычислительный процесс некоторой схемы)*
- Параллельные модели - есть узлы, которые взаимодействуют друг с другом асинхронно. Пример, сети Кана (Golang), сети Петри, акторные модели (актор - универсальный примитив параллельного исполнения, взаимодействующий путем обмена сообщениями) (распространены в языках программирования), *описывает параллельные процессы и их взаимодействия (пример - модель акторов: актер - универсальный примитив параллельного исполнения, взаимодействующий путем обмена сообщениями)*
- Отсюда вышли распределенные модели - куча контроллеров, на которых выполняется один алгоритм

Модель вычислений сравнивают с такими понятиями, как парадигма программирования и стиль программирования. Но МоС - это в большей степени инструмент с точки зрения науки и высокой инженерии, которые позволяют оперировать вычислениями, как математическим объектом (оценка свойств, параметров, характеристик). Парадигма программирования - это инструмент дизайна для программиста, какими инструментами будет пользоваться для реализации. Стиль программирования решает вопросы, как конкретно реализовать алгоритм. Модели вычислений остаются минималистичными и простыми.

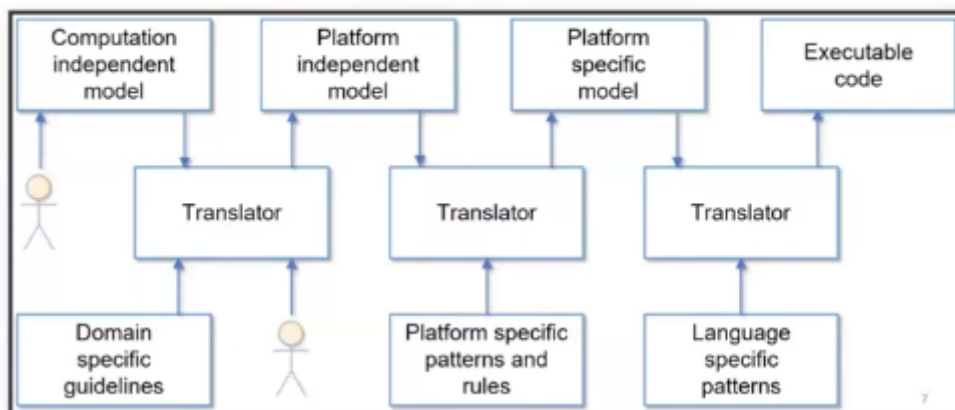
Эти понятия отчасти пересекаются, но имеют разный скоуп. **Модель вычислений** ориентирована на понимание вычислительного процесса, а **парадигма программирования** на то, чтобы облегчить жизнь программирования

МоС на практике:

- Computer Science. Формальные модели
- Дизайн языков программирования (понимание какие элементы можно сочетать)
- Управление сложностью (задаем себе жесткие рамки и получаем простую и предсказуемую программу)
- Ограничение творческого порыва
- Модель-ориентированная инженерия (xtUML, Capella, Switch-технология и т.п.) (для ответственных применений, для переносимости, для экспертизы)

Модель-ориентированная инженерия: есть цепочка трансляторов, 1 этап – определяем процессы, которые мы хотим писать в модели, которая не зависит от реализации (в бизнес-требованиях), далее добавляем руководства с точки зрения того, как это должно быть реализовано в прикладной задаче, потом формируем абстрактный вычислитель, которые сможет выполнить спецификации, которые есть, далее этап, где мы фиксируем как достигаются задачи, которые мы декларируем, потом транслируем систему четко в тех терминах, на мощностях на которых может быть запущена задача, она может быть решена, далее кодогенерация и запуск. Получается много независимости от реализации и аппаратного уровня.

Model-Driven Engineering



За любой парадигмой идет модель вычисления. Все языки современные обычно мультипарадигменные

Модель вычисления как организовать контрол флоу и дата флоу, что-то под конкретный класс специализированных задач (например конечные автоматы) **Парадигма** не про то как эффективно

и с какими свойствами, а какие **инструменты мы дадим** чтобы прогер решал задачи эффективно.

№9 Универсальный процессор и его свойства. Машина Тьюринга и полнота по Тьюрингу. Виды процессоров (СБИС, FPGA, CGRA, GPU, DSP, CPU) и их сопоставление с точки зрения универсальности и эффективности.

Информационный процессор - система (электрическая, механическая или биологическая), которая принимает информацию (последовательность символов или состояний) в одной форме и обрабатывает (преобразует) ее в другую форму. (было ранее)

Как сделать универсальнее?

Универсальный процессор - система, решающая широкий круг задач, настройка которых производится после производства “по месту” или настройка конфигурирования (runtime)

Чтобы **информационный процессор стал универсальным**, он должен обладать следующими свойствами:

1. Решать относительно большой круг задач, иметь возможности настройки конфигурирования, которые позволят использовать процессор под разные применения/фишки.
 2. Эту конфигурацию мы должны иметь возможность задать отдельно от этапа производства.
- Универсальность != не полнота по Тьюрингу - больше теоретическое замечание (универсальность - про решение широкого круга задач и конфигурируемость уже после этапа производства, полнота - алгоритм может быть вычислен на машине тьюринга. проще - условная возможность выполнить любой алгоритм)
 - Теоретическая унив != не практическая унив (машина тьюринга теор. унив., но никто не будет на ней ничего делать)
 - Универсальность противоречит эффективности (По сути, **швейцарский нож** (trade-off), если в нем будет 50 функций, никто им пользоваться не будет)
 - Современный процессор - аппаратура + ПО

Свойства:

- 2-этапное производство (HW и SW, если разделения нет, то это комбинационная схема)
- Полнота по Тьюрингу
- Изменяемость ПО
- Отсутствие серьезных ограничений на объем программы

Машина Тьюринга - абстрактная вычислительная машина. Относится к последовательным моделям. Включает:

- Неограниченную двустороннюю ленту из ячеек

- Устройство управления (головка записи-чтения) с конечным числом состояний (может перемещаться по ленте влево/вправо, читать и записывать в ячейки символы конечного алфавита)

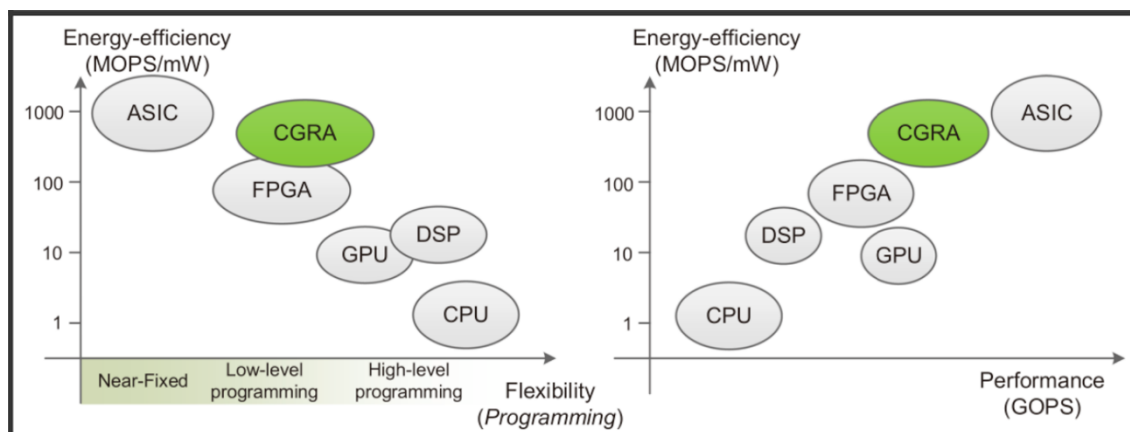
Она нереализуема, полна по Тьюрингу (позволяет реализовать любой известный алгоритм), имеет проблему остановки, данные и управление отделены (как в процессорах Фон Неймана), ориентирована на реализацию

Полнота по Тьюрингу - способность реализовать любой из известных алгоритмов

Полнота по Тьюрингу - свойство системы или языка программирования, в строгом смысле значит, что алгоритм может быть вычислен на машине Тьюринга, то есть функционально эквивалентна машине Тьюринга (позволяет реализовать любой известный алгоритм)

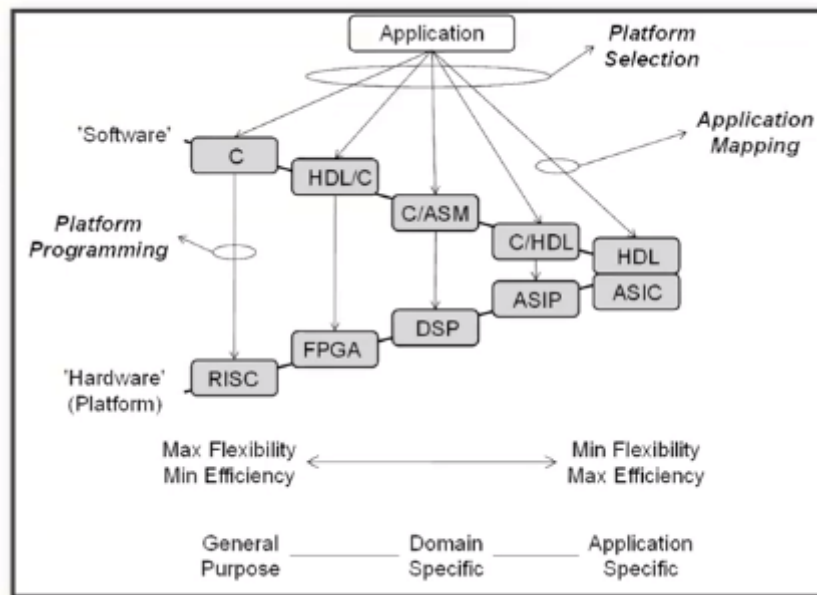
Виды процессоров:

- Различаются по гибкости с точки зрения программирования и энергетической эффективности.
- СБИС (ASIC) - схема специального назначения, решение конкретной задачи (отсутствие гибкости, но большая энергоэффективность) Пример - ASIC T5L. Например считает хэш суммы для биткоинов (майнит)
- FPGA (Программируемая пользователем вентильная матрица) - состоит из множества больших функциональных блоков, между которыми выстроена коммуникация, возможность конфигурации производителем или разработчиком после изготовления Пример – Xilinx.
- CGRA - программируемые энергоэффективные ускорители (компромисс между FPGA и процессорами параллельной обработки вида GPU). Фишка - ядро с крупными функ. блоками (не только примитивные). Из них строится программируемый вычислитель
- GPU – графический процессор, высокая эффективность благодаря конвейерной архитектуре, но имеет ограниченный набор команд
- DSP – процессоры для обработки сигналов, имеет программируемую логику, но имеет экзотический набор команд и инструкция, циклическая архитектура с циклом определенной длины
- CPU - универсальная схема с программируемой логикой и низкой энергоэффективностью



По горизонтали гибкость программного обеспечения (насколько мы можем это программировать), по вертикали энергетическая эффективность. Чем более гибкая

система, тем у нее более низкая энергетическая эффективность, чем больше возможностей для настройки, тем более энергетически-эффективная система. Вторая схема: по горизонтальности производительность. Чем более глубоко-специализированная железка под задачу, тем она лучше работает. Мы варьируемся от максимально гибкой истории с программным управлением (где мы инструкция за инструкцией обрабатываем данные, которые хранятся в памяти для CPU) в сторону того, что мы делаем структуру того же самого процессора всё более жесткими структурами (определяем их топологию, их мутацию уже аппаратными средствами, жестко их фиксируем между собой, фиксируя все более жестко те атомарные операции, которые они могут выполнять, в конечном счете придет к полностью аппаратному решению).



№10 Архитектура фон Неймана. Принципы. Свойства. Особенности и ограничения. Применение на практике. Машинное слово. Понятие системы команд и её роль в построении процессоров. Control Unit и DataPath.

Архитектура процессора - то, как видит компьютер программист. Определена набором команд (язык), местом нахождения операндов (регистры и память) и вычислительными механизмами (кеш, прерывания).

Чтобы понять архитектуру, надо в первую очередь понять систему команд - язык, на котором программист может общаться с процессором (словами являются "инструкции"/"команды". Систему команд (архитектуру системы команд) называют еще ISA.

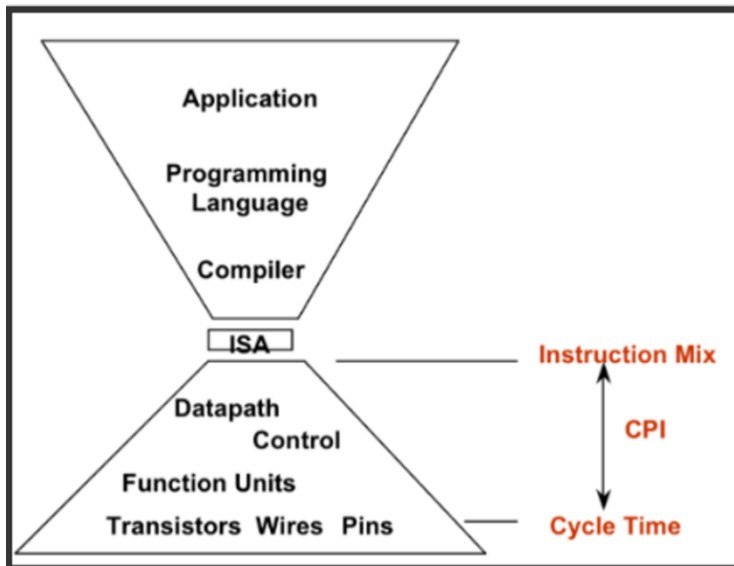
Система команд процессора - (Instruction Set Architecture — ISA) абстрактная модель процессора, формирующая интерфейс взаимодействия между программным обеспечением и процессором. В любой системе команд есть:

- Типы данных (поддерживаемые со стороны аппаратуры)
- Модель памяти, система адресации
- Набор инструкций

- Механизмы обработки прерываний и исключений
- Методы ввода и вывода

Машинное слово - это минимальная единица данных, с которой оперирует процессор. Размер машинного слова определяет максимальное количество информации, которое процессор может обрабатывать за одну операцию.

Машинное слово - фрагмент данных фиксированного размера, обрабатываемый как единое целое процессором.



Разнообразие систем команд:

- есть наборы регистров, адресное пространство и просто выполняем инструкции: произвольной сложности и длины (CISC)/фиксированного формата (RISC) (действия над памятью/регистрами (фон Нейман))
- Действие без указания обрабатываемых данных (стековый) (отсутствие инструкций, но полон по Тьюрингу) (*работаем с данными наверху стека*)
- Управляющие сигналы процессора под видом команды (NISC) (нет кодов и регистров, просто прямые сигналы процессору)
- Группа команд под видом одной команды (VLIW) (ориентировано на параллельную обработку)
- Действие над шиной данных (TTA) (кодируют не действие, перекидывают данные между вычислительными блоками)
- Команды без последовательного исполнения (редукционные) (программа в памяти, но не определяет команды для исполнения и их порядок)
- Система команд из одной команды (URISC) (1 возможная команда, но Тьюринг полные)
- Управляющие сигналы процессоров под видом команды (NITTA) (комбинация 3 и 5)

Микроархитектура процессора (microarchitecture) - соединение простейших цифровых элементов в логические блоки, предназначенные для выполнения команд определенной архитектуры (короче - то, что программист уже не видит):

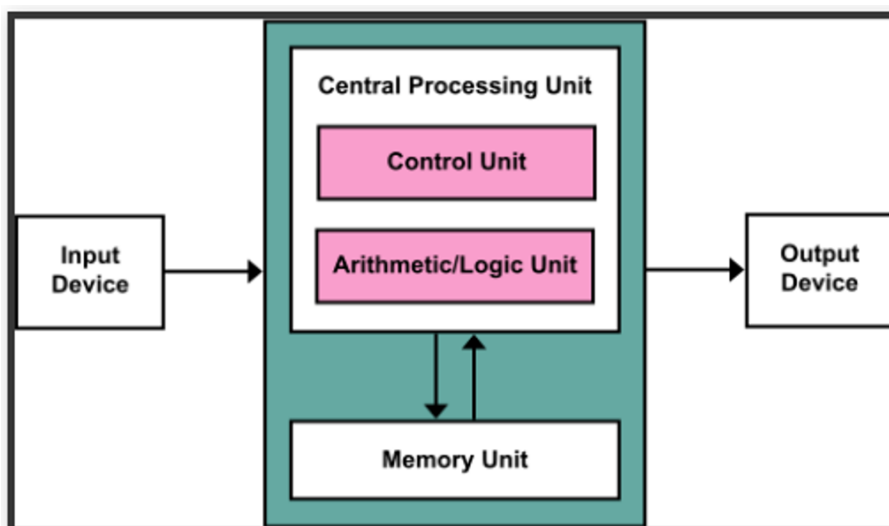
- соединение регистров
- АЛУ

- *блоки памяти*
- *интерфейсы ввода/вывода*
- *Описывает расположение и соединение простейших элементов*
- *Может быть много микроархитектур под свои задачи*
- *Если ISA и микроархитектура отличаются на уровне МоС – обычно требуется уровень виртуализации (ПО, аппаратура, транслятор, ...)*

Машина Фон Неймана - развитие машины Тьюринга. То есть у машины Тьюринга есть лента и головка, у которой есть возможность ездить по ленте, прочитывать и записывать данные. Здесь практически то же самое, только более простое для программирования: у нас есть Central Process Unit – вычислительный узел, который выполняет любое действие, которое нам нужно, который взаимодействует с памятью, читает и записывает данные оттуда, и имеет доступ к любой ячейке памяти. Т.е. отличия:

- *лента заменена на Random-Access Memory RAM (чтение и запись в произвольные ячейки памяти)*
- *Объединены инструкции и данные*

Призвана быть максимально простой в реализации.



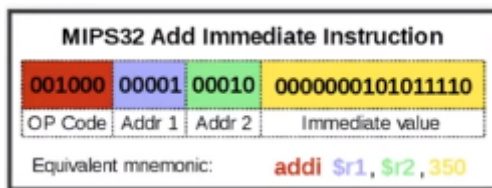
Особенности:

- *Использование двоичного кодирования (встречается троичное и двоично-десятичное кодирования)*
- *Программное управление. Команды выполняются последовательно (изменяя данные в памяти и осуществляя операцию ввода или вывода). Последовательность сегодня условна.*
- *Память компьютера однородно хранит данные и программы (память едина, мы из нее читаем и пишем данные, инструкции тоже положили в память, в результате есть область памяти, в которой хранятся инструкции и область памяти, в которой хранятся данные, и мы дальше оперируем с ними одним и тем же самым узлом)*
- *Доступ по адресам (не мотаем ленту), т.е. Ячейки памяти компьютера имеют адреса (но адрес не сводится просто к целому числу (несколько адресов сдвигов) и память уже не является пассивным элементом - она работает чуть ли не больше самого процессора)*
- *Возможность условных переходов (программа выполняется не последовательно, может прыгнуть в произвольное место), цикличность, но сейчас процессор отошел от канона и используют системы прерываний*

Виды инструкций (разделение условное, т.к. есть граничные варианты):

1. **Работа с памятью:** запись констант, копирование данных (память, регистры, порты I/O).
2. **Арифметические, логические и битовые** операции (int, float).
3. **Управляющие операции (направлены на контрол флоу):** безусловный, условный и косвенный переходы, вызов и возврат из подпрограмм.
4. **Инструкции сопроцессоров:**
 - Загрузка и выгрузка данных.
 - Управление сопроцессором

1 и 2 – это датапас, 3 – контрол юнит



- *OP Code* — код операции.
- *Addr 1/2* — номера регистров общего назначения.
- *Immediate value* — непосредственное значение.
- $\$r1 \leftarrow \$r2 + 350$
- При переполнении:
 - $\$r1$ — без изменений,
 - установить флаг *overflow*.

Control Unit - это блок управления, основной задачей которого является управления процессором. Преобразует закодированные **инструкции в сигналы** синхронизации и управления

Реализации:

- *Hardwire* (декодирование инструкций в последовательные сигналы при помощи **аппаратных схем**, простота за счет системы просто конечных автоматов, но растет по сложности от системы команд)
- *Microwired* (декодирование при помощи **микропрограммы** (программа, реализующая набор инструкций процессора, делает из инструкций в памяти микрооперации (управляющие сигналы) и подает их на Control Unit) с необходимыми функциями). По сути, **битовая маска**. При этом для каждой инструкции - свой набор микрокоманд
 - Просто в реализации, изменение системы команд на лету, влияние на эффективность через адаптирование системы команд
 - Микрокод в процессоре (занимает место), сложно в изучении, разнообразие архитектур (поэтому много проблем с адаптацией к каждому отдельному), разнообразие команд (сложно оптимизировать, инструментарий), все проблемы программирование (отладка)
- **hardwired** - есть куча аппаратной логики, которая получает operation code на вход, считает сколько тактов понадобится на выполнение операции, отправляет управляющие сигналы как цифровая схема, сводится к небольшой системе конечных автоматов, но есть большой недостаток, так как при росте

сложности системы команд конечный автомат также очень быстро растет по сложности и появляется большой шанс накосячить

- **microcoded** - парсим нашу команду на множество микро-команд, которые тоже являются набором управляющих сигналов, но так как мы читаем эту инфу из данных, а не реализуем эту логику аппаратно, мы получаем свободу модификации и право на ошибку

Datapath - набор компонентов, обеспечивающих передачу и управление данными между компонентами процессора (**регистры, шины, алу**) (эту вещь еще называют трактом данных)

Практика:

- В чистом виде практически не встречается (много параллельности + проц сам решает выполнение, **Память может сказать, что она занята (не пассивный элемент), однородность спорна**)
- Огромное влияние на индустрию
- Элементы архитектуры присутствуют в подавляющем большинстве процессоров и средств разработки
- Структурное программирование, ООП - наследие фон Неймана (подход на базе вызова методов, у нас есть процессор, это наш объект, далее мы вызываем метод, который изменяет его внутреннее состояние в зависимости от того какая инструкция написана)
- Ловушка обратной совместимости (все завязано на этом, перейти к другому крайне сложно) (мы стали заложниками данной технологии - что делать с программистами, которые привыкли к такой архитектуре?)

Развитие архитектуры ФН (вариации):

- Память команд и данных – память это узкое место, потому что почти каждый раз нам надо ходить в память за инструкцией, в память за данными, постоянное общение с памятью (Гарвардская - сложнее и Принстонская - узкое место шина данных)
 - о типы данных, машинные слова, теги
- По адресации:
 - о абсолютная
 - о относительная ($addr + base$)
 - о косвенная (что в ячейке)
 - о подразумеваемая (опкод)
 - о непосредственная (хардкод)
 - о укороченная (часть адреса)
 - о адресация слов переменной длины
 - о стековая
 - о автоинкремент автодекремент
- Принцип работы с данными в DataPath

Accumulator (центральный регистр, работа с которым построена на том, что мы все время туда что-то добавляем, вычитаем, сбрасываем и все вокруг вокруг одной ячейки, простое решение, но не очень эффективно, становится узким местом), Register-to-Register (работа выполняется на конечном множестве регистров, допустим 5 аккумуляторов, и есть буферы промежуточных значений, штуки 3, которые позволяют оптимизировать вычисления), Register-to-Memory (использует одну ячейку памяти, один регистр и результат можно сохранить в памяти), Memory-to-Memory, Stack Architectures

- "Мощность" системы команд: CISC (много инструкций на любой случай жизни, выигрываем в производительности), RISC (мало инструкций, но сделаны очень хорошо)
 - Количество операндов (0, 1, 2...)
 - Кодирование инструкций
 - Принципы построения Control Unit
- Ввод-вывод, параллелизм...

№11 Гарвардская архитектура и её отличия от архитектуры фон Неймана. Принципы. Свойства. Особенности и ограничения. Применение на практике. Подходы к обходу ограничений архитектуры. Машинное слово. Control Unit и DataPath.

Архитектура процессора - то, как видит компьютер программист. Определена набором команд (язык), местом нахождения операндов (регистры и память) и вычислительными механизмами (кеш, прерывания).

Чтобы понять архитектуру, надо в первую очередь понять систему команд - язык, на котором программист может общаться с процессором (словами являются "инструкции"/"команды". Систему команд (архитектуру системы команд) называют еще ISA.

Система команд процессора - (Instruction Set Architecture — ISA) абстрактная модель процессора, формирующая интерфейс взаимодействия между программным обеспечением и процессором. В любой системе команд есть:

- Типы данных (поддерживаемые со стороны аппаратуры)
- Модель памяти, система адресации
- Набор инструкций
- Механизмы обработки прерываний и исключений
- Методы ввода и вывода

Машинное слово - фрагмент данных фиксированного размера, обрабатываемый как единое целое процессором.

Машинное слово - это минимальная единица данных, с которой оперирует процессор. Размер машинного слова определяет максимальное количество информации, которое процессор может обрабатывать за одну операцию.

Разнообразие систем команд:

- есть наборы регистров, адресное пространство и просто выполняем инструкции: произвольной сложности и длины (CISC)/фиксированного формата (RISC) (действия над памятью/регистрами (фон Нейман))
- Действие без указания обрабатываемых данных (стековый) (отсутствие инструкций, но полон по Тьюрингу) (работаем с данными наверху стека)

- Управляющие сигналы процессора под видом команды (NISC) (нет кодов и регистров, просто прямые сигналы процессору)
- Группа команд под видом одной команды (VLIW) (ориентировано на параллельную обработку)
- Действие над шиной данных (TTA) (кодируют не действие, перекидывают данные между вычислительными блоками)
- Команды без последовательного исполнения (редукционные) (программа в памяти, но не определяет команды для исполнения и их порядок)
- Система команд из одной команды (URISC) (1 возможная команда, но Тьюринг полные)
- Управляющие сигналы процессоров под видом команды (NITTA) (комбинация 3 и 5)

Микроархитектура процессора (microarchitecture) - соединение простейших цифровых элементов в логические блоки, предназначенные для выполнения команд определенной архитектуры (короче - то, что программист уже не видит):

- соединение регистров
 - АЛУ
 - блоки памяти
 - интерфейсы ввода/вывода
- Описывает расположение и соединение простейших элементов
 - Может быть много микроархитектур под свои задачи
 - Если ISA и микроархитектура отличаются на уровне МоС – обычно требуется уровень виртуализации (ПО, аппаратура, транслятор, ...)

Гарвардская архитектура – архитектура, в которой есть Central Process Unit – вычислительный узел, который выполняет любое действие, которое нам нужно, который взаимодействует с памятью, сама память команд и память данных физически разделена.

Т.е. Гарвардская архитектура решает проблему узкого места фон Неймановской (принстонской) архитектуры - совместной памяти (доступ к инструкциям и данным, по очереди по одному каналу). Память была физически разделена на отдельную память команд и данных - каналы передачи инструкций и данных физически разделены.

Достоинства:

1. Два физических канала между процессором и памятью (в теории производительнее (два канала), безопаснее)
2. Одновременный доступ к памяти (быстрота)
3. Разная ширина машинного слова (фрагмент данных, обрабатываемый как единое целое процессором, минимальная единица данных, с которой оперирует процессор. Размер машинного слова определяет максимальное количество информации, которое процессор может обрабатывать за одну операцию) и адреса для данных и программ
4. Оптимизация под решаемую задачу (так как обособленность программ от данных)
5. Изоляция памяти инструкций (безопасность доступа, нельзя подсунуть данные в виде инструкций)

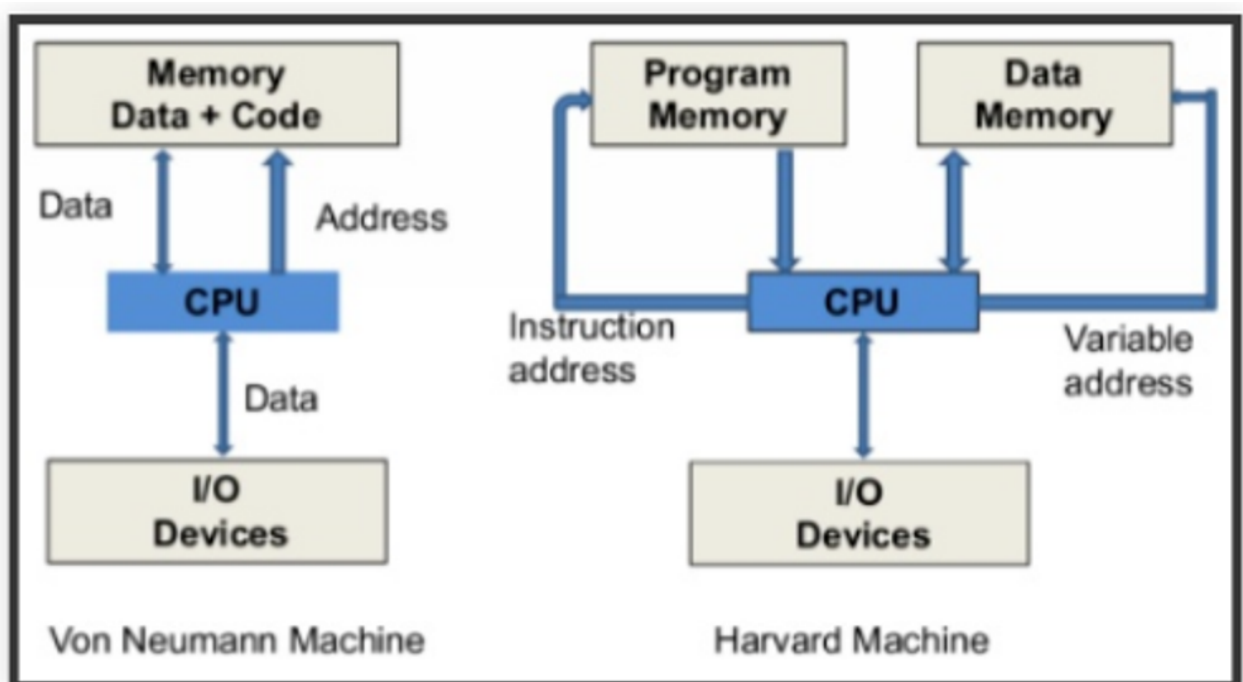
Недостатки:

1. Сложность и стоимость реализации

2. Внутренние проблемы данной архитектуры, изоляция инструкций и данных (нужно, чтобы инструкции исполнялись как данные и в обратную сторону):
 1. Запуск результата компиляции
 2. Указатели на функции

Подходы к решению проблем (вариации):

1. Архитектура “Память инструкций как данные” - реализуется возможность читать и писать данные в память программ. Позволяет генерировать и запускать машинный код. *(набор инструкций для чтения и записи инструкций в память)*
2. Архитектура “Память данных как инструкции” - реализуется возможность запуска инструкции из памяти данных. Позволяет генерировать и запускать машинный код, но параллельный доступ ограничен. *(запись и чтение данных в инструкции, убивает параллельный доступ схлопнули в один канал)*
3. Модифицированная Гарвардская архитектура - доступ к памяти реализуется через независимые кешей для данных и программ, за счет чего, с точки зрения внутренней организации процессора, доступ реализован независимо, при это канал между процессором и памятью один. *(использование кэшей данных и программ для доступа к памяти, использует физически 1 канал, но с внутренней организации процессора доступ независимый)*



Особенности и ограничения:

- Разрядность процессора: 8 бит.
- Организация памяти: гарвардская, с отдельными блоками памяти для команд и данных.
- Внешние устройства отображаются в адресное пространство данных. Работа с ними выполняется по запросу.
- Регистры:
 - PC -- счетчик команд
 - IR -- регистр инструкций

- AR -- регистр адреса операнда
- C -- флаг переноса/займа
- Z -- флаг нуля

● Команды выполняются за 2 или 3 такта (в зависимости от типа команды):

- 1)Выборка команды
- 2)Выборка операндов
- 3)Выполнение команды.

● Подсистема обработки прерываний и команды вызова подпрограмм отсутствует

Control Unit - это блок управления, основной задачей которого является управления процессором. Преобразует закодированные **инструкции в сигналы** синхронизации и управления

Реализации:

- *Hardwire* (декодирование инструкций в последовательные сигналы при помощи **аппаратных схем**, простота за счет системы просто конечных автоматов, но растет по сложности от системы команд)
- *Microwired* (декодирование при помощи **микропрограммы** (программа, реализующая набор инструкций процессора, делает из инструкций в памяти микрооперации (управляющие сигналы) и подает их на Control Unit) с необходимыми функциями). По сути, **битовая маска. При этом для каждой инструкции - свой набор микрокоманд**
 - Просто в реализации, изменение системы команд на лету, влияние на эффективность через адаптирование системы команд
 - Микрокод в процессоре (занимает место), сложно в изучении, разнообразие архитектур (поэтому много проблем с адаптацией к каждому отдельному), разнообразие команд (сложно оптимизировать, инструментарий), все проблемы программирование (отладка)
- **hardwired** - есть куча аппаратной логики, которая получает operation code на вход, считает сколько тактов понадобится на выполнение операции, отправляет управляющие сигналы как цифровая схема, сводится к небольшой системе конечных автоматов, но есть большой недостаток, так как при росте сложности системы команд конечный автомат также очень быстро растет по сложности и появляется большой шанс накосячить
- **microcoded** - парсим нашу команду на множество микро-команд, которые тоже являются набором управляющих сигналов, но так как мы читаем эту инфу из данных, а не реализуем эту логику аппаратно, мы получаем свободу модификации и право на ошибку

Datapath - набор компонентов, обеспечивающих передачу и управление данных между компонентами процессора (**регистры, шины, алу**) (эту вещь еще называют трактом данных)

Принципы работы с Datapath:

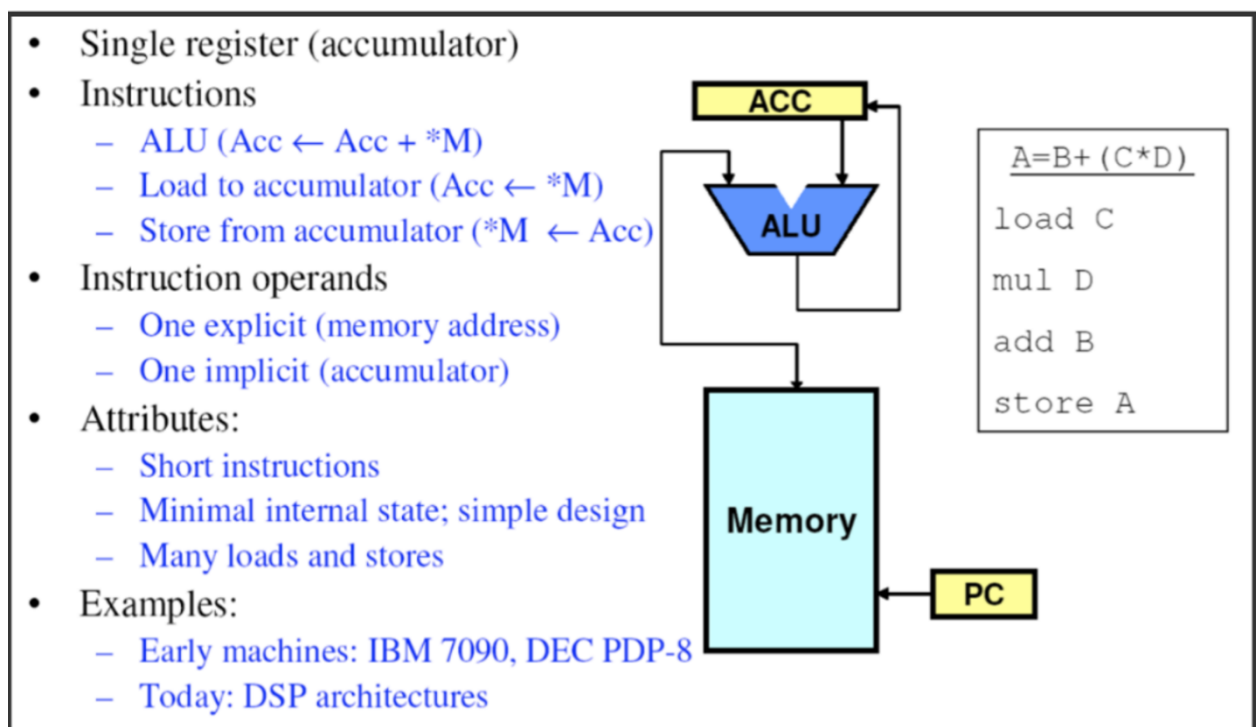
Accumulator (центральный регистр, работа с которым построена на том, что мы все время туда что-то добавляем, вычитаем, сбрасываем и все вокруг вокруг одной ячейки, простое решение, но не очень эффективно, становится узким

местом), *Register-to-Register* (работа выполняется на конечном множестве регистров, допустим 5 аккумуляторов, и есть буферы промежуточных значений, штуки 3, которые позволяют оптимизировать вычисления), *Register-to-Memory* (использует одну ячейку памяти, один регистр и результат можно сохранить в памяти), *Memory-to-Memory*, *Stack Architectures*

Аккумуляторная архитектура (используют накопитель, через который происходит преобразование накопленных значений в цикле, без сохранения и загрузки)

- Единый регистр (накопитель)
- Инструкции
 - $ALU (Acc \leftarrow Acc + *M)$
 - Загрузка в накопитель ($Acc \leftarrow *M$)
 - Сохранение из накопителя ($*M \leftarrow Acc$)
- Операнды команд
 - Один явный (адрес памяти)
 - Один неявный (аккумулятор)
- Атрибуты:
 - Короткие инструкции
 - Минимальное внутреннее состояние; простая конструкция
 - Множество загрузок и хранилищ

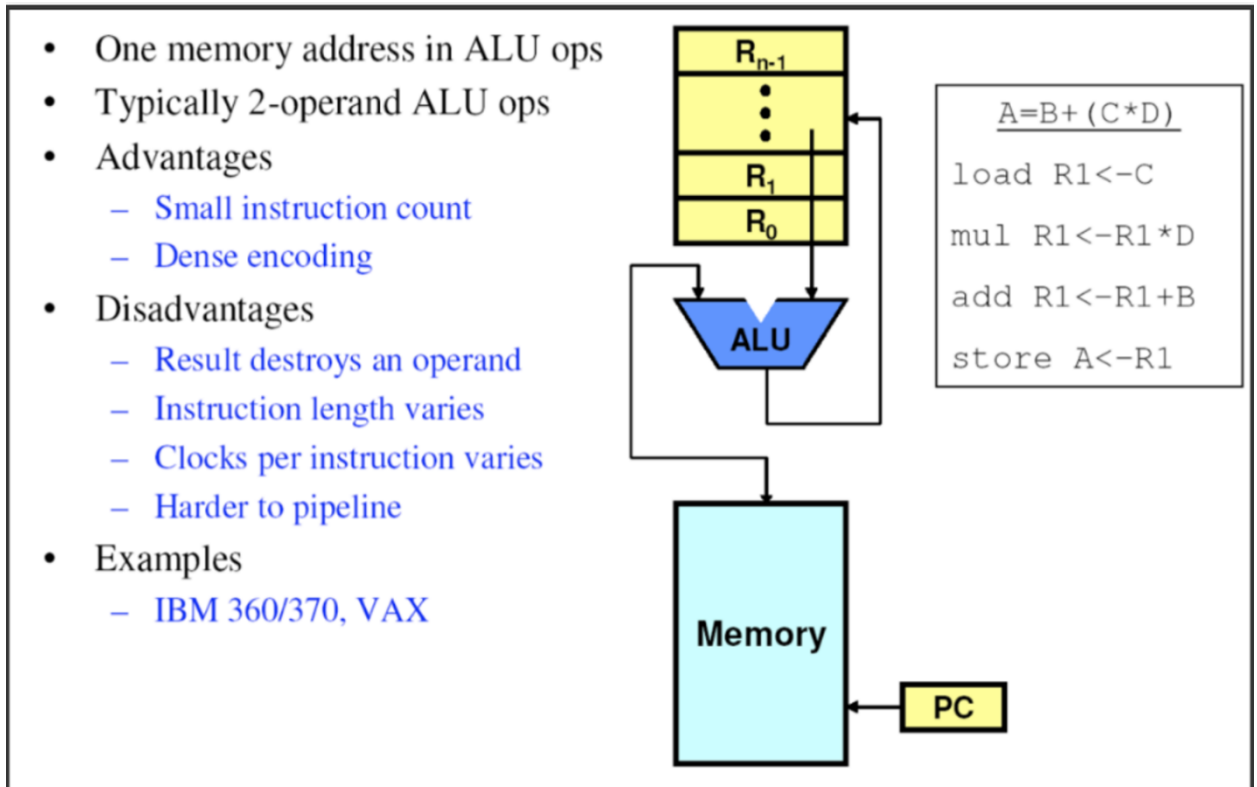
Простая реализация, проблема, если надо данные постоянно сохранять в память как промежуточные значения



Register-to-Memory

- Регистр ту память (регистр лучше аккумулятора - не надо постоянно состояние грузить куда-то)
- (команды сразу задействуют память и регистр, все действия происходят через них, возможность сохранения промежуточных данных при вычислениях в другой регистр)

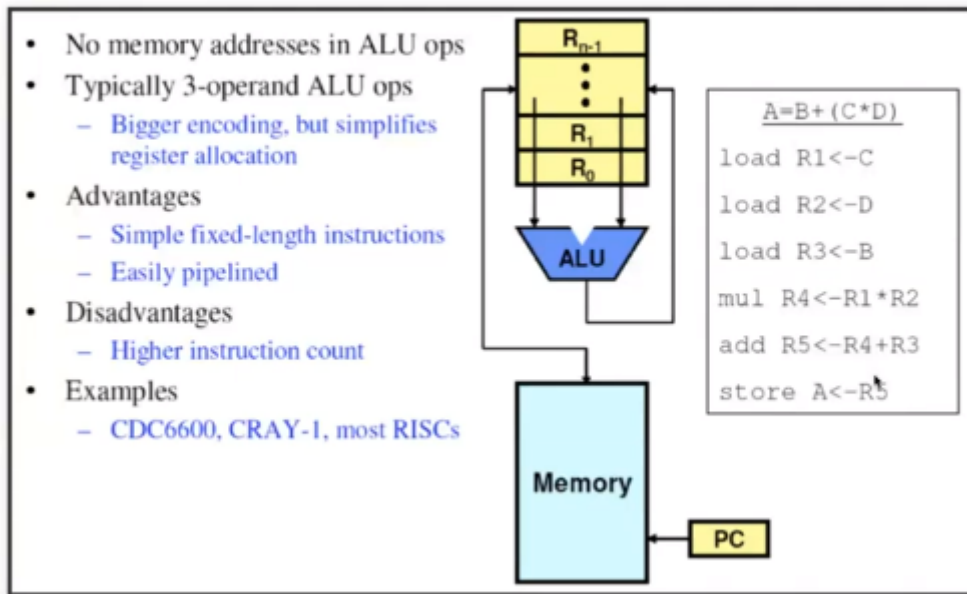
- Один адрес памяти в ALU-операциях
- Обычно ALU-операции состоят из 2 операндов
- Преимущества
 - о - Малое количество команд
 - о - *Плотное кодирование*
- Недостатки
 - о Результат приводит к уничтожению операнда
 - о Длина команды меняется
 - о Время, затрачиваемое на каждую команду, меняется
 - о Сложнее конвейеризовать



Register-to-Register (RISC)

- В ALU-операциях нет адресов памяти
- Обычно ALU-операции состоят из 3 операндов
 - о Более сложная кодировка, но упрощает распределение регистров
- Преимущества
 - о Простые инструкции фиксированной длины
 - о Легко конвейеризуемые
- Недостатки
 - о Большое количество инструкций

Register-to-Register: Load-Store Architectures



Memory-to-Memory (CISC)

- Память ту память (если память быстрая)
- (чтение и запись из только памяти, нет возни с регистрами, так как автоматическая работа с ними)
- Все операнды ALU, адресованные из памяти
- Преимущества
 - Нет потерь регистров
 - Минимальное количество команд
- Недостатки
 - Большие различия в длине команд
 - Большие различия в тактах для каждой команды
 - Огромный объем памяти

- All ALU operands from memory addresses

- Advantages

- No register wastage
- Lowest instruction count

- Disadvantages

- Large variation in instruction length
- Large variation in clocks per instructions
- Huge memory traffic

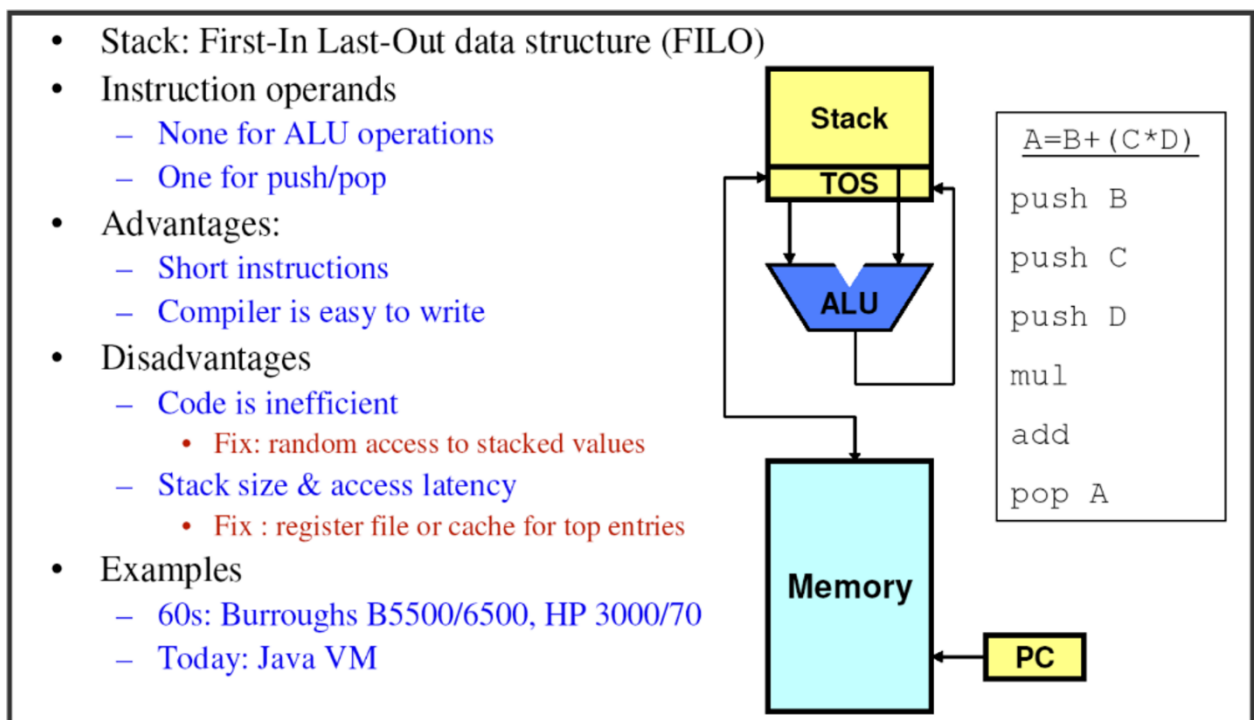
- Examples

- VAX

$D = B + (C * D)$
 mul D <- C*D
 add D <- D+B

- Стековая

- (работа происходит через стек, легко писать компиляторы, так как нет возни с регистрами и адресациями, но не самый эффективный код и проблема размера со стеком, хотя они уже решены, пример - **Java VM**)
- Стек: Структура данных "Первый вошел - последний вышел" (FILO)
- Команды-операнды
- Нет для операций ALU
- Преимущества:
 - Один для push / pop
 - Короткие инструкции
 - Компилятор прост в написании
- Недостатки
 - - Код неэффективен
 - Fix: произвольный доступ к накопленным значениям
 - - Размер стека и задержка доступа
 - Fix: регистрировался файл или кэш для самых популярных записей



Архитектура широко применяется во внутренней структуре современных высокопроизводительных микропроцессоров, где используется отдельная кэш-память для хранения команд и данных.

Гарвардская архитектура используется в ПЛК и микроконтроллерах, таких, как Microchip PIC, Atmel AVR, Intel 4004, Intel 8051, процессорах серии TMC320 фирмы Texas Instruments, DSP фирм Motorola и Analog Devices, а также в кэш-памяти первого уровня x86-микропроцессоров, делящейся на два равных либо различных по объёму блока для данных и команд.

№12 Механизм микроопераций, микропрограммирование и его роль в развитии компьютерных систем. Особенности и ограничения. Применение на практике. Пример оптимизации через микрокод. NISC архитектура.

Механизм микроопераций в свое время являлся прорывным решением в создании компьютеров с общей архитектурой. Такую идею изначально предложили IBM в 50е годы: были как дорогие аппаратные машины, в которых выполнялись большие наборы сложных инструкций, так и дешевые, в которых был другой, облегченный набор команд. Было предложено решение реализовывать комплексный набор команд (CISC) микропрограммным путем: в устройстве управления хранится специальная память микрокоманд - это микрооперации, которые выставляют управляющие сигналы процессору. При поступлении команды уровня ISA в процессор внутри устройства управления, в памяти микрокоманд, ищется блок памяти, в котором хранится интерпретатор соответствующей команды.

Механизм микроопераций - преобразование операций в управляющие сигналы посредством использования микропрограмм (программа, реализующая набор инструкций процессора).

Механизм микроопераций - это метод управления операциями в процессоре, при котором каждая инструкция разбивается на более мелкие операции, называемые микрооперациями.

Микропрограммирование - это способ управления выполнением микроопераций путем использования микропрограммы, которая представляет собой последовательность микроопераций для выполнения каждой инструкции.

Роль в развитии компьютерных систем:

Микропрограммирование позволяет создавать более гибкие и модульные процессоры, так как изменение микропрограммы позволяет изменить поведение процессора без изменения аппаратной части. Это упрощает разработку новых процессоров и обновление существующих.

Механизм - делает из инструкций в памяти микрооперации и подает их Control Unit:

- *Выделяем сигналы управления (защёлкивание регистров, чтение/запись в память, селекторы мультиплексоров - устройство, имеющие множество входов и единственный выход. Позволяет передавать сигнал с одного из входов на выход в зависимости от комбинации управляющих сигналов. Позволяет сократить число сигнальных линий и обеспечить сохранение качества без потери сигнала. Демультимплексор работает ровно наоборот).*
- *Создаём память микрокоманд. Ширина слова соответствует количеству сигналов.*
- *Определяем микрокоманду чтения инструкции (00-01).*
- *Создаём декодер кода операции в адрес микрокоманд.*
- *Реализуем операции в рамках микрокоманд (n1-n4), завершая их micro_jmp 00 (возвращаемся к чтению инструкции).*

	00	01	...	n1	n2	n3	n4
latch_PC	1	0	...	1	1	1	0
mux_PC	+1	0	...	+1	+1	+1	0
latch_IR	1	0	...	0	0	0	0
latch_AR	0	0	...	0	1	1	0
MUX2	0	0	...	PM	0	DM	0
ALU	0	0	...	+0	0	+A	0
OE	0	0	...	0	0	1	0
WR	0	0	...	0	0	0	1
latch_mPC	1	1	...	1	1	1	1
mux_mPC	+1	DC	...	+1	+1	+1	=0
		+--	mpc	←	decoder(IR)		

Первый столбец - выделенные сигналы управления, следующие столбцы - конкретные управляющие сигналы к компонентам

Преимущества:

- Простота реализации - можно реализовать сложный набор инструкций
- Возможность “программирования” системы команд - за счет микроуправления можно оптимизировать команды (экономить число тактов)
- Доступ к микрокоду для программиста - можно менять, понимать и писать оптимальный код
- Генерация ISA под задачу (сократить объём, повысить эффективность)

Недостатки:

- Хранение микрокода в процессоре - нужна площадь на плате под память и размещение памяти на процессоре - дорого
- Усложнение разработки и оптимизации процессора - т.к. теперь тут решает микрокод (Разнообразие команд (форматы, размеры, длительности, доступ))
- Приносит больше проблем с программированием - начинается отладка, появляются проблемы со связыванием между аппаратурой и программной составляющих

Использование: реализации Control Unit:

- Hardwire (декодирование инструкций в последовательные сигналы при помощи аппаратных комбинационных схем, декодирующих инструкции в последовательности сигналов, простота за счет системы просто конечных автоматов, но растет по сложности от системы команд)
- Microcoded (декодирование при помощи **микропрограммы** (программа, реализующая набор инструкций процессора, делает из инструкций в памяти микрооперации (управляющие сигналы) и подает их на Control Unit) с необходимыми функциями). По сути, **битовая маска. При этом для каждой инструкции - свой набор микрокоманд**
 - Просто в реализации, изменение системы команд на лету, влияние на эффективность через адаптирование системы команд

- o Микрокод в процессоре (занимает место), сложно в изучении, разнообразие архитектур (поэтому много проблем с адаптацией к каждому отдельному), разнообразие команд (сложно оптимизировать, инструментарий), все проблемы программирование (отладка)
- o *Microcoded при помощи исполнения микропрограммы, реализующей необходимые функции.*
- o *Микропрограмма (микрокод) программа, реализующая набор инструкций процессора*

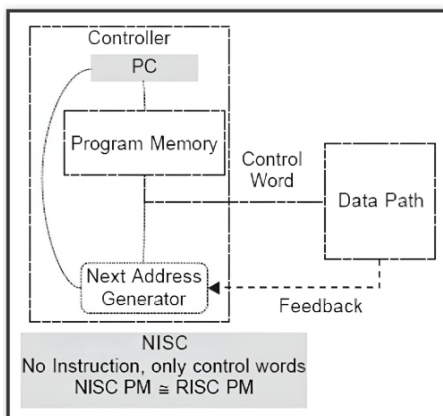
Оптимизация через микрокод - это процесс оптимизации работы процессора путем изменения или добавления микрокодовых инструкций. Микрокод представляет собой набор инструкций, которые используются для управления внутренними операциями процессора.

Пример оптимизации через микрокод может включать в себя добавление новой микрокодовой инструкции для улучшения производительности или исправления ошибки в работе процессора. Например, если обнаруживается, что определенная последовательность инструкций выполняется неэффективно, разработчики могут создать новую микрокодовую инструкцию, которая будет выполнять эту последовательность более эффективно.

Другой пример оптимизации через микрокод - это исправление ошибок без необходимости выпуска нового процессора. Если обнаруживается ошибка в работе определенной инструкции, разработчики могут создать новый микрокод для исправления этой ошибки и распространить его в виде обновления микрокода (**microcode update**).

NISC - компьютерная архитектура для специализированных процессоров, дает прямой контроль компилятору над аппаратными ресурсами, имеет узкое использование (спец. вычислители, ускорители (отказ от системы команд и инструкций, использование напрямую системы микрокоманд))

T.e. No Instruction Set Computer - архитектура, в которой команды процессора - это микрооперации



Достоинства NISC:

- Упрощение аппаратуры - не нужно использовать Control Unit (убрать интерпретаторы инструкций)
- Максимальная эффективность программного управления - компилятор может заранее все просчитать (нет системы команд - нет ее проектирования и архитектуры)
- Нет архитектуры системы команд (ISA) - нет проблем проектирования

Недостатки:

- Невозможность бинарной совместимости - сильная привязка к конкретной архитектуре процессора (поменяли архитектуру - будь добр компилируй заново) (Сложно организовать совместимость для процессоров общего назначения)
- Низкая плотность машинного кода (много нулей)

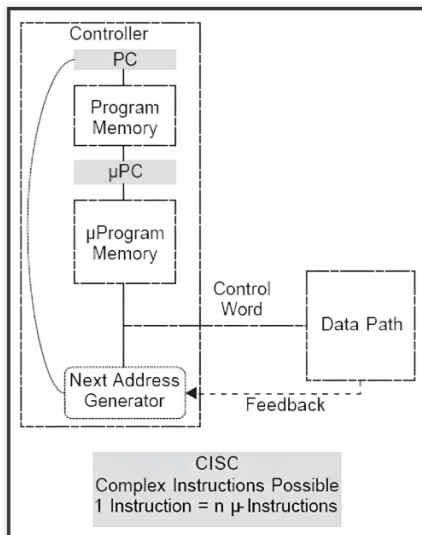
Использование. Применяется в ускорителях, в высокоуровневом синтезе (HLS), спец. вычислителях. *Проект NITTA (проект Пенского) — CGRA процессор, где вычислительные блоки управляются в стиле NISC.*

№13 Что такое CISC? Роль в развитии компьютерных систем. Применение на практике. Достоинства и недостатки. Отличия от архитектуры фон Неймана. Особенности программирования.

CISC - компьютерная архитектура, в которой отдельная инструкция способна выполнять несколько операций низкого уровня или многошаговые операции (будь то загрузка из/в память, арифм. операции и т.д.). Больше и разнообразнее система команд

В отличие от RISC (Reduced Instruction Set Computer), где используется набор более простых инструкций, CISC процессоры могут выполнять сложные операции с помощью одной инструкции.

Complex Instruction Set Computer - процессор умеет все, что он мог бы сделать. Достигается это за счет сложного набора инструкций.



хочешь писать мало - пиши команды мощнее

Изначально в прошлом все большие компьютеры в плане обработки команд имели преимущественно аппаратную составляющую (были сложные схемы по обработке команд). Это, естественно, увеличило их стоимость. В этот период (в 50е годы) компания IBM задумалась, а почему бы не сделать так, чтобы сложные мейнфреймы и дешевые малые компьютеры были одной архитектуры. Из-за того, что сложную и дорогую комб. схему не запихнешь в малый компьютер,

придумали интерпретаторы (обработка команд процессоров сместилась в сторону микрокоманд и программной составляющей).

Еще причины появления:

- Низкоуровневые языки и неразвитость компиляторов - писали на ассемблере и нужна была спецификация языка, а также были трудности с компиляцией программ
- Разнообразие архитектур - не было стандартизации, было нормой, что писали под разные архитектуры
- Неразвитость компиляторов (было мало компиляторов, сбоили, были коммерческими)
- Удобство программирования
- Минимизация объема программ
- Минимизация накладных расходов

Проблемы:

- Сложная система команд (*много команд*)
- Сложное устройство процессора и Control Unit
- Сложно генерировать эффективный машинный код

Про реализации Control Unit:

- *Hardwire* (декодирование инструкций в последовательные сигналы при помощи аппаратных комбинационных схем, декодирующих инструкции в последовательности сигналов, простота за счет системы просто конечных автоматов, но растет по сложности от системы команд)
- *Microcoded* (декодирование при помощи **микропрограммы** (программа, реализующая набор инструкций процессора, делает из инструкций в памяти микрооперации (управляющие сигналы) и подает их на Control Unit) с необходимыми функциями). По сути, **битовая маска. При этом для каждой инструкции - свой набор микрокоманд**
 - o Просто в реализации, изменение системы команд на лету, влияние на эффективность через адаптирование системы команд
 - o Микрокод в процессоре (занимает место), сложно в изучении, разнообразие архитектур (поэтому много проблем с адаптацией к каждому отдельному), разнообразие команд (сложно оптимизировать, инструментарий), все проблемы программирование (отладка)
 - o *Microcoded* при помощи исполнения микропрограммы, реализующей необходимые функции.
 - o Микропрограмма (микрокод) программа, реализующая набор инструкций процессора

Насчет того, что исполнение сложных команд становится быстрее - можно поспорить, но, что точно можно сказать - устройство управления становится намного больше.

В плане особенностей программирования: плотность кода выше, но сложно держать в голове такой большой набор инструкций.

Влияние на:

Низкоуровневые языки - Это помогло программистам на ассемблере, так как они смогли писать более простые программы, ведь всегда найдется инструкция, которая выполняет то, что нужно.

CISC архитектура была широко использована в ранних компьютерах, так как позволяла выполнять сложные операции с помощью одной инструкции, что упрощало программирование и повышало производительность для некоторых типов задач (память была очень дорогая в те годы).

Характеризуется следующим набором свойств:

- нефиксированное значение длины команды;
- арифметические действия кодируются в одной команде;
- небольшое число регистров, каждый из которых выполняет строго определённую функцию.

- Асс. Через аккумулятор:
 - ариф. операции, ввод-вывод.
 - 1 оп. — явный, 2 — неявный.
- CISC. Сложные инструкции.
 - Переменная длина.
 - Спец. регистры.
 - Ариф. операции и доступ(-ы) в память за одну инструкцию.
- RISC. Простые инструкции.
 - Единый размер инструкций.
 - Операции и работа с памятью — разные инструкции.
 - Регистры — одинаковые.
- Stack. Через стек:
 - Ариф. операции, ввод-вывод.
 - Операнды — неявные (часто).

CISC можно реализовать:

- Аппаратно
- Микропрограммно

Описание:

Архитектура CISC в значительной степени стала возможна благодаря возможностям микропрограммного управления. Для каждой инструкции из набора создается подпрограмма, которая состоит из простых инструкций и хранится в специальной памяти внутри микропроцессора. Таким образом, процессор содержит небольшой набор простых инструкций. На их основе можно создать множество сложных инструкций из набора команд с помощью добавления подпрограмм в микрокод.

Микропрограмма позволяет относительно легко формировать большое количество команд со сложным поведением. Поднять уровень машинных инструкций ближе к уровню языков программирования высокого уровня. К примеру:

- относительно легко поддержать все интересующие варианты адресаций для всех типов команд;

- реализовать операции с произвольным набором аргументов, к примеру инструкцию для расчета многочленов в рамках одной инструкции;

- реализовать операции, работающие в потоковом режиме.

+:

- Позволяет сокращать количество инструкций кода на низкоуровневых языках
- Унификация системы команд для разных архитектур
- Незрелость компиляторов - необходимость писать на низком уровне
- Высокая производительность (т.к. низкоуровневые языки)

-:

- Сложная система команд (использовать и анализировать), долгое исполнение инструкции. Такие команды можно заменить группой
- Усложнение устройства процессора и Control Unit (из-за числа команд)
- Из-за навороченности инструкций в командах и сложности написания компиляторов под систему команд каждого процессора сложно генерировать эффективный машинный код
- сложность растет комбинаторно

Отличия от архитектуры = подвох. Архитектура фон Неймана, описывает структуру компьютера. CISC (Complex Instruction Set Computing) — это тип процессорной архитектуры, а не общая компьютерная архитектура, как фон Нейман. *Архитектура фон Неймана (или принцип фон Неймана) определяет основные принципы построения компьютеров, включая хранение программ и данных в одной памяти. CISC и RISC относятся к типам архитектур процессоров, определяющим набор инструкций и способы их выполнения.*

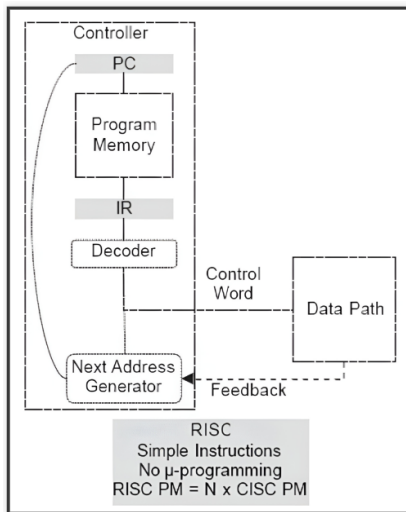
Применение: В рабочих станциях, серверах среднего звена и персональных компьютерах используются процессоры с CISC.

№14 Что такое RISC? Роль в развитии компьютерных систем. Применение на практике. Достоинства и недостатки. Отличия от архитектуры фон Неймана. Особенности программирования.

RISC появился после CISC. Разработчики тогда поняли, что эффективность достигается не только за счет комплексного выполнения инструкций, но и за счет того, сколько инструкций выполняет процессор за единицу времени. В этом плане RISC изначально проигрывал CISC, т.к. основная память была медленнее памяти микрокоманд. Но после того момента, как этот нюанс был исправлен, начали посматривать на RISC. Его создатели пересмотрели то, как нужно проектировать процессора, системы команд. Такой подход позволял сделать архитектуру процессора упрощенной.

RISC - архитектура с простым кодированием упрощенного набора команд, что увеличивает быстродействие.

В отличие от RISC (Reduced Instruction Set Computer), где используется набор более простых инструкций, CISC процессоры могут выполнять сложные операции с помощью одной инструкции.



RISC: упростим процессор за счёт языков высокого уровня!

Обрубим все до минимума и оптимизируем его, разложим по полочкам.

Предпосылки

- Сложные операции (встречаются редко, заменимы группами команд и процедурами)
- Появились ЯП высокого уровня (компилятор сам оптимизирует) - нам без разницы, какой ассемблер на низком уровне, когда пишем на высоком уровне
- Одинаковая длина инструкции, простота декодера
- Место памяти микрокоманд и декодера можно использовать для регистров и кеша - выделение места на плате - дорогое дело
- Легче оптимизировать меньшее число команд
- Параллелизм уровня инструкций (разбиваем обработку инструкции на несколько этапов и выполняем параллельно для разных команд) **Pipeline**
- Простой машинный код
- Ускорение программы за счет простых частых инструкций

Роль **RISC** в развитии компьютерных систем:

1. **Увеличение производительности:** За счет использования простых инструкций RISC-процессоры способны выполнять инструкции быстрее, чем процессоры с более сложными инструкциями.
2. **Уменьшение стоимости** и упрощение конструкции процессора: Простота инструкций позволяет создавать более компактные и эффективные процессоры.

Разбиваем обработку инструкции на несколько этапов и выполняем их параллельно для разных команд.

Один такт — одна стадия конвейера.

Недостатки **RISC**:

1. Увеличенное количество инструкций для выполнения определенных операций
2. Ограниченная гибкость: Некоторые задачи могут потребовать большего количества инструкций, чем предусмотрено в RISC-архитектуре.

Как построить конвейер?

1. выделить стадии выполнения команд;
2. организовать внутренние структуры процессора так, чтобы:
 - у процессора был входной (поступают команды) и выходной конец (команды "покидают" процессор);
 - структура процессора должна соответствовать стадиям выполнения команд;
 - сегменты связаны регистрами, комбинационные схемы сбалансированы;
 - все части процессора управляются одним тактовым сигналом;
3. загружать в процессор команды каждый такт;
4. получать результаты выполнения команд каждый такт;
5. разрешать конфликты параллельно выполняемых команд.

Типовые стадии конвейера

1. **Instruction Fetch.** Чтение инструкции по счётчику команд.
2. **Instruction Decode.** Декодировать инструкцию и считать регистры.
3. **Instruction Execute.** Операций изменения данных.
4. **Memory Access.** Чтение/запись операндов из памяти/в память.
5. **Write Back.** Запись результата в регистры.

Операции:

- *Register-Register (Single-cycle latency):* Сложение, вычитание, сравнение и логические операции.
- *Memory Reference (Two-cycle latency):* Подготовка адресов для доступа к памяти.
- *Multi-cycle (Many cycle latency):* Целочисленное умножение, деление и все операции с плавающей запятой.

Программирование для RISC процессоров включает следующие особенности:

- **Инструкции фиксированной длины:** Обычно инструкции имеют одинаковую длину, что упрощает декодирование и исполнение.
- **Простые и однородные инструкции:** Программистам приходится использовать более простые операции, что может увеличить объем исходного кода.
- **Необходимость оптимизации:** Для достижения максимальной производительности требуется грамотное использование регистров и инструкций.
- **много регистров, они одинаковые с точки зрения процессора**

- Асс. Через аккумулятор:
 - ариф. операции, ввод-вывод.
 - 1 оп. — явный, 2 — неявный.
- CISC. Сложные инструкции.
 - Переменная длина.
 - Спец. регистры.
 - Ариф. операции и доступ(-ы) в память за одну инструкцию.
- RISC. Простые инструкции.
 - Единый размер инструкций.
 - Операции и работа с памятью — разные инструкции.
 - Регистры — одинаковые.
- Stack. Через стек:
 - Ариф. операции, ввод-вывод.
 - Операнды — неявные (часто).

Применение на практике (*почему RISC не вытеснил CISC*):

- Мобильные - уже. ПК и сервера - в процессе
- Инструментальная поддержка - много, чего написано для RISC
- Бинарная совместимость (виртуализация, аппаратная трансляция, транспилиция) - если даже что-то специально не написано, существует немало софта, который транслирует код другой архитектуры на нужную (например, Rosetta)
- Зависимость себестоимости от серийности - чем больше, тем дешевле (пока не про RISC)
- RISC ядро внутри CISC

Отличия от архитектуры = подвох. Архитектура фон Неймана, описывает структуру компьютера. RISC — это тип процессорной архитектуры, а не общая компьютерная архитектура, как фон Нейман. *Архитектура фон Неймана (или принцип фон Неймана) определяет основные принципы построения компьютеров, включая хранение программ и данных в одной памяти. CISC и RISC относятся к типам архитектур процессоров, определяющим набор инструкций и способы их выполнения.*

№15 Конвейеризированное исполнение команд. Стадии конвейера. Виды конфликтов (по данным, по управлению), их примеры и влияние на производительность. Достоинства и недостатки.

Конвейеризированное исполнение команд - это параллелизм уровня команд, в котором за один такт выполняется одна стадия конвейера.

Используется в RISC, основано на разбивке обработки инструкции на несколько этапов и выполнении параллельно для разных команд, 1 такт - 1 стадия конвейера

Для создания конвейера требуется:

- выделить стадии выполнения команд (нужно, чтобы каждые стадии конвейера были по времени эквивалентны)
- организовать внутренние структуры процессора так, чтобы
 - у процессора был входной (поступают команды) и выходной конец (команды "покидают" процессор);
 - структура процессора должна соответствовать стадиям выполнения команд;
 - сегменты связаны регистрами (через них передача данных), комбинационные схемы сбалансированы;
 - все части процессора управляются одним тактовым сигналом;
- загружать в процессор команды каждый такт;
- получать результаты выполнения команд каждый такт;
- разрешать конфликты параллельно выполняемых команд (возможность взаимодействия инструкций, обращения к одному участку памяти).

Типовые стадии:

- Instruction Fetch. Чтение инструкции по счётчику команд и помещаем в регистр для дальнейшей обработки.
- Instruction Decode. Декодировать инструкцию и считать регистры (комбинационная схема, которая анализирует этот декодер и понимает какие управляющие сигналы дальше отправлять).
- Instruction Execute. Операций изменения данных (алу).
- Memory Access. Чтение/запись операндов из памяти/в память.
- Write Back. Запись результата в регистры.

Операции:

- *Register-Register (Single-cycle latency): Сложение, вычитание, сравнение и логические операции.*
- *Memory Reference (Two-cycle latency): Подготовка адресов для доступа к памяти.*
- *Multi-cycle (Many cycle latency): Целочисленное умножение, деление и все операции с плавающей запятой.*

Проблемы:

- Структурные конфликты (Конфликт из-за ресурсов: аппаратура не позволяет выполнить все возможные комбинации инструкций (например, одновременный доступ к памяти, доступ к одному и тому же ресурсу памяти в 1 такт). Решение - гарвардская архитектура, двухпортовая память, использование пустых инструкций для конфликтных мест (но снижает эффективность), Пузырек - когда понимаем, что текущая операция не может быть выполнена - мы пускаем по нескольким стадиями конвейера заглушку, вы которой процессор простаивает (конкретно на следующих командах, предыдущие выполняются)
- Конфликты по данным:
 - RAW (Read after Write) - возникает тогда, когда мы записываем значение в регистр и сразу хотим прочитать его (не успеет обновиться регистр), запаздывание по записи результата относительно его выборки → **чтение старых данных.**
 - WAR (Write after Read) - возникает тогда, когда мы сначала используем значение, а потом его меняем (потенциально проблема может возникнуть в случае перестановки

инструкций - реордерингов), чтение данных для операции уже после перезаписи (приколы компилятора с перемешиванием данных) → неверная арифм. операция

- WAW (Write after Write) - возникает тогда, когда мы подряд перезаписываем значение (например, если значение используется для ввода/вывода, то нам принципиален порядок), 2 записи подряд в одно место → смешение их при выполнении
- RAR (Read after Read - не проблема) - 2 чтения подряд из 1 регистра

Решение: исполнение не по порядку (компилятор сует между смежными операциями другие), переименование регистров на уровне компилятора (если зависимость по данным ложная. Запись может быть переназначена на другой регистр (пример WAW)), пузырьки (он отличается тем, что вставляется в ходе выполнения команды, а не вместо нее), проброс операндов (bypassing, operand forwarding) между стадиями процессора, минуя регистровый файл - можно прокинуть результаты выполнения одной команды сразу же как входные данные для другой (иногда используется пузырек) (проброс результата на нужную стадию, только вперед, поэтому либо напрямую, либо используя пузырек)

- Проблема - в конвейер может быть загружена команда, не должная к исполнению (происходит из-за операций условного и безусловного перехода) → **конфликт по управлению** так как может быть загружена операций, не должная к исполнению и их нужно **откатить, решения:**
 - o Пузырек - видим команду перехода - кидаем пузырек до тех пор, пока не выполнится эта команда
 - o Сброс конвейера - нужно сделать так, чтобы загруженные команды, которые пропускаются, были сброшены и не повлияли на наши дальнейшие вычисления
 - o Минимизаций условных переходов - с точки зрения работы компилятора и программ:
 - loop unrolling - удаление или изменение циклов таким образом, чтобы уменьшить их или число их итераций. Увеличивает скорость за счет удаление инструкций управления циклом и тестирования (убираем часть итераций за счет увеличения тела цикла)
 - Условное перемещение данных (такой подход используется при оптимизациях компилятора)
 - o Статистическое и динамическое предсказания переходов (branch prediction) - когда нужно будет грузим, иначе - не грузим (угадывание данных и их выборки). Пример - переход назад (из цикла) произойдет почти всегда.

Предсказание может быть двух видов: статическое и динамическое.

Статический подход основан на статистике (например, на виде переходов: вперед, назад и прочие).

Динамический подход основан на том, что мы смотрим, как работает наша программа. Много математики и статистики. Например, можно использовать счетчик с накоплением (2-bit) или многоуровневые предсказатели.

Преимущества:

- Повышение производительности и уровня утилизации ресурсов

Недостатки:

- Снижение скорости исполнения отдельной команды
- Не все операции за один машинный цикл (деление)
- Разрешение конфликтов
- Непредсказуемое время исполнения (из-за брэнч предикторов)
- Уязвимости “косвенных каналов” (Meltdown, Spectre) (из-за многостадийного выполнения можно вынуть данные из регистра как вариант) (*произошло обращение памяти, кеш прогрелся, мы получили из другой инструкции доступ к данным, к которым доступа иметь не должны были*)
- Как быть с виртуальными методами?

№16 Виды конфликтов при работе конвейера и механизмы их разрешения, сокращения их числа. Пузырёк, разворачивание циклов, предсказания переходов (статические и динамические).

Проблемы:

- Структурные конфликты (Конфликт из-за ресурсов: аппаратура не позволяет выполнить все возможные комбинации инструкций (например, одновременный доступ к памяти, *доступ к одному и тому же ресурсу памяти в 1 такт*). Решение - гарвардская архитектура, двухпортовая память, *использование пустых инструкций для конфликтных мест (но снижает эффективность)*, Пузырек - когда понимаем, что текущая операция не может быть выполнена - мы пускаем по нескольким стадиями конвейера заглушку, вы которой процессор простаивает (конкретно на следующих командах, предыдущие выполняются)
- Конфликты по данным:
 - RAW (Read after Write) - возникает тогда, когда мы записываем значение в регистр и сразу хотим прочитать его (не успеет обновиться регистр), запаздывание по записи результата относительно его выборки → **чтение старых данных.**
 - WAR (Write after Read) - возникает тогда, когда мы сначала используем значение, а потом его меняем (потенциально проблема может возникнуть в случае перестановки инструкций - реордерингов), чтение данных для операции уже после перезаписи (приколы компилятора с перемешиванием данных) → неверная арифм. операция
 - WAW (Write after Write) - возникает тогда, когда мы подряд перезаписываем значение (например, если значение используется для ввода/вывода, то нам принципиален порядок), 2 записи подряд в одно место → смешение их при выполнении
 - RAR (Read after Read - не проблема) - 2 чтения подряд из 1 регистра

Решение: исполнение не по порядку (компилятор суёт между смежными операциями другие), переименование регистров на уровне компилятора (если зависимость по данным ложная. Запись может быть переназначена на другой регистр (пример WAW)), пузырек (он отличается тем, что вставляется в ходе выполнения команды, а не вместо нее), проброс операндов (bypassing, operand forwarding) между стадиями процессора, минуя регистровый файл - можно прокинуть результаты выполнения одной команды сразу же как входные данные для другой (иногда используется пузырек) (проброс результата на нужную стадию, только вперед, поэтому либо напрямую, либо используя пузырек)

Bypassing + bubble - Слишком большое смещение между инструкцией которой нужны актуальные данные и инструкцией, которая этими данными обладает

- Проблема - в конвейер может быть загружена команда, не должная к исполнению (происходит из-за операций условного и безусловного перехода) → **конфликт по управлению** так как может быть загружена операций, не должная к исполнению и их нужно **откатить, решения:**
 - Пузырек - видим команду перехода - кидаем пузырек до тех пор, пока не выполнится эта команда, **(пустые операции 0, не НОП тк НОП все еще инструкция) низкая эффективность, занимает конвейер, не выполняет никаких действий**
 - Сброс конвейера - нужно сделать так, чтобы загруженные команды, которые пропускаются, были сброшены и не повлияли на наши дальнейшие вычисления
 - Минимизаций условных переходов - с точки зрения работы компилятора и программ:
 - loop unrolling - удаление или изменение циклов таким образом, чтобы уменьшить их или число их итераций. Увеличивает скорость за счет удаление инструкций управления циклом и тестирования (убираем часть итераций за счет увеличения тела цикла)

```
for (int x = 0; x < 100; x++) {
    delete(x);
}
```

↓

```
for (int x = 0; x < 100; x += 5 ) {
    delete(x);
    delete(x + 1);
    delete(x + 2);
    delete(x + 3);
    delete(x + 4);
}
```

- Условное перемещение данных (такой подход используется при оптимизациях компилятора)
- Статистическое и динамическое предсказания переходов (branch prediction) - когда нужно будет грузим, иначе - не грузим (угадывание данных и их выборки). Пример - переход назад (из цикла) произойдет почти всегда.

Предсказание может быть двух видов: статическое и динамическое.

- **Статистическое** - определяется инструкцией перехода и заложенными паттернами. Статический подход основан на статистике (например, на виде переходов: вперед, назад и прочие).
- **Динамическое** - по истории переходов программы. Механизм - по заданной семантике создаем структуру, в ходе работы программы накапливаем опыт переходов, и реализуем его для этих же инструкций в ходе выполнения. Динамический подход основан на том, что мы смотрим, как работает наша программа. Много математики и статистики. Например, можно использовать счетчик с накоплением (2-bit) или многоуровневые предсказатели.

табличка с переходами

Можно анализировать не 2 последних ифа, а например 10 по определенному ифу или по всем, таким образом можно повторять паттерны

Внутри процессора на аппаратном уровне формируется таблица, где есть *branch program counter* - адрес команды, внутри которой условный переход, *predicted* - куда пойдет переход.

Подход - сравнение с прошлым разом - если в прошлый раз перешли, то и в следующий раз перейдем.

Счетчик с накоплением 2 бита - значит нам нужно 2 раза подряд взять или не взять прыжок, чтобы повторять это

Накапливаем статистику

- **Многоуровневые предсказатели переходов** - из-за периодичности условного перехода (циклов) есть паттерн переходов. Эти паттерны в похожих условиях воспроизводятся. Логика - смотрим как переходили раньше, сохраняем этот паттерн в рамках предсказателя и воспроизводим.

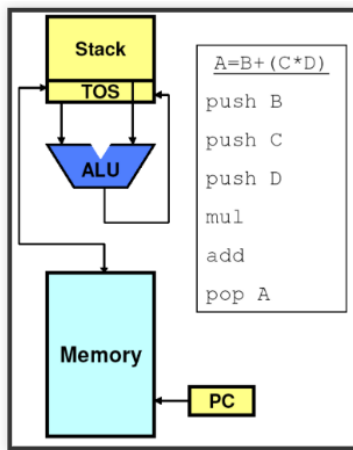
Конвейер:

- **Плюсы**
 - Общее повышение производительности
- **Недостатки**
 - Замедление исполнения отдельной команды
 - Не все операции выполняются за 1 машинный цикл
 - Конфликты (смотри выше)
 - Непредсказуемое время исполнения
 - Уязвимости косвенных каналов (из-за многостадийного выполнения можно вынуть данные из регистра как вариант)

HyperThreading - было бы классно взять два комплекта регистров и выполнить типа параллельно. Создает на одно физ ядро также виртуальное. Увеличивает количество независимых инструкций в пайплайне

№17 Что такое SOP (Stack-Oriented Processors, стековый процессор)? Роль в развитии компьютерных систем. Применение на практике. Достоинства и недостатки. Отличия от архитектуры фон Неймана. Особенности программирования.

ROSC - *Reduced Operands Set Computer* - стековый процессор



SOP (Stack-Oriented Processors) - это тип процессора, который использует стек для хранения и управления данными и инструкциями. В отличие от процессоров с регистровой архитектурой, где данные хранятся в регистрах, в стековом процессоре данные хранятся в стеке, который работает по принципу Last In First Out (LIFO).

SOP имеют свои преимущества в некоторых специализированных областях, таких как встраиваемые системы, где их простота и низкое энергопотребление могут быть выгодными. Собственно, используются во встраиваемых системах, микроконтроллерах, некоторых специализированных вычислительных устройствах. *Программирование для стековых процессоров требует использования специальных инструкций для работы со стеком, что может потребовать дополнительной тщательности и внимательности при написании кода.*

Основной идеей был отказ от регистров и выбор стека как основную структуру.

Обработка данных, ввод/вывод, условные/безусловные переходы производятся не через регистры, а стек.

Стековые процессора также могут классифицироваться по трем параметрам:

- Число стеков (одиночный/много)
- Размер стеков (малые/огромные)
- Число операндов у инструкций (0/1/2)

Стековый язык программирования - Forth - императивный язык программирования на основе стека, особенности которого:

- Структурное программирование
- Отражение (возможность исследовать и изменять структуру программы во время выполнения)
- Последовательное программирование
- Расширяемость (новые команды)
- Используется обратная польская нотация

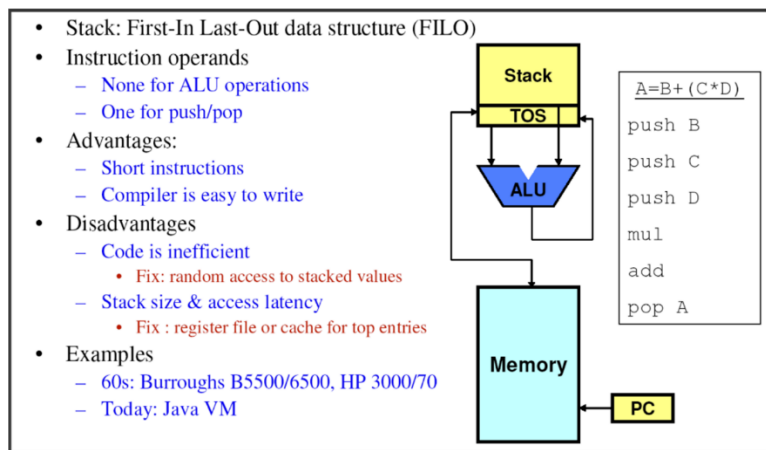
Достоинства:

- Нет понятия операнда (кроме взаимодействия со стеком), только алу и стек
- Обработка данных - на стеке
- Условные и безусловные переходы - через стек
- Обычно несколько стеков

- Достоинства:
 - Высокоуровневая архитектура (позволяет писать высокоуровневый код, поддержка процедур, автоматическая память, рекурсия, простой компилятор, сложные операции представляются польской нотацией)
 - Простая система команд
 - Высокая производительность (команды выполняются за малое кол-во тактов)
 - Кэш и потоки - проще реализация (так как обращения всегда происходят к вершине стека (про кеш), чтобы поменять контекст, нужно просто переключиться на другой стек, а не мучаться с сохранением и восстановлением данных в регистрах (про треды))
- Недостатки:
 - Низкая эффективность при большом размере стека (когда нужно взять несколько значений со стека для одной команды становится тяжело, так как обновлять данные в стеке очень дорого)
 - Динамические структуры не поддерживаются
 - Нет параллелизма инструкций (каждые инструкции завязаны друг на друге) - процессор рассчитан на то, что будет работа только с вершиной стека (ни о каком конвейере речь и не идет)
 - Фундаментально отличается от классических процессоров

Отличия от архитектуры = подвох. Архитектура фон Неймана, описывает структуру компьютера. Стекковый процессор — это тип процессорной архитектуры, а не общая компьютерная архитектура, как фон Нейман. *Архитектура фон Неймана (или принцип фон Неймана) определяет основные принципы построения компьютеров, включая хранение программ и данных в одной памяти. CISC, RISC, Stack относятся к типам архитектур процессоров, определяющим набор инструкций и способы их выполнения.*

- Асс. Через аккумулятор:
 - ариф. операции, ввод-вывод.
 - 1 оп. — явный, 2 — неявный.
- CISC. Сложные инструкции.
 - Переменная длина.
 - Спец. регистры.
 - Ариф. операции и доступ(-ы) в память за одну инструкцию.
- RISC. Простые инструкции.
 - Единый размер инструкций.
 - Операции и работа с памятью — разные инструкции.
 - Регистры — одинаковые.
- Stack. Через стек:
 - Ариф. операции, ввод-вывод.
 - Операнды — неявные (часто).



№18 Поддержка операций ввода-вывода в фон Неймановских процессорах. Поддержка на уровне системы команд (порт, отображение в память). Программно-управляемый ввод-вывод. Ввод-вывод через механизм прерываний. Механизм прямого доступа к памяти.

Устройства ввода-вывода - набор регистров (статуса и данных), через которые процессор наблюдает за этим устройством, также имеется протокол взаимодействия (определяющий, как взаимодействует процессор в этом устройстве).

Типовое взаимодействие по шагам:

- Конфигурация устройства (скорость передачи, настройки протокола)
- Проверка состояния (конфигурация, ее функционирование, *таймер*)
- Отправка данных
- Проверка статуса ответа
- Чтение ответа

Варианты ввода-вывода:

- Программно-управляемый (реализует процессор, все через его инструкции)
 - Проблемы:
 - Занимает процессор, включая имитацию таймера
 - Процессор (алгоритм) должен регистрировать сигнал на частоте в два раза выше частоты сигнала (теорема Котельникова)
 - Высокое энергопотребление
 - Проблемы работы одновременно с несколькими устройствами (например, можно было решить через конечные автоматы)
 - Проблемы в случае сложных протоколов ввода/вывода
- По прерыванию (использовать прерывание или иное внешнее событие, чтобы переключиться на ввод-вывод), снимает задачу наблюдения и позволяет это реализовать по внешнему событию
- Channel I/O и прямой доступ к памяти (Direct Memory Access - DMA) - процессор оповещается об операции завершения ввода-вывода (*добавить спец. проц, который сам занимается*)

С точки зрения системы команд (ISA):

- Через порты (PMIO) (Port-Mapped) - специальные инструкции для IO (in, out). Отдельное адресное пространство устройств ввода-вывода, адресация регистров не связана с адресацией памяти.

Преимущества:

- o Минимизация логики управления (малое адресное пространство). Оптимизация IO - ввод/вывод и работа с памятью физически разделены
- o Ввод-вывод и доступ к памяти разделены (разные каналы → параллельность)
- o Однородность пространства памяти (*общей*, нет разделения на устройства ввода-вывода и память), *в памяти только память и ничего стороннего (полностью ожидаемое поведение)*
- o Простота в целом (отдельный control unit и datapath для внешних устройств, что облегчает модификацию)

Недостатки:

- o Усложнение системы команд и процессора (контроллер ввода-вывода, новые инструкции) (так как теперь данные можно брать не только из памяти команды усложняются)
- o Операции ввода-вывода уступают прочим операциям с доступом в память (все сводится только к простым операциям с внешними устройствами, чтобы не усложнять систему команд) (*- данные второго сорта (особенно для CISC процессоров)*) - это касается усложнения системы команд: у нас появляется еще одно адресное пространство, с которым надо как-то взаимодействовать (нужно ли усложнять операции ввода/вывода?)
- o Лишнее копирование (регистры → память → регистры) (сначала читаем из порта, потом записываем в память) - как в RISC: сначала считали (скопировали откуда-то) и потом только работать
- Через память (MMIO) (Memory-Mapped) - внешние устройства (регистры) отображаются в основную память. (процессор думает, что работает с памятью). На самом деле эти ячейки памяти никак не задействованы - есть специальный декодер адреса, который определяет, куда/откуда пойдут данные. За счет этого нам не нужно усложнять ISA новыми инструкциями - нам достаточно работать с памятью. *Регистры внешних устройств отображаются в память команд, с которыми работает процессор (он не знает, что делает конкретно).* Ввод-вывод - через инструкции доступа к памяти.

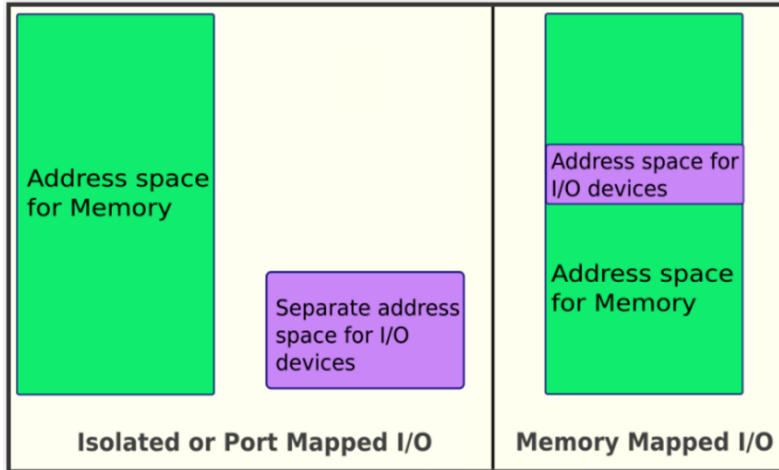
Достоинства:

- Простота процессора
- Единый набор механизмов доступа: автоинкремент, векторные операции, работа с барьерами
- Обработка данных без переноса в память - можем сразу получить данные
- Адресное пространство памяти огромно. Количество устройств не ограничено устройством процессора

Недостатки:

- Одна шина для ввода/вывода и доступа к памяти. Разница скорости шин
- Неоднородность памяти, сложная конфигурация системы (включая инструментарий) - в зависимости от случая будет разный отклик

- Устройства ввода/вывода работают с избыточным адресом - даже если одно устройство ввода и вывода, то все равно нужно пронумеровать порты и к ним по этим номерам обращаться
- Конфликты с кешами и параллелизмом уровня инструкций:
 - o flush при вводе/выводе
 - o порядок записи регистров



Программно-управляемый ввод-вывод - выполнение программы управления вводом-выводом:

- Занимает процессор, включая имитацию таймера
- Высокое энергопотребление
- Регистрация сигнала на частоте в 2 раза выше частоты сигнала (при работе на частоте процессора - регистрации не будет)
- Много ключей - проблемы
- Совмещение с другими задачами
- Сложные протоколы ввода-вывода - проблемы

Параллелизм ввода-вывода (ПРИМЕР КНОПКИ и печатные женщины)

- Усложнить цикл наблюдения (sleep + проверки)
- Прерывания
- Разорвать поток через конечный автомат (во время ожидания прохождения дребезгов возвращаться к наблюдению)

Теорема Котельникова: частота принимающего сигнал должна быть минимум вдвое больше отправителя

DMA

Процессор может работать на другой частоте,

+:

- скорость и эффективность повышаются

Проблемы синхронизации доступа к МЕМ:

- пакетный режим - приоритет DMA
- прозрачный режим - приоритет CPU
- циклический - приоритет меняется

IO Channel это старье, что вытеснили DMA

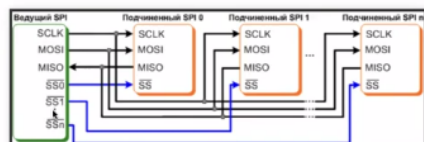
SPI (serial peripheral interface) - последовательный интерфейс передачи данных - обеспечивает синхронную передачу данных между микроконтроллера и устройства io

МОСИ - отдаем данные в устройство МИСО - получаем данные из устройства Сигнал клок- чтобы ведомый понимал когда вы шлете данные CS - выбор чипа или устройства

Работа с вводом-выводом при программном управлении реализуется следующим образом: процессор в цикле (polling) опрашивает состояние внешнего устройства и по наступлению необходимости начинает с ним взаимодействовать. К примеру наше устройство это Slave шины SPI реализованная через порты общего назначения (GPIO):

- регистрируем сигнал CS (Chip Select) который говорит о том что вот-вот начнётся передача данных, а значит опрос нужно начать производить чаще;
- регистрируем изменение сигнала SCLK (сигнал синхронизации), причем частота опроса должна быть как 2 раза выше частоты передачи данных (в теории по теореме Котельникова) или больше (на практике), но и не в порядке (регистрация электрического дребезга);
- и далее, в зависимости от настроек SPI, фиксируем биты на линию данных (Master Out Slave In, MOSI или Controller Out Peripheral In, COPI) в память побитно на положительный или отрицательный фронт SCLK, а также выставляем наши данные на линию (Master In Slave Out, MISO или Controller In Peripheral Out, CIPO);
- завершаем работу с интерфейсом по установлению сигнала CS.

Пример: Serial Peripheral Interface (SPI)



MOSI (Master Out Slave In) или **COPI** — выход ведущего, вход ведомого. Передача данных от ведущего устройства ведомому.
MISO (Master In Slave Out) или **CIPO** — вход ведущего, выход ведомого. Передача данных от ведомого устройства ведущему.
SCLK (Serial Clock) или **SCK** — последовательный тактовый сигнал. Передача тактового сигнала для ведомых устройств.
CS (Chip Select) или **SS** (Slave Select) — выбор микросхемы, выбор ведомого.

Программно-управляемый ввод-вывод. Проблемы:

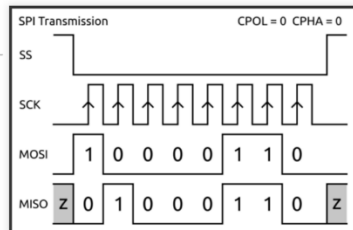
- Занимает процессор, включая имитацию таймера
- Высокое энергопотребление
- Регистрация сигнала на частоте в 2 раза выше частоты сигнала (при работе на частоте процессора - регистрации не будет)
- Много ключей - проблемы
- Совмещение с другими задачами
- Сложные протоколы ввода-вывода - проблемы

Ввод/вывод через механизм прерывания - это вещь, которая позволяет сильно повысить отклик системы на запросы от внешних устройств. Как правило, это реализуется при помощи дополнительного устройства - контроллера прерываний, который имеет свою внутреннюю логику, как реагировать на поступающие запросы на прерывание.

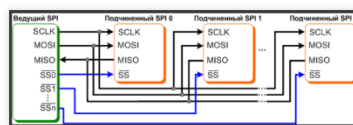
Система прерываний. Пример. SPI. Подходы

- Прерывание от CS
- Передача в блокирующем ISR

- Прерывание от CS
 - начало/завершение передачи
- Прерывание от SCLK
 - маска — CS = true
 - работа с буфером передачи



(длительность — произвольна)



Т.е. ввод-вывод по прерыванию снимает с процессора задачу постоянного наблюдения за внешним устройством и позволяет это реализовать по внешнему событию, к примеру готовность устройства к передаче данных или наступление заданного события (срабатывание таймера, нажатие на клавишу и т.п.). Непосредственно ввод-вывод реализуется по-прежнему процессором.

Получив прерывание, процессор незамедлительно прерывает текущий поток управления и передает управление по вектору прерывания, сохраняя состояние прерванного потока, а после завершения, возвращается к нему, восстанавливая состояние исходного.

В зависимости от **источника возникновения сигнала прерывания** делятся на:

- **асинхронные**, или внешние (аппаратные) – события от ВУ, могут произойти в произвольный момент относительно внутренней работы процессора
- **синхронные**, или внутренние – события в самом процессоре как результат нарушения каких-то условий при исполнении машинного кода, например, деление на ноль или переполнение стека.
- **программные** (частный случай внутреннего прерывания) – инициируются исполнением специальной инструкции в коде программы. Как правило, используются для обращения к функциям встроенного программного обеспечения (firmware), драйверов и операционной системы.

По принятию решения о необходимости обработки прерывания прерывания делятся на:

- **Маскируемые прерывания** (разработчику доступны регистры, конфигурация которых позволяет их игнорировать). К примеру: отключение прерываний в рамках выполнения критической секции кода.
- **Немаскируемые прерывания** (которые нельзя проигнорировать). К примеру: ошибка доступа к основной памяти или обработки сторожевого таймера (WatchDog таймер).

Приоритизация прерываний:

- **относительные прерывания** (может быть обработано немного позднее);
- **абсолютное** (необходимо прервать текущее прерывание и перейти к обработке нового немедленно).

Прерывания могут срабатывать по **разным видам событий**, а именно:

- по фронту (о котором сигнализирует переход уровня сигнала на линии прерывания)
- по уровню сигнала
- по сообщению (*message signaled*), реализуется очередь сообщений о наступлении прерываний)
- по дверному звонку (*doorbell*), сигнализируется наступление прерывания, в то время как информация о нём сохраняется в условленном месте.

Данные задачи обычно решаются на аппаратном уровне (**контроллер прерываний**).

Прямой доступ к памяти появился ради того, чтобы освободить процессор от работы с вводом/выводом и дать ему возможность заниматься другим полезным делом. DMA-контроллер (*DMA – прямой доступ к памяти*) подключается к периферийному устройству и получает команды от CPU о вводе/выводе и напрямую записывает/копирует в/из памяти. Такой подход действительно универсален. Для обмена нужно сообщить:

- Количество слов в передаче данных
- Адрес памяти
- Режим передачи данных

Виднеется проблема - гонки за общую шину между CPU и DMA.

Режимы работы DMA с CPU:

- Пакетный режим - приоритет DMA: передача данных осуществляется единой операцией, которая не может быть прервана процессором
- Циклический режим - приоритет конфигурируется: для процессора и DMA выделяется фиксированный слот времени в рамках цикла
- Прозрачный режим - приоритет процессора: передача данных, когда процессор не взаимодействует с памятью

Достоинства DMA:

- Скорость
- Эффективность. Снижает нагрузку на процессор
- Параллелизм: DMA может иметь несколько каналов для параллельной работы
- Интерактивность, так как процессор разгружен от “рутины”

Недостатки DMA:

- Проблемы совместимости - нужен специализированный интерфейс (*процессор, память, периферия*)
- Сложность при непоследовательном доступе к памяти (*накладные расходы на управляющее воздействие при непоследовательном обращении*)
- Ограниченный контроль за системной шиной. Синхронизация работы процессора и DMA
- Конфликты использования DMA разными устройствами ввода/вывода

Channel I/O и прямой доступ к памяти (Direct Memory Access -- DMA). В случае, если необходимо передать большой объём данных, использование для этого процессора является нецелесообразным по причине его откровенной избыточности для этих задач. В таком случае могут применяться процессоры ввода-вывода и контроллеры прямого доступа к памяти, которые позволяют процессору задекларировать необходимость передачи данных (откуда, куда, сколько), после чего её сможет реализовать специализированное устройство, которое собственно и обеспечит передачу данных и уведомит процессор о результатах через систему прерываний.

Выделяют два основных принципа работы:

- **Third-party**, где работа с DMA управляется полностью процессором, и любая передача данных должна инициализироваться им.
- **Bus mastering**, где работа с DMA управляется в том числе и со стороны устройств ввода-вывода, что позволяет инициализировать передачу данных без участия процессора.

Так как в подавляющем большинстве случаев одновременная работа контроллера прямого доступа с памятью и процессора невозможна, **режимы функционирования DMA по отношению к памяти:**

- **Пакетный режим (Burst Mode).** Передача данных осуществляется единой операцией, которая не может быть прервана процессором.
- **Циклический режим (Cycle stealing mode).** Работа с памятью для процессора и устройств ввода-вывода имеет циклическую природу, где для каждого вида операций выделяется свой временной слот. Это позволяет избежать непредсказуемых задержек в работе процессора, но может снижать общую производительность системы.
- **Прозрачный режим (Transparent Mode).** Передача данных осуществляется в те моменты времени, когда процессор не взаимодействует с памятью.

№19 Параллелизм уровня задач. Кооперативная многозадачность. Принцип работы и подходы к реализации. Примеры использования. Достоинства и ограничения. Зелёные процессы. Проблема синхронизации процессов по управлению и по данным. Сравнение с альтернативами.

Параллелизм:

1. Когда нужно работать сразу над несколькими задачами (ввод/вывод, системы управления)
2. Когда нужно повысить уровень утилизации ресурсов (не простаиваем, а занимаемся чем-то полезным)
3. Когда нужно повысить производительность компьютера (делаем больше дел за единицу времени)

Виды параллелизма:

- Уровень битов (на уровне комбинационных схем - “ширина” комбинационных схем, шин данных и машинного слова)
- Уровень инструкций (на уровне инструкций - конвейер)
- Уровень задач (прикладная параллельность, уровня бизнес-задач, параллельное выполнение нескольких программ)

Параллелизм уровня задач - это подход к параллельному выполнению задач, при котором каждая задача выполняется независимо от других задач. Этот подход позволяет максимально эффективно использовать ресурсы системы, такие как многопроцессорные системы или многопоточные среды.

Центральное место занимает вопрос о том, как происходит переключение между потоками команд, влияние этого на разработку ПО и синхронизацию потоков исполнения.

Проблемы:

- Фон Неймана не рассчитана на параллелизм (либо следующая инструкция, либо переход)
- Единый поток инструкций - один (процессор работает только с одним потоком инструкций - мы ставим перед собой задачу “обмануть” процессор и давать ему одну большую программу)
- Прямое выполнение (до остановки)

Варианты параллелизма:

- Кооперативная многозадачность - следующая задача выполняется после готовности текущей отдать процессорное время. Соответствует Фон Нейману
 - о Текущая задача забирает все процессорное время
 - о Фоновые - заморозка
 - о Приложение захватывает столько ресурсов, сколько хочет
 - о Приложения делят процессор, передавая управление следующему

Например, такое проявляется при системных вызовах.

Требуются следующие вычислительные механизмы:

- о Остановка выполнения - добровольная передача управления диспетчеру
- о Сохранение состояния - регистры, стек, память, состояния сопроцессоров,, ввод/вывод, кеш, состояние предсказателя переходов и т.п.
- о Планирование - какой задаче отдать процессорное время следующей
- о Возобновление остановленного процессора - восстановление состояния и передача управления
- о Изоляция задачи - независимое выполнение, безопасность
- о Взаимодействие между задачами - передача данных и сигналов, общие ресурсы
- о Подходы к реализации:
 - Имитация через конечные автоматы
 - ОС (системные вызовы передачи управления)
 - Виртуальные машины, шитый код (вперемешку байт-код и машинный код), *time_slice*:
 - в интерпретаторе
 - в машинном коде

Добавляются в машинный код вставки по проверке.

- Программный код (во фреймворках и UI)
 - *event-loop + callback*
 - *async/await, yield*
- о Практика:
 - Пакетный режим и медленный ввод/вывод (в мейнфреймах), чтобы освободить процессор на время I/O (исторический пример)
 - Простые встроенные системы, bare-metal программирование
 - Статическое планирование (повторение цикла выполнения задач в определенном самом эффективном порядке)
 - Оптимизации систем, требующих частого переключения задач:
 - пример: single thread vs. multi thread web server (nginx/apache) (потоки задач серверов)
 - Node.js. Green Threads (зеленые процессы) - потоки выполнения, управление которыми вместо операционной системы производит виртуальная машина. Эмулируют многопоточную среду вместо ОС, быстрее обычных потоков)
 - *Green Threads - возможность создать легкие процессы (например, в Erlang и Golang) при помощи виртуальной машины (создаются внутри одного процесса/потока).*
 - *в программном коде в сетях через event loop, callback, async/await*
- о Преимущества:
 - *нет гонок, транзакций*
 - Контроль за ресурсами со стороны задачи
 - Известные точки остановки. Отсутствие гонок (задача сама прерывается, а не ее прерывают)
 - Легкость и эффективность при программной реализации
- о Недостатки:
 - Контроль за ресурсами осуществляется задачей. *Заметить бесконечный loop*
 - Сбой задачи может быть глобальным (если процесс решил войти в бесконечный цикл - в него войдет и вся система, т.к. процесс не отдаст нам процессор)
 - Низкая эффективность и трудоемкость ввода/вывода (долго работает процесс - приложение зависает)
 - Сложность разработки интерактивных приложений (очередь задач)

"Зелёные процессы" - это термин, который обычно используется для обозначения процессов, выполняемых в рамках многозадачной среды, такой как операционная система или ВМ (например, Erlang), и имеют свой собственный планировщик. Они могут быть асинхронными и выполняться параллельно, что может привести к проблемам синхронизации по управлению и по данным.

Green thread (зеленый поток) - это абстракция потоков выполнения, которая управляется на уровне операционной системы или виртуальной машины и не зависит от нативных потоков операционной системы. В отличие от традиционных потоков операционной системы, которые управляются ядром операционной системы, зеленые потоки полностью реализованы на уровне пользовательского пространства и работают путем планирования и выполнения потоков программного обеспечения (например, виртуальной машины или библиотеки) без вмешательства операционной системы.

Проблема синхронизации по управлению возникает, когда несколько процессов пытаются управлять общими ресурсами или выполнять последовательные операции в определенном порядке. Это может привести к конфликтам и нежелательному поведению программы.

Например:

Deadlock (Взаимная блокировка): Это ситуация, когда два или более процесса заблокированы и ожидают события, которое может произойти только при выполнении другого процесса или освобождении ресурса. Каждый процесс удерживает ресурс, необходимый другому процессу, и они не могут продолжать свою работу, в результате чего система оказывается в застойном состоянии.

Проблема синхронизации по данным возникает, когда несколько процессов пытаются одновременно получить доступ к общим данным, что может привести к гонкам данных, потере информации или некорректным результатам.

Например:

Race condition (Состояние гонки): Это ситуация, когда два или более процесса состязаются за общий ресурс или производят операции над общими данными без должной синхронизации. В результате могут возникать непредсказуемые или некорректные результаты. Например, при параллельной записи в общий файл двумя процессами, порядок записи может быть непредсказуемым или данные могут быть потеряны.

Существует несколько **альтернативных методов** обеспечения синхронизации процессов:

1. Мьютексы и семафоры:

- Это классические методы синхронизации, которые позволяют процессам ожидать доступа к общим ресурсам или выполнить определенные операции в определенном порядке.

2. Модель акторов:

- В этой модели каждый процесс представлен как актер, который может отправлять и получать сообщения. Это позволяет избежать проблем синхронизации по данным путем изоляции акторов и использования асинхронной коммуникации.

3. Транзакционная память:

- Этот подход позволяет процессам выполнять операции над общими данными в рамках транзакций, гарантируя целостность данных и избегая проблем с гонками данных.

4. Каналы и сообщения:

- Вместо явной синхронизации по управлению или по данным, процессы могут обмениваться сообщениями через каналы, что позволяет им взаимодействовать без явной блокировки или ожидания.

Каждый из этих подходов имеет свои преимущества и недостатки в зависимости от конкретных требований приложения и характеристик системы.

№20 Параллелизм уровня задач. Вытесняющая многозадачность. Механизмы переключения задач. Примеры использования. Достоинства и ограничения. Проблема синхронизации процессов по управлению и по данным. Сравнение с альтернативами.

Параллелизм:

1. Когда нужно работать сразу над несколькими задачами (ввод/вывод, системы управления)
2. Когда нужно повысить уровень утилизации ресурсов (не простаиваем, а занимаемся чем-то полезным)
3. Когда нужно повысить производительность компьютера (делаем больше дел за единицу времени)

Виды параллелизма:

- Уровень битов (на уровне комбинационных схем - “ширина” комбинационных схем, шин данных и машинного слова)
- Уровень инструкций (на уровне инструкций - конвейер)
- Уровень задач (прикладная параллельность, уровня бизнес-задач, параллельное выполнение нескольких программ)

Параллелизм уровня задач - это подход к параллельному выполнению задач, при котором каждая задача выполняется независимо от других задач. Этот подход позволяет максимально эффективно использовать ресурсы системы, такие как многопроцессорные системы или многопоточные среды.

Центральное место занимает вопрос о том, как происходит переключение между потоками команд, влияние этого на разработку ПО и синхронизацию потоков исполнения.

Проблемы:

- Фон Неймана не рассчитана на параллелизм (либо следующая инструкция, либо переход)
- Единый поток инструкций - один (процессор работает только с одним потоком инструкций - мы ставим перед собой задачу “обмануть” процессор и давать ему одну большую программу)
- Прямолинейное выполнение (до остановки)

Варианты параллелизма:

- Вытесняющая многозадачность (истинная многозадачность) - переключение по внешнему событию - ОС передает проц. время между программами в случае завершения операции ввода/вывода, событий в аппаратуре компьютера, истечения таймеров и квантов времени, поступления сигналов.
 - о между любыми двумя инструкциями
 - о Распределение процессорного времени осуществляется планировщиком
 - о Возможна мгновенная реакция на действия пользователя
 - о Вычислительные механизмы кооперативной многозадачности:

Проще говоря, процессы не знают об существовании друг друга и работают тогда, когда им планировщик скажет. Под капотом вытесняющей многозадачности лежит механизм прерываний, что позволяет делать переключение между задачи между любыми двумя инструкциями.

Требуются следующие вычислительные механизмы (Так же как у кооперативной, **НО вместо паузы → прерывание**):

- Механизм прерывания процесса: забрать процессор у задачи независимо от ее желания

- Механизм сохранения состояния задачи: регистры, стек, состояния сопроцессоров, память, ввод/вывод, кеш, состояние предсказателя переходов и т.п.
- Механизм планирования - какой задаче отдать процессорное время следующей
- Механизм возобновления остановленного процесса: восстановление состояния и передача управления
 - *Практика: Вытесняющая многозадачность используется в большинстве современных операционных систем общего назначения, например: Windows, Linux (и другие UNIX), Mac OS.*

Преимущества:

- Простота разработки ПО с одним процессом - большая часть программ однопоточная
- Контроль за ресурсами со стороны ОС. Изоляция (*относительная защищенность процессов и ОС друг от друга с точки зрения захвата ресурсов процессора*)
- Интерактивность системы (почти мгновенная реакция)

Недостатки:

- “Лишние” переключения (контекстов, процессов)
- “Тяжесть” процессов (прерывания, состояния и т.п.) - много действий по переключению между контекстами
- Разделяемые состояния и непредсказуемые переключения. Синхронизация. Гонки - мы без понятия, когда именно произойдет прерывание
- Непредсказуемая длительность работы. Реальное время - мы без понятия, сколько раз нашу программу прервут

Проблемы синхронизации:

- *MapReduce - запускаем алгоритм на разных компонентах нашей системы и на одном из них происходит ошибка - возникает сложность остановки остальных процессов*

Проблема синхронизации по управлению возникает, когда несколько процессов пытаются управлять общими ресурсами или выполнять последовательные операции в определенном порядке. Это может привести к конфликтам и нежелательному поведению программы.

Например:

Deadlock (Взаимная блокировка): Это ситуация, когда два или более процесса заблокированы и ожидают события, которое может произойти только при выполнении другого процесса или освобождении ресурса. Каждый процесс удерживает ресурс, необходимый другому процессу, и они не могут продолжать свою работу, в результате чего система оказывается в застойном состоянии.

Проблема синхронизации по данным возникает, когда несколько процессов пытаются одновременно получить доступ к общим данным, что может привести к гонкам данных, потере информации или некорректным результатам.

Например:

Race condition (Состояние гонки): Это ситуация, когда два или более процесса состязаются за общий ресурс или производят операции над общими данными без должной синхронизации. В результате могут возникать непредсказуемые или некорректные результаты. Например, при параллельной записи в общий файл двумя процессами, порядок записи может быть непредсказуемым или данные могут быть потеряны.

Существует несколько **альтернативных методов** обеспечения синхронизации процессов:

1. Мьютексы и семафоры:

- Это классические методы синхронизации, которые позволяют процессам ожидать доступа к общим ресурсам или выполнить определенные операции в определенном порядке.

2. Модель акторов:

- В этой модели каждый процесс представлен как актер, который может отправлять и получать сообщения. Это позволяет избежать проблем синхронизации по данным путем изоляции акторов и использования асинхронной коммуникации.

3. Транзакционная память:

- Этот подход позволяет процессам выполнять операции над общими данными в рамках транзакций, гарантируя целостность данных и избегая проблем с гонками данных.

4. Каналы и сообщения:

- Вместо явной синхронизации по управлению или по данным, процессы могут обмениваться сообщениями через каналы, что позволяет им взаимодействовать без явной блокировки или ожидания.

Каждый из этих подходов имеет свои преимущества и недостатки в зависимости от конкретных требований приложения и характеристик системы.

В реализации вытесняющая многозадачность отличается от кооперативной, в частности, тем, что требует обработки системного прерывания от аппаратного таймера. По истечении кванта времени, отведённого процессу, происходит прерывание и вызывается планировщик процессов. Частота вызова планировщика критична: слишком частый его вызов будет расходовать процессорное время впустую.

№21 Система прерываний. Виды прерываний. Механизм обработки прерываний по шагам. Задачи, решаемые механизмом прерываний. Сторожевой таймер.

система Направлена на описание спец. функции, **вызываемой по внешнему событию**. Позволяет переключиться (совершить переход к заданной инструкции) по событию и вернуться назад после окончания обработки

Принцип работы:

- Выполнение основного потока
- Запрос прерывания
- Сохранение адреса возврата (сохранение PC в регистр или стек)
- Вызов обработчика прерывания и передача ему управления (ISR) - обычного кода в специальном месте
- Завершение обработчика (ISR)
- Восстановление адреса возврата (из PC)
- Продолжение потока

Обеспечиваемые уровни задачи:

- Основной поток
- Поток обработчика прерывания

Источники прерывания (виды прерываний):

- Аппаратные
 - о Внешние (асинхронные для внутренних циклов процессора): переполнение таймера, нажатие клавиш, сетевой пакет
 - о Внутренние (синхронные, по итогу обработки команды - как деление на 0, ошибка доступа к памяти и тд)

Асинхронно - в любой момент может произойти (тут имеются в виду такты)

Синхронно - должны быть готовы обработать прерывание в определенное время (такт)

- Программные (вызывается инструкцией) - взаимодействие (по нуждам) программы и ОС

Для внешних устройств используется контроллер прерываний:

- Реализуется аппаратно (высокая скорость и надежный параллелизм)
- Конкуренция между прерываниями (выдача приоритетом, очереди):
 - о Определяет приоритет контроллер прерывания
 - о Маскируемые/Немаскируемые (какие устройства лучше приглушить, а какие почаще слушать)
 - о Относительные/Абсолютные (прерывает прерывание) - на сколько один прерывания могут прерывать другие (вопрос: а что делать, если во время прерывания пришел запрос на прерывание?) - относительное - игнорируем, абсолютные – обрабатываем
- Виды события (*тоже виды прерываний*):
 - о По фронту (по изменению уровня) - нажатие кнопки
 - о По уровню (прерывания по уровню сигнала все время, поэтому его нужно сбрасывать) - *сигнал пошел в 1*
 - о По сообщению - протокол ИО (содержит дополнительные данные)
 - о По doorbell (программа уведомляет аппаратуру о необходимости произвести работу и сделать прерывание). Разделены этапы информирования процессора и непосредственной передачи данных

Благодаря прерываниям можно осуществлять:

- Ввод/вывод
- Обработку аварийных ситуаций
- Совершать системные вызовы

Сторожевой таймер - аппаратно-реализованная схема контроля от зависания системы. Он позволяет обнаружить, что система попала в неадекватное состояние (если это подтверждается - перезагрузка системы). Смысл работы простой - кидаем сигнал проверки → если процесс ответил - классно (сигнал сбрасывается), не ответил - ужас. *Он подключается к сбросу, отслеживая работу процессора (получая от него сигналы о нормальной работе) в течение некоторого времени. Если сигналы выходят за пределы таймера - перезагрузка процессора.*

Механизм работает так: когда все работает корректно - таймер запускает отчет и ожидает сигналы от процессора с таким интервалом, на который он настроен, информируя о продолжении работы. Следовательно, пока сторожевой таймер укладывается в нужный диапазон по времени - то все функционирует корректно.

В случае если таймер выходит за пределы - он понимает, что что-то пошло не так - в этот момент он его аварийно перегружает, процессор запускается сначала, в надежде на то, что тот ущерб, который был нанесен, не критичен и система восстановится.

SPI (serial peripheral interface) - последовательный интерфейс передачи данных - обеспечивает синхронную передачу данных между микроконтроллера и устройства io

МОСИ - отдаем данные в устройство МИСО - получаем данные из устройства Сигнал клок- чтобы ведомый понимал, когда вы шлете данные CS - выбор чипа или устройства

SPI

- *от SS (ведомого) - плохо, зависает в прерывании*
- *от SS и SCLK - проще (наш выбор)*
- *SS+SCLK - сложно*

+:

- *с одним процем - просто*
- *повышены изоляция, контроль, стабильность, интерактивность*
- *регистрируем неадекватное поведение*

-:

- *лишние переключения*
- *тяжесть процессов*
- *синхронизация и гонки*
- *нельзя предсказать время работы (пока попадет в очередь..)*

№22 Параллелизм уровня задач. Проблемы совмещения, изоляции и взаимодействия задач между собой. Методы разрешения данных проблем (с точки зрения опыта программиста и пользователя): распределение по адресному пространству, банки памяти, сегментная организация памяти и виртуальная память.

Параллелизм уровня задач это несколько выполняемых одновременно программ. Данный процесс сопровождается определенными трудностями (например, из-за того, что сам по себе классический фон Неймановский процессор не приспособлен к данному параллелизму). Требуется решить следующие вопросы:

- Обеспечить совмещение и изоляцию между задачами
- Обеспечить взаимодействие между задачами

Если брать за основу фон Неймановский процессор, то у нас единое адресное пространство под:

- Программы
- Данные

Без изоляции в одном адресном пространстве надо проверять, что мы не записали в 1 адрес и использовать сложную адресацию (мы поменяли адрес, чтобы избежать коллизий, но тогда нужно поменять и адреса всех переходов)

Так как единое адресное пространство программ и данных, поэтому нужна изоляция процессов. Есть исключения (динамические библиотеки), но необходима изоляция:

- Адресов инструкций (переходы) - *тут либо надо указывать компилятору, либо линковщику*
- Статических данных и переменных- уникальная ячейка памяти под конкретный процесс
- Динамическая память (куча) - каждому процессу - своя куча плохо фрагментация
- Автоматический память (стек) - возможны коллизии между стеками

Если список задач фиксирован (встроенная система), то изолируем при компиляции, иначе - происходит автоматически при выполнении

Инструкции и данные разных процессов ни в коем случае не должны лежать в одних и тех же адресах.

Но как же решать эти проблемы совместимости/изоляции? - В compile-time и run-time.

Методы решения проблемы изоляции:

- Банки памяти, отчасти - как правило, такой подход используется для расширения машинного слова или увеличения адресного пространства, но также такой подход можно и применить для изоляции (например, в одном банке лежит ОС, в другом - программа), *технология подключения нескольких чипов памяти к процессору (параллель, что расширяет машинное слово или последовательное, что расширяет память (адрес в процессоре))*, но также такой подход можно и применить для изоляции (например, в одном банке лежит ОС, в другом - программа)
 - o *Расширение адресного пространства*
 - o *Расширение машинного слова*
 - o *Переключение режима работы между задачами (аппаратная изоляция)*
 - o *Редко использует изоляцию, так как разным задачам - разные банки памяти (но больше костыль)*
- Сегментная адресация памяти - способ логической адресации памяти, где адрес: сегмент + смещение. Смещение - адрес ячейки памяти относительно начала сегмента.
- Виртуальная память - Предоставим каждой задаче свое независимое адресное пространство. Пусть каждый процесс думает, что все адресное пространство его. Пусть процесс думает, что его адреса начинаются от нуля и вплоть до бесконечности..

Взаимодействие между задачами

Типовые виды:

- Исходный поток - инструкции, которые выполняет с самого начала
- Прерывания - код обработчиков прерывания
- Потоки - несколько работающих в параллель задач с одним адресным пространством
- Процессы - изолированные адресные пространства
- Зеленые потоки

Из-за непредсказуемой последовательности исполнения инструкций с недетерминированным результатом пришла мысль искусственно согласовывать потоки:

- Атомарные операции (например, compare&swap)
- Mutex, Semaphore
- Transactional Memory (как в БД)
- Process Network, Promise, Futures, Actor-Model... + async/await

Искусственной остановки потока практически нет в ЯП, т.к. потоки не отслеживаются ОС, и кто знает, какие данные нужно высвободить и нет.

Взаимодействие процессов

Проблема - получить доступ к изолированной памяти. Решения:

- Shared memory - доступ к одинаковым данным разных процессоров
- Signals - передача управления между ОС и ядром (обработчиками сигналов) с помощью сигналов (сисколы)
- Ввод-вывод IO: Network, Files, Pipes

№23 Сегментная и виртуальная память. Решаемые задачи и принципы работы. Проблема фрагментации. Достоинства и недостатки.

Что если разметим МЕМ заранее? Наведем порядок!

Идея возникновения сегментной памяти возникла с предложением: “а давайте во время программирования указывать, в каком месте будет лежать код, в каком - переменные, куча и т.д.). Сегментная адресация памяти - способ логической адресации памяти, где адрес: сегмент + смещение. Смещение - адрес ячейки памяти относительно начала сегмента.

Сегмент - выделенная область памяти пространства определенного размера. Вся память разделяется на сегменты, в которой любой процесс работает с данными (выход за пределы - ошибка). Упрощает изоляцию процесса, позволяет создавать и использовать механизмы памяти в ПО (segfault).

Принцип работы:

- Получаем номер сегмента и смещение от начала сегмента
- Пересчет номера сегмента в физ. адрес
- Проверка (адрес + смещение) $\leq$ следующий сегмент

- Если да - складываем и получаем адрес в физ памяти.
- Если нет - ошибка

Позволяет:

- Сделать независимую адресацию внутри сегмента (нет коллизий)
- Управление доступом (для процессов) (чтение, запись). Запрет доступа к “чужим” сегментам
- Взаимодействие через общий сегмент
- Изоляция программных модулей, *динамические библиотеки* (меньше вероятность ошибок)
- Перекрывание сегментов (высокая эффективность доступа) можно выстроить сложное взаимодействие между сегментами

Плюсы:

- Маленькие таблицы сегментов
- Таблицы просты в обработке и обращении
- Сегменты обычно больше страниц памяти, что позволяет хранить больше данных процесса
- Отсутствие внутренней фрагментации (процессу выделяется ровно столько, сколько нужно, без пустых мест) это про степени двойки и пустые ячейки внутри

Минусы:

- Сложно поддерживать со стороны ПО (никто не будет переписывать все ПО под это)
- Использование требует участие программистов (количество сегментов, размер сегментов)
- На сегодня - считается устаревшей и имеет ограниченную поддержку со стороны ОС (Linux)
- Подвержена внешней фрагментации (между занятыми участками остаются свободные доступные участки, которые не получается использовать из-за их малого размера)

У нас есть ФН, совмещаем последовательность с многозадачностью

Виртуальная память - механизм изоляции через разделение адресного пространства на одинаковые части (страницы памяти) и передачи каждого своему процессу. Каждая страница отображается на физическую память в случайном порядке. Память разных задач разделяется на страницы и работая с разными страницами. Также можем вынести страницы с основной памяти на внешнюю.

Предоставим каждой задаче свое независимое адресное пространство. Пусть каждый процесс думает, что все адресное пространство его. Пусть процесс думает, что его адреса начинаются от нуля и вплоть до бесконечности.

Принцип работы:

- Есть логический адрес и смещение
- Логический пересчитывается в физический через таблицу страниц
- (физический + смещение) → физический адрес

Позволяет (тут же в целом плюсы):

- Прозрачная изоляция задачи для программиста (не нужно задумываться)
- Нелинейное физическое размещение данных (если сегмент был единым, то страницы могут быть раскиданы)
- Не фиксировать объем памяти, используемый задачей (не нужно указывать размеры, адрес начала/конца и прочее)
- Использовать больше памяти, чем есть физически. Выгрузка на диск части задачи (задач)
- *Права доступа*. Отображение страниц на разные адресные пространства
- *Отсутствует внешняя сегментация*

Недостатки:

- Большой объем таблиц страниц, длительный поиск (кеш)
- Высокие накладные расходы (ввод/вывод, перенос страниц и т.п.)
- Нет изоляции внутри адресного пространства процесса
- Непредсказуемая длительность доступа к памяти
- Сложно реализовать

Фрагментация бывает внутренней и внешней

Внутренняя фрагментация - имеется блок памяти фиксированного размера, но наш процесс требует меньше, то будет пустовать память (впустую потратили).

Внешняя фрагментация - когда требуем/освобождаем память, то возникают участки памяти, которые не задействованы

Сегментация страдает внешней фрагментацией, а виртуальная память - внутренней.

№24 Иерархия памяти (явная и скрытая). Виды памяти. Особенности использования на практике. Устройство памяти с произвольным доступом. Устройство и принцип работы ROM, SRAM, DRAM ячеек.

Иерархия памяти - модель представления памяти по убыванию ее скорости и возрастанию быстродействия.

Если представить иерархию памяти, то перед нами появится классическая пирамида, на вершине которой стоит самая быстрая память (регистры), затем с прохождением вниз основание у нас расширяется (что соответствует увеличению объема памяти и уменьшению ее стоимости).

Сама пирамида представляет из себя следующее (идем сверху-вниз):

1. Регистры процессора
2. Кэш разных уровней
3. RAM
4. Основная память компьютера
5. SSD-диски
6. Магнитные диски, ленточные накопители

Задачи:

- *Исходные данные для запуска системы*
- *Рабочие данные, с которыми работает система*

Проблема - с ростом скорости растет стоимость (т.к. утыкаемся в стоимость производства или стоимость размещения (не можем на кристалле разместить большие объемы - просто не влезут)). Нерешаема из-за вытеснения старых технологий новыми на рынке при сохранении общей иерархии + разные технологии или принципы организации памяти на каждом уровне → разная скорость

Иерархия памяти для разработчика

Явная иерархия - контроль за использованием лежит на программисте, каждый вид памяти - своя шина с процессором, проц выбирает. Использование с умом:

1. Регистры, SRAM, DRAM, диски доступны как альтернативные хранилища

Можно явно обращаться к любой памяти, делая программу наиболее эффективной. Но стоит учесть, что если железо поменяется, то придется менять программу.

Неявная иерархия - единое адресное пространство, прозрачное обращение к какому-то виду памяти (использование конкретной памяти скрыто от процессора и программиста). Система сама разберется:

1. *Модель: одна память, одно адресное пространство*
2. *Зависит от использования. Определяется прозрачно*
3. *Посыл: мы все сделаем за тебя*

У нас есть “умная” железка, которая сама определяет, куда что пойдет.

Преимущественно используется скрытая иерархия, потому что прозрачна для программиста.

Типы доступа к памяти:

- С произвольным доступом (Random Access Memory) - задержка доступа не зависит от истории запросов, можем изменять адрес доступа в произвольное время в произвольном месте без накладных расходов
- С последовательным доступом (жесткие диски, магнитные ленты, AWS Tape Gateway) - длительная задержка при смене адреса, часто высокая скорость последовательной записи/чтения
- Гибридная - библиотека магнитных лент (рандомно выбираем ленту), векторные операции

RAM

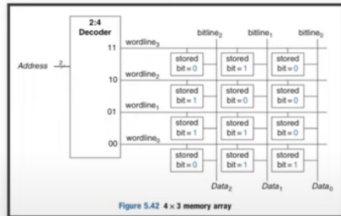
Принцип работы:

wordline - слово, bitline - значение

- На вход декодеру подается адрес, декодирующийся в wordline (линия, активирующая ячейки запрашиваемого машинного слова)
- wordline активирует все битовые ячейки

- Параллельно с этим на bitline выставляется значение необходимой ячейки
- Захват ячейки

Память с произвольным доступом (RAM). Принцип работы



Массив ячеек памяти 4×3
(4 слова по 3 бита)

- Address — адрес ячейки памяти шириной 2 бита.
- Decoder — **дешифратор**, активирует линию (1 из 4 шт.).
- wordline_i — линия, активирующая ячейки требуемого машинного слова.
- bitline_i — линия, на которую выставляется/читается значение бита определённой позиции.
- stored_bit — ячейка памяти.

Каждая ячейка хранит один бит. По реализации битовые ячейки бывают: **Flip-flop, ROM, SRAM, DRAM**

Flip-flop

Триггеры, используемые для построения регистров процессора (очень быстрые, очень дорогие, малой емкости), представляет ячейка на один бит, триггер ориентирован на хранение одного состояния, набор связанных триггеров - регистр

ROM (Read Only Memory Cell - память только для чтения.)

Ячейка памяти с постоянным значением (вбито аппаратно), состояние хранит 1 транзистор.

Способы реализации ROM:

- *Физическое (не)размещение транзисторов (на производстве)*
- *Пережигание перемычек при однократном программировании*

Принцип работы:

- wordline замыкает перемычку
- bitline по перемычке читает 0 (в землю)
- Если нет замыкания, то wordline переходит в bitline и передает 1

На bitline подается единица, если мы замыкаем ее дополнительным транзистором - будет нуль, иначе, при замыкании только с wordline, будет единица.

SRAM (Static Random Access Memory) - память, которая хранит данные при помощи состояния группы транзисторов (4 - инвенторы, 2 - доступ):

- Быстрый доступ на чтение и запись
- Значение хранится до отключения питания (но самоподдерживающийся благодаря циклическому расположению транзисторов)
- Требуется довольно большое количество транзисторов (низкая плотность ячеек)

Принцип работы:

- 2 bitline - прямой и инверсный
- wordline открывает обе перемычки
- 1 сигнал с одного из bitline производит операцию со значением

SRAM — это действительно память *произвольного* доступа, доступ к любой ячейке памяти в любой момент занимает одно и то же время.

Сенсоры внизу фиксируют падение сигнала на битлайне

Проблема: много транзисторов и дорого.

DRAM (Dynamic Random Access Memory - динамическая память, состояние хранится в конденсаторе.)

Состояние можно сосчитать один раз, затем нужен снова зарядить. Со временем конденсатор разряжается - эту проблему решает контроллер памяти, регенерирующий заряд (проходит по всем ячейкам, читает значение и записывает аналогичное), но блокирует доступ на время регенерации; один транзистор и конденсатор на ячейку памяти.

Нужен контроллер для регенерации, блокирует доступ при ней

Наиболее компактные и дешевые → Позволяют хранить большой объем данных.

№25 Механизм кеширования в компьютерных системах, принцип локальности. Функционирование кеш памяти процессора (чтение, запись). Виды кеш промахов. Механизм вытеснения (LRU, PLRU).

Кэш - промежуточный буфер с быстрым доступом, содержащей наиболее возможную к использованию информацию. Позволяет не думать об устройстве памяти, ее механизмах.

- Малый объем быстрой памяти
- Целесообразен при неравномерном доступе к памяти (низкая пространственная и временная локальность данных)

Должна обеспечиваться как пространственная, так и временная локальности данных. Мы работаем с относительно небольшим объемом данных. Пространственная локальность - это про адреса, а временная - мы работаем с определенным объемом данных в единицу времени. Идея в том, что мы должны предугадать, какой набор данных нам потребуется в ближайшее время.

Если много данных и разные адреса данных, то кэш неэффективен (каждый раз доступ в разные места), но если мало и эффективное расположение по памяти, то большая часть кэша простаивает.

Кеш память является таблицей, хранящей идентификатор

Основные характеристики:

- Индекс - номер кеш-линии в самой памяти
- Тег - показывает, какая основная память отображена

Кеш — это память и логика. Логика реализует:

1. Поиск кеш-линии по тегу (компараторы).
2. Вытеснение, replacement (определение ненужной кеш-линии).
3. Предзагрузка (prefetch) данных и инструкций.
4. Взаимодействие с памятью (группировка операций и т.п.).
5. Синхронизации кешей разных уровней.

Больше данных → больше логики.

Масштабирование по времени или по площади:

- 2 линии, 2 компаратора → 1 такт,
- 2 линии, 1 компаратор → 2 такта.

Принцип работы:

- Кеш состоит из набора кеш-линий (блоков кеша, записей).
- Кеш-линия ассоциирована с элементом в медленной памяти.
- Соответствие кеш линии и адресу памяти определяет тег

Нужный тег - попадание, нет - промах

Память может изменяться независимо от кэша (DMA)

Чтение данных:

- Обращаемся к кешу по некоторому адресу основной памяти. Запись по адресу есть - кеш-попадание (данные читаются и отдаются процессору)
- Записи нет - кеш-промах (кеш останавливает программу, запрашиваем данные из памяти или следующего кеша и сохраняем себе). Сохранение:
 - о Есть пустая линия - сохраняем
 - о Нет - анализ неактуальной линии и вытеснение ее
- Длительность получения данных - произвольна, так как кеш многоуровневый или память может быть заблокирована (DMA, зарядка конденсатора в DRAM)
- Уровень попадания - характеристика эффективности кеша.

Запись данных (попадание. пишем в MEM и в кэше уже есть данные):

При доступе к данным **на запись** процесс немного сложнее, так как подразумевает перенос внесенных изменений из кеша в основную память, реализация которого регулируется политикой записи. Выделяются следующие варианты:

- Немедленная запись - запись в кеш данных и из кеша в память. Поддерживает актуальное состояния в памяти
- Отложенная запись - обновление памяти происходит при вытеснении кеш-линии, периодически или по запросу, но нужно хранить признак модификации ("грязный")
- Группировка изменений, сокращение промежуточных состояний

(пример в цикле делается +1 мы сгруппировали все эти изменения и отправили одной пачкой)

- неконсистентное состояние MEM и кэша (для процессора невидимое, для DMA требуется принудительная запись)
- Гибридный вариант - немедленная запись с буферизацией

Запись данных (при промахе):

- Запись с размещением - сначала пишем в кэш, затем в MEM
 - о Данные загружаются в кэш, после в них вносится исправление.
 - о Два обращения: запись вытесняемого и чтение необходимого.
 - о Поведение аналогично промаху по чтению.
- Запись без размещения - запись напрямую в память или буфер, в обход кэша

Влияние кеш-промаха (по убыванию):

- Запись данных - минимальная задержка, т.к. положили в какой-нибудь буфер и там уже когда-нибудь да запишется (можно отложить) - низкое
- Чтение данных - средняя задержка, может компенсироваться параллелизмом уровня инструкций (например, суперскалярные процессора) - среднее
- Чтение инструкции - большая задержка, процессор простаивает в ожидании инструкции (процессор сидит и ждет) - высокое

Вытеснение и замещение кеш записи - при кеш промахе необходимо поместить новые данные в кеш линии. Куда класть - на место не требующейся линии. Реализация аппаратным путем.

Когда дело касается замещения (а особенно в случаях, когда у нас полностью ассоциативный или множественно ассоциативный кеш), перед нами ставится вопрос, какие данные нам вытеснять (это когда у нас кеш уже полный и свободных ячеек уже нет). Существуют следующие алгоритмы:

Алгоритмы вытеснения:

- LRU - проставляем момент доступа к каждой кеш-линии и выкидываем самую маленькую дату. Так как даты хранить не удобно, используется общий счетчик, инкрементирующийся при доступе к любой линии. Линия с самым маленьким значением вытесняется, на ее место новое значение с увеличенным счетчиком.
 - о Дорого сравнивать каждый счетчик
 - о Слишком большие значения для счетчика (много линий)
 - о Модификация Victim cache - в специальную изолированную кеш линию выносим линию с наименьшим счетчиком, затем выкидываем насовсем
 - Заранее выбираем для выброса линию, что ускоряет вытеснение
 - о Можно запретить кеширование - имеет + и -, но иногда помогает (аппаратно)

PLRU (Pseudo-LRU) - представление записей в виде дерева, где листья - кеш линии, узлы - биты кода пути к последней используемой записи.

Алгоритм:

Изначально все НЕ листовые вершины заполнены ноликами. При записи мы идем от корня и проходя путь инвертируем значения. Таким образом, когда нам нужно будет выполнить замещение, мы просто пройдем по этому пути и получим ячейку для записи.

- Спускаемся вниз, при прохождении через вершину - инвертируем ее и меняем стрелку родительского узла на соседний дочерний узел
- Записываем в лист значение кеша
- Легче, т.к. меньше логики

Алгоритм

- Бинарное дерево:
 - о вершины хранят путь: $left(0) / right(1)$;
 - о листья: кеш-линии.
- Поиск записи для вставки или вытеснения:
 - о спуск по дереву согласно значениям в вершинах;
 - о инверсия всех пройденных вершин.
- Кеш попадание — инверсия пути от корня до листа.
- Значительно "легче" LRU.

По идее программист вообще не должен думать о кэшах, НО:

- Meltdown
- Перемножение матриц (предварительно транспонировать)
- Массив структур или структура массивов

Программно легко, аппаратно много логики:

- компаратор, вытеснение, предзагрузка, группировка, много уровней

№26 Устройство кеш-памяти процессора. Ассоциативность кеш-памяти (полностью ассоциативная, прямое отображение, множественно-ассоциативный кеш). Принципы работы. Детальное описание принципов работы кеш-памяти с разными вариантами ассоциативности.

(если данных много и они случайно распределены - кеш только нам во вред)

Кеш - промежуточный буфер с быстрым доступом, содержащий информацию, которая может быть запрошена с наибольшей вероятностью.

Устройство:

- Совокупность памяти (ячейки памяти) и логики (реализация поиска нужной кеш-линии)
- Логика реализует:
 - о Поиск линии по тегу (занимается компаратор)
 - о Вытеснение (определение ненужной линии)
 - о Предзагрузка (заранее предугадать инструкции и данные) - предполагаем, что данные нам понадобятся
 - о Взаимодействие с памятью (группировка операций чтения и записи)
 - о Синхронизация кешей разного уровня
- Можем масштабировать по времени или площади (больше компараторов)
- Больше данных - больше логики

Масштабирование по времени или по площади (trade of):

- Если хотим быстро выполнять поиск - используем больше аппаратуры
- Если хотим экономить площадь - жертвуем временем (меньше компараторов)

Ассоциативность кеш-памяти - политика размещения данных в памяти в кеше для оптимизации поиска. Мы стараемся добиться компромисса в соотношении между аппаратурой и логикой. Связана с устройством тегов, которые напрямую связывают адрес памяти с линией.

Строение адреса (сначала старшие биты, затем средние и младшие)

Тег - идентификатор области памяти, сохраненной в кеш-линии. Выбор тега может быть следующим (имеется в виду, какую часть адреса памяти берем):

- Полный адрес памяти - занимает много места, требует больших компараторов (нецелесообразно)
- Убираем младшие биты - особенно хорошо, когда работаем блоками, а не байтами
- Средние биты - часто циклически меняются, т.к. особенность устройства битовой последовательности (адреса) (принцип временной и пространственной локальности)
- Старшие биты - относительно уникальны (если используются)

Варианты ассоциативности:

- Полностью ассоциативный кеш
 - Любая кеш линия может отобразить любой блок из памяти
 - Тег использует средние и старшие биты
 - Лучшая эффективность, но много логики и компараторов
- Кеш с прямым отображением
 - Кеш линия может отобразить несколько ячеек (одна кеш линия - много ячеек, одна ячейка - одна кеш линия)
 - Использует средние биты и старшие биты, но средние указывают на номер линии, в которую пишем (то есть все ячейки с одинаковыми средними битами будут отображаться в одну линию), что помогает быстрее искать раскиданные ячейки только через 1 линию. Старшие также отображают адрес.
 - Меньше памяти для тегов (только средние биты), меньше компараторов (работают только со старшими битами), нет вытеснения (так как только одна линия, куда можем сохранить ячейку), но возможны коллизии адресов (работать только с одной и той же линией, без возможности изменить линию записи для ячейки)
- Множественно-ассоциативный кеш
 - Похож на кеш с прямым отображением, но группируем линии в банки, которые отображают соседние блоки, т.е. сочетание полностью ассоциативного кеша и кеша с прямым отображением. Кеш с прямым отображением (1 банк), воспроизведенный N раз.
 - Старшие также отображают адрес и множество линий, средние конкретную линию.
 - Выше эффективность, ниже скорость, выше эффективность
 - Выбор банка зависит от алгоритма вытеснения (что решает проблему предыдущего)

По идее программист вообще не должен думать о кэшах, НО:

- *Meltdown*
- *Перемножение матриц (предварительно транспонировать)*
- *Массив структур или структура массивов*

№27 Иерархия кеш-памяти процессора. Разделённый/унифицированный, включающий/исключающий, частный/общий. Причины множества уровней кеша. Типовые уровни кеша в современных процессорах.

Кеш - промежуточный буфер с быстрым доступом, содержащий информацию, которая может быть запрошена с наибольшей вероятностью.

Многоуровневый кеш - иерархия с несколькими уровнями кеша, взаимодействующими между собой.

Свойства каждого уровня зависят от:

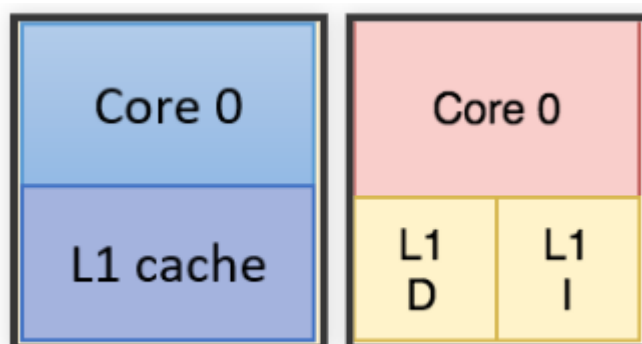
- Технология ячеек памяти
- Устройство кеша (размер, ассоциативность и т.д.)

Задача:

- Повысить скорость доступа
- Синхронизация процессорных ядер
- Оптимизация интерфейсов

Способы организации кеша:

- Разделенный/унифицированный - кеш память может быть реализована одним или несколькими банками памяти. Иметь один канал доступа или несколько. По принципу деления могут быть:
 - о По типу данных:
 - Кеш данных.
 - Кеш инструкций.
 - Trace cache (микрокод).
 - о По процессорному ядру



Левая часть изображения:

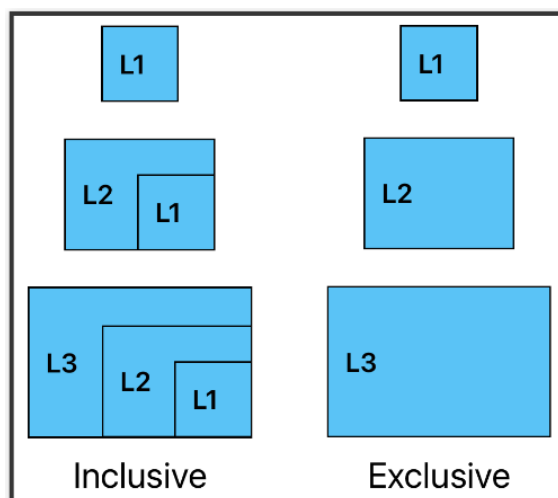
- Вариант, где кэш первого уровня (L1) объединен (unified) и используется для хранения как данных, так и инструкций.
- **Core 0:** Само процессорное ядро.
- **L1 cache:** Единый кэш, который содержит как данные (Data), так и инструкции (Instructions).

Правая часть изображения:

- Вариант, где кэш первого уровня (L1) разделен на два отдельных кэша: один для данных (Data) и один для инструкций (Instruction).
- **Core 0:** Само процессорное ядро.
- **L1 D (Data cache):** Кэш данных, который хранит только данные.
- **L1 I (Instruction cache):** Кэш инструкций, который хранит только инструкции.
- Достоинства разделенного кэша:
 - о Повышение скорости доступа (больше каналов)
 - о Адаптация устройства под специфику использования (скорее всего, тут речь про гарвардскую архитектуру)
- Недостатки разделенного кэша:
 - о Усложнение процессора
 - о Конфликты между банками кеш-памяти
- Включающий/исключающий - должен ли кэш одного уровня включать кэш другого уровня?

Может ли присутствовать и в кэше, и в MEM

- о Инклюзивная (включающая) - дублирование данных линий (за счет резервирования места в блоке более медленного кэша), более быстрый доступ, но потеря части памяти (так как часть памяти более медленного кэша теряется для резервирования)
- о Эксклюзивная (исключающая) - данные не дублируются, память используется эффективнее
- о Разного рода гибридные варианты (Non-Inclusive, Non-Exclusive - NINE)



- Частные/общие - доступ процессора/ов к кешу

- Частный - эксклюзивный доступ ядра к кешу. Высокая скорость доступа, дублирование и конфликты
- Общий - общий доступ ядер. Меньше промахов, нет дублирований, но нужна постоянная синхронизация

Уровни кешей:

- L0 (опция) - специальный внутренний кеш процессора для стека, int/float чисел. Больше похож на регистр. Обычно доступен за такт.
- L1 - часть процессора, быстрый, обычно разделен для инструкций и данных. Обычно разделён (Banked). *Работают параллельно. Обеспечивает фишки Гарвардской архитектуры*
- L2 - часть процессора, обычно общий, От 128 Кбайт до 1-12 Мбайт.
- L3 - используются для синхронизации множества ядер, до 24 Мбайт и более. *Параллелизм*
- L4 - (экзотика для серверов и мейнфреймов), оптимизирует интерфейсы доступа к памяти

По идее программист вообще не должен думать о кэшах, НО:

- Meltdown
 - Перемножение матриц (предварительно транспонировать)
 - Массив структур или структура массивов
- L0 -- наиболее **специализированный кеш** (обычно размером 256 байт) который далеко не всегда выделяется в рамках процессоров. Как правило он имеет узкую специализацию, к примеру: предназначен для **кеширования стека, кеширования целочисленных переменных, кеширование чисел с плавающей точкой** для FPU (floating-point unit) и т.п. Как правило кеш данного уровня обеспечивает доступ к данным за один такт, что обеспечивается его непосредственной интеграцией в процессор.
- L1 cache (level 1 cache), **самый быстрый кеш**. По сути, он **является неотъемлемой частью процессора**, поскольку расположен на одном с ним кристалле и входит в состав функциональных блоков. В современных процессорах обычно L1 **разделён на два кэша**: 1) кэш команд (инструкций) и 2) кэш данных (в соответствии с Гарвардской архитектурой). Большинство процессоров без L1 не могут функционировать. L1 работает на частоте процессора, и, в общем случае, обращение к нему может производиться каждый такт. Зачастую является возможным выполнять несколько операций чтения/записи одновременно.
- Примечание: кэш команд может быть реализован на разном уровне, в частности для процессоров Pentium 4, отличающихся высокой частотой и сверхдлинным конвейером, кеширование производилось на уровне микроинструкций.
- L2 cache, который обычно, **как и L1, расположен на одном кристалле с процессором**. В ранних версиях процессоров L2 реализовывался в виде отдельного набора микросхем памяти на материнской плате. **Объём L2 от 128 кбайт до 1-12 Мбайт**. В современных многоядерных процессорах кэш второго уровня, находясь на том же кристалле, является памятью раздельного пользования -- при общем объёме кэша в n Мбайт на каждое ядро приходится по n/c Мбайта, где c -- количество ядер процессора.
- L3 может достигать **24 Мбайт и более**, медленнее предыдущих кэшей, но всё равно **значительно быстрее, чем оперативная память**. В многопроцессорных системах находится в

общем пользовании и предназначен для синхронизации данных различных L2, что особенно актуально для многоядерных систем поддерживающих параллельное программирование.

● L4, на сегодня весьма **экзотический вариант кеша**, применение которого оправдано только для многопроцессорных высокопроизводительных серверов и мейнфреймов. Обычно он реализован отдельной микросхемой. Как отмечают специалисты, по видимому, кеш L4 будет реализовываться при помощи eDRAM (что позволит обеспечить большой объём хранимых данных) и будет ориентирован в первую очередь на повышение пропускной способности к основной памяти, чем на быстрый доступ к данным.

Логика такова, что чем больше кэш, тем дольше время доступа к нему. Поэтому L1-кэш очень маленький и быстрый, в то время как L2 и L3 имеют больший объём, но и большую задержку.

Использование кэшей позволяет уменьшить среднюю задержку доступа к данным, так как в случае промаха в L1 кэш-подсистема проверяет наличие данных в L2 и L3 перед обращением к основной памяти, что снижает общее время доступа.

№28 Когерентность кеш-памяти. Возможные состояния кеш линий.

Механизмы обмена информацией между кешами: справочник, отслеживание и перехват. CAP теорема.

Кеш - промежуточный буфер с быстрым доступом, содержащий информацию, которая может быть запрошена с наибольшей вероятностью.

Когерентность - целостность данных между кешами и памятью (существует из-за параллельного доступа к данным (чтение/запись) множеством акторов (процессорные ядра, DMA)). Для поддержания когерентности каждая кеш-линия имеет свои состояния, отражающие записанные в нее данные. Одна из систем состояний - **MOESI**:

- Модифицировано - в линии свежие уникальные данные, не нужны запросы для модификации
- Владелец - только одна запись, могут быть другие данные в копии кеша или памяти
- Эксклюзивное - свежие данные в линии или памяти для одного ядра, но не для другого
- Разделяемое - свежие данные, могут быть копии в других структурах, нужна синхронизация
- Не актуальное - нет корректных данных

Пример состояний кеш-линий (MOESI)

1. **Модифицировано (Modified)**. L — свежие уникальные данные.
 - Модификация возможна без запросов.
 - $L \rightarrow Exclusive$ (запись в память).
2. **Владелец (Owned)**. Только одна запись в состоянии.
 - L — свежие данные, C — возможны, M — может устареть.
 - $L \rightarrow Modified (\forall C \rightarrow Invalid)$ или $\rightarrow Shared$ (запись в память).
3. **Эксклюзивное (Exclusive)**. L, M — свежие данные, C — нет.
 - $L \rightarrow Modified$ (запись в кеш) или $L \rightarrow Invalid$ в любой момент.
4. **Разделяемое (Shared)**. Содержит свежие данные, могут быть копии.
 - Запись запрещена, требуется $L \rightarrow Exclusive, \forall C \rightarrow Invalid$.
 - $L \rightarrow Invalid$ в любой момент.
5. **Не актуальное (Invalid)**. Не содержит корректных данных.

(L — кеш-линия, C — копия(-ии) кеш-линии в другом процессоре,
M — запись в памяти)

О состоянии кеш-линий следует как-то оповещать. Существует несколько вариантов, как это делать:

- Общий справочник источник информации - в процессор вводится блок, который хранит всю информацию об состоянии кеш-линий:
 - Перед загрузкой записи осуществляется проверка через справочник
 - При изменении каталог либо обновляется, либо аннулирует другие кешы с этой записью

Достоинства:

- Масштабирование
- Количество процессоров может быть велико

Недостатки:

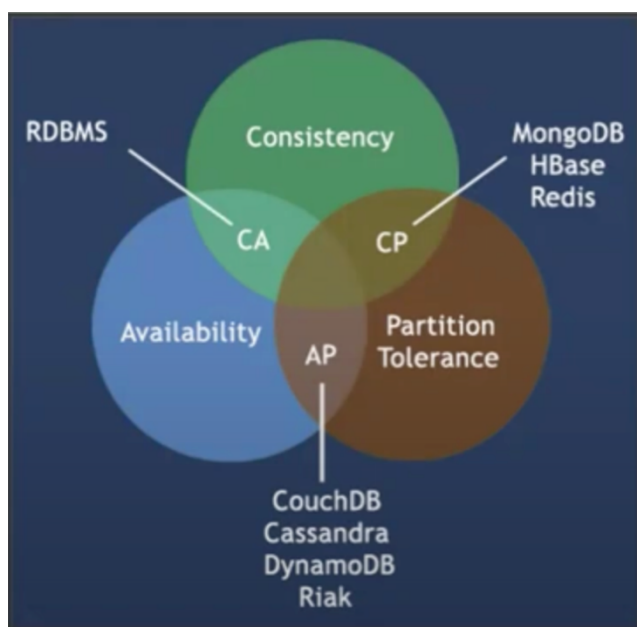
- Задержка запроса к справочнику
- Отслеживание (snooping шины) - отслеживание всех записей в память и их помечивание, сообщаются другому процессору
 - o Кешы отслеживают адресные линии на предмет обращений к данным своих кеш-линий
 - o Если наблюдается запись - кеш помечает линию как “не актуальную”
 - o Быстро, но ограниченная масштабируемость (нужно иметь столько входов, сколько входов у других процессоров)
- Перехват (snarfing)
 - o Как отслеживание, но при этом еще воруются данные для обновления линии
 - o Кешы отслеживают адресные линии и линии данных на предмет обращений к данным своих кеш-линий
 - o Если наблюдается запись - кеш обновляет кеш-линию.
 - o Дополнительная шина данных — сложно
- **Коротко snooping:** процесс следит за памятью на предмет изменения данных другими процессорами.

- **Коротко snarfing:** процесс следит за памятью на предмет изменения данных другими процессорами и ворует их, изменяя данные у себя

У нас такие же проблемы, что и при глобальном хранении данных на разных серверах датацентра

CAP теорема - утверждение о том, что в распределенных системах нельзя одновременно обеспечить:

- **Согласованность** - каждое обращение получает последнюю запись или ошибку
- **Доступность** - каждый запрос получает ответ (без ошибок), но и без гарантии, что он содержит последнюю запись
- **Устойчивость к разбиению** - модули могут работать обособленно, без связи друг с другом. *Не начнут жить независимой жизнью.* (система продолжает работать, несмотря на произвольное количество потерь в сети между узлами)



CAP теорема — это просто один из примеров фундаментального факта, что невозможно достичь одновременно безопасности и живучести в ненадежной распределенной системе.

По идее программист вообще не должен думать о кэшах, НО:

- *Meltdown*
- *Перемножение матриц (предварительно транспонировать)*
- *Массив структур или структура массивов*

Модифицирована (Modified)

Кэш-линия в модифицированном состоянии содержит наиболее свежие, корректные данные. Копия данных в основной памяти устарела и некорректна, и ни один другой процессор не имеет копии данных. Данные в кэш-линии могут быть повторно модифицированы без каких-либо запросов и изменений состояния. Состояние может поменяться на Exclusive при записи измененных данных в основную память.

Владелец (Owned)

Кэш-линия в состоянии *owned* содержит наиболее свежие, корректные данные. Состояние *Owned* похоже на состояние *Shared* тем, что другие процессоры могут иметь копию наиболее свежих и корректных данных. В отличие от состояния *Shared*, однако, копия в основной памяти может быть устаревшей и некорректной. Только один из процессоров может иметь данную кэш-линию в состоянии *Owned*, все остальные процессоры могут иметь эти данные только в состоянии *Shared*. Кэш-линия может перейти в состояние *Modified* после снятия актуальности (принудительного перевода в состояние *Invalid*) всех разделяемых копий в других процессорах, или в состояние *Shared* при записи измененных данных в основную память.

Эксклюзивное (Exclusive)

Кэш-линия в эксклюзивном состоянии содержит наиболее свежие, корректные данные. Копия в основной памяти также содержит наиболее свежую, корректную копию данных. Ни один другой процессор не имеет копии данных в своем кэше. Состояние может измениться на *Modified* в любой момент для модификации содержимого данной кэш-линии. Также состояние в любой момент может измениться на *Invalid*.

Разделяемое (Shared)

Кэш-линия в разделяемом состоянии содержит наиболее свежие, корректные данные. Другие процессоры в системе могут иметь копии данных в разделяемом состоянии. Копия в основной памяти также содержит наиболее свежую, корректную копию данных, если ни один другой процессор не имеет данной кэш-линии в состоянии *owned*. Запись в данную кэш-линию запрещена, и требует перевода её в эксклюзивное состояние с одновременным переводом всех остальных разделяемых копий в состояние *Invalid*. Также состояние может измениться на *Invalid* в любой момент.

Не актуальное (Invalid)

Кэш-линия в разделяемом состоянии не содержит корректных данных. Корректные копии данных могут быть либо в основной памяти, либо в кэше другого процессора.

№29 Закон Мура. Закон Деннарда. Закон Амдала. Power-wall. Memory-wall. Их роль в развитии компьютерных систем. Источники роста производительности процессоров тогда и сегодня.

Закон Мура - наблюдение, согласно которому количество транзисторов на кристалле интегральной схемы удваивается каждые 2 года. Иными словами, мощность производимых процессоров возрастала примерно в 2 раза (может быть больше, может быть меньше). Но этот закон перестал быть актуальным примерно с середины нулевых годов 21 века.

Стоит выделить 3 типа влияния закона Мура на мир:

- **соперничество разработчиков**, т.е. борьба за первенство в создании более быстрого и более мощного процессора;
- **разработка архитектуры вычислительных мощностей**: обновление технологических алгоритмов происходит постоянно и регулярно (раз в 2 года);
- **прогнозирование рынка**.

Начиная с 2010 года, следование закону Мура перестало быть выгодным для разработчиков. На его соблюдение нужно тратить множество ресурсов: материалы,

оборудование, увеличение штата и проч. На 2019 год закон Мура не работает эффективно, эра кремниевых транзисторов завершится предположительно до 2030 года.

Закон Деннарда - основа нашей вычислительной техники - транзистор. Данный закон рассматривает вопрос “А почему бы не уменьшить размер транзисторов?”. С одной стороны, это и логично. Меньше размер транзисторов - больше число транзисторов. Но не все так просто:

- Дороговизна дальнейшего уменьшения транзисторов - сложное оборудование
- Токи утечки - это физика (там еще квантовые и тепловые утечки)
- Power Wall и Dark Silicon - с большими токами утечки идет активный теплообмен, поэтому на плате процессора приходится оставлять пустые от транзисторов площади и, как следствие, мы ограничиваем мощность нашего процессора

Проще говоря, мы не можем бесконечно наращивать число транзисторов.

*Правило Деннарда закрепило уменьшение ширины проводника (техпроцесса) в качестве главного показателя прогресса в индустрии микропроцессорной техники. Но закон масштабирования Деннарда перестал действовать примерно в 2006 году. Количество транзисторов в чипах продолжает увеличиваться, но этот факт **не дает значительного прироста** к производительности устройств.*

*Например, представители TSMC (производитель полупроводников) говорят, что переход с 7-нм техпроцесса на 5-нм **увеличит** тактовую частоту процессора всего на 15%. Причиной замедления роста частоты, являются утечки токов, которые Деннард не учитывал в конце 70-х. При уменьшении размеров транзистора и повышении частоты ток начинает сильнее нагревать микросхему, что может вывести ее из строя. Поэтому производителям приходится балансировать выделяемую процессором мощность. В результате с 2006 года частота массовых чипов установилась на отметке в 4–5 ГГц.*

Закон Амдала - например, у нас есть алгоритм, и мы его начали параллелить. Данный закон гласит, что наращивание числа ядер даже при идеальном планировании все равно утыкает нас в потолок производительности. В этом законе учитывается процент распараллеливания и число процессоров (ядер).

Memory Wall - связано с различием в развитии скорости работы процессора и памяти: со временем развития скорость работы памяти увеличилась в пару десятков раз (если не меньше), а процессора - в тысячи. Мысль заключается в том, что наращивать скорость процессора дальше не имеет смысла, потому что мы не успеем загрузить в него данные из памяти.

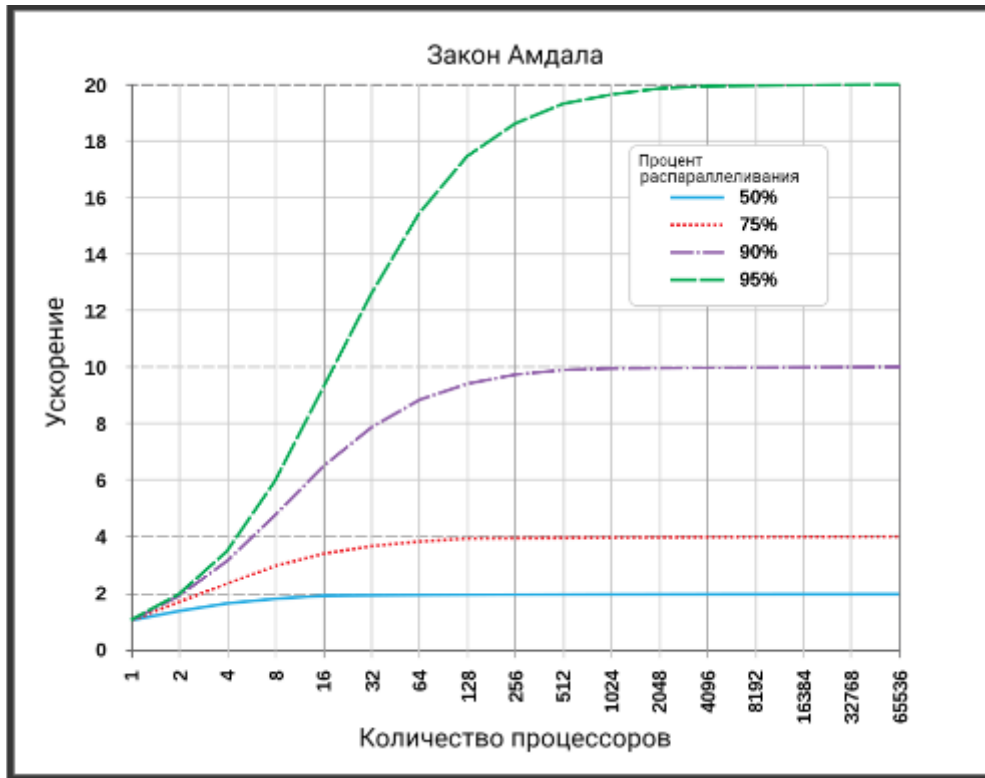
В то время источников производительности было достаточно много:

- Можно было ускорить работу памяти
- Можно было увеличить тактовую частоту процессора
- Можно было поменять архитектуру процессора (например, набор инструкций)

Формула ускорения при увеличении числа обработчиков (точнее реальное ускорение не будет превышать данного значения):

$$S_p = \frac{1}{\alpha + \frac{1 - \alpha}{p}}$$

p - кол-во процессоров, α - доля последовательной работы



Перспективы: уровень специализации (более узконаправленное приложение усилий к решению задачи), адаптации структуры (оптимизация алгоритмов планирования, систем прерывания или проще - повышение уровня параллелизма), реконфигурация

На данный момент производительность ищут не в совершенствовании процессоров общего назначения CPU, а в развитии специальных интегральных схем ASIC или появление вычислителей с параллельной архитектурой (GPU).

Также ставится вопрос выбора между временными (последовательное выполнение задач, как CPU) и пространственными вычислениями (работает, как конвейер, потоки - переплетаются). Но тут тяжело ответить, что лучше выбрать: в первом случае у нас все универсально, но медленно относительно второго, но во втором, понятное дело, есть свои трудности по согласованию задач.

№30 Проблема обеспечения реального времени в современных компьютерных системах. Влияние параллелизма уровня инструкций, языков программирования высокого уровня, многозадачности и организации памяти.

Система реального времени — система, которая должна реагировать на события во внешней, по отношению к системе, среде или воздействовать на среду в рамках требуемых временных ограничений.

Проблема обеспечения реального времени заключается в том, чтобы гарантировать, что задача будет выполнена в течение заданного времени. Это особенно важно для систем, в которых задержка или пропуск выполнения задачи может иметь серьезные последствия, например, для систем управления, систем безопасности или систем обработки сигналов.

Параллелизм уровня инструкций позволяет выполнять несколько инструкций одновременно. Это может повысить производительность системы, но может также привести к задержкам в выполнении задач реального времени. Это связано с тем, что инструкции могут выполняться в произвольном порядке, и система может не быть уверена, что задача будет выполнена в течение заданного времени.

Языки программирования высокого уровня могут сделать код более сложным и менее предсказуемым. Это может привести к снижению производительности системы и увеличению вероятности задержки в выполнении задач реального времени.

Многозадачность позволяет системе выполнять несколько задач одновременно. Это может повысить производительность системы, но может также привести к задержкам в выполнении задач реального времени. Это связано с тем, что система должна распределять ресурсы между задачами, и задача реального времени может быть поставлена в очередь за другими задачами.

Организация памяти может повлиять на производительность системы и вероятность задержки в выполнении задач реального времени. Например, если система использует виртуальную память, то задача реального времени может быть вытеснена из оперативной памяти другой задачей. Это может привести к задержке в выполнении задачи реального времени. Также можно привести в пример многоуровневые кеши и блокировку памяти ее обновлением (DRAM) и DMA

Для обеспечения реального времени в современных компьютерных системах используются различные методы. К ним относятся:

- 1. **Операционные системы реального времени (OS RB):** Специально разработанные операционные системы, которые обеспечивают гарантированное выполнение задач и обработку событий в строго определенных промежутки времени. OS RB допускают предсказуемость и быстроедействие даже в условиях высокой нагрузки.
- 2. **Планирование в реальном времени:** Применяются различные алгоритмы планирования для определения приоритетов и управления выполнением задач в реальном времени. Некоторые из них включают алгоритмы с фиксированным временным интервалом (Fixed Interval Scheduling), Rate-Monotonic Scheduling, Earliest Deadline First и другие.
- 3. **Аппаратная поддержка в реальном времени:** Некоторые компьютерные системы имеют специальную аппаратную поддержку, такую как сопроцессоры в реальном времени (Real-Time Coprocessors) или аппаратные таймеры, которые обеспечивают точное измерение и управление временными интервалами.

- 4. **Изоляция и приоритезация:** Для обеспечения реального времени, критические задачи могут быть изолированы и иметь приоритет над другими задачами в системе. Используются механизмы приоритетов, прерываний, блокировок и других методов, чтобы гарантировать выполнение важных задач в заданные сроки.
- 5. **Оптимизация производительности и латентности:** В компьютерных системах, где требуется реальное время, проводятся специальные оптимизации и настройки, чтобы уменьшить задержки и увеличить производительность. Это может включать оптимизацию кода, минимизацию использования внешних устройств с высокой степенью блокировки и т.д.
- 6. **Верификация и тестирование (Timed Input Output Automata):** Реальное время требует тщательной верификации и тестирования системы. Это включает использование средств проверки временной границы (Boundary Checking), анализ временного выполнения (Timing Analysis), симуляцию и другие методы, чтобы убедиться, что система отвечает требованиям реального времени.

Система реального времени — система, которая должна реагировать на события во внешней, по отношению к системе, среде или воздействовать на среду в рамках требуемых временных ограничений.

ПРЕДСКАЗУЕМО

Говорят, что система работает в режиме реального времени, если для описания работы этой системы требуются количественные временные характеристики

Проблемы:

- Привязка событий внутри системы к условным моментам времени
- Захват и освобождение системных ресурсов в строгих установленных рамках
- Синхронизация вычислительных процессов
- Буферизация потоков со всеми вытекающими
- Кеширование - без понятия, когда будут промахи
- Конвейер - без понятия, когда выполнится одна инструкция

Решение:

- Специальное оборудование (процессоры с оптимизацией и конфигурацией под условия выполнения в жестком реальном времени как ARM Cortex-R)
- ОСРВ (предсказуемое поведение при любом сценарии загрузки, в том числе прерывания и выполнения потоков) операционная система реального времени

Все влияния направлены на предсказуемость выполнения команд

Влияния:

- Явная иерархия памяти для предсказуемости доступа (есть шины от кпу ко всем)
- Организация памяти - **ВИРТУАЛЬНАЯ ПАМЯТЬ МИМО** в ОСРВ (ОС реального времени) подкачка/выгрузка на диск недопустимы, так как непредсказуемы по длительности. Paging должен быть частью контекста процесса (иначе непредсказуемость). Для систем жесткого реального времени применяется статистическое распределение памяти. Для мягких можно динамически, но без виртуальной памяти
- Многозадачность - **КООПЕРАТИВНАЯ** поддерживает многозадачность, приоритеты. При этом задержка прерывания (время между поступлением аппаратного прерывания и вызовом его обработчика) и задержка планировщика

(время между становлением задачи работоспособной и началом ее работы) должны быть как можно меньше. Взаимодействие между процессами - передача сообщений за постоянное время, поддерживает семафоры и мьютексы

- Параллелизм уровня инструкций - скорее всего про такие же ограничения на обработку инструкций для соблюдения строгого порядка между зависимыми инструкциями (как RAW WAW)
- КЭШ МИМО (нельзя предсказать)
- ЯП - необходима наивысшая производительность → выбор компилируемых языков (есть исключения как Java), выбор наиболее оптимизированных компиляторов, возможность описания параллельных процессов, их переключение на основе динамических приоритетов, их синхронизация, обмен данными, обработке прерываний и исключений, прямой доступ к внешним портам. Пример - ADA или C

№31 Уровневая организация компьютерных систем. Элементы уровня организации. Уровневый архитектурный стиль. Примеры. Явление разделения на уровни (disaggregation) и их смещения. Документирование инструментальных цепочек.

Уровневая организация - естественный способ организации вычислительной системы

Уровневый архитектурный стиль - стиль проектирования, в котором все модули, которые нам доступны мы разбиваем на независимые множества, где вышележащие множества могут обращаться к нижележащим

Пример: Есть бизнесовая компания, у которой есть отдел бизнес-логики и инфраструктуры. Ребята из отдела инфраструктуры реализуют модули, которые ложатся на нижний уровень, а работники отдела бизнес-логики на их основе эту самую бизнес логику реализуют на более верхнем уровне

Примеры:

- Lava flow - наслоение архитектурных решений
- Уровневый архитектурный стиль - наращивание идет постепенно: строим поверх языка фреймворк, затем прикладную программу и прочее. Если мы изначально качественно сделали такое уровневое разделение, то наша система будет поддерживаема и масштабируема
 - o Разбиваем модули на независимые множества, где вышележащие обращаются к нижележащим (но не наоборот) jvm → ось → железка
 - o Изоляция, модифицируемость, переносимость, управление сложностью, разделение интересов
- Модель ОСИ (не совсем подходит, так как работают как одно целое) - эта модель описывает то, как все процессы рассматриваются как единое целое из 7 уровней: если убрать один из уровней - другой перестанет работать
 - o Разделение на уровни обеспечивает модульность и вариативность
 - o Прикладной, представления, сеансовый, транспортный, сетевой, каналный, физический

- Уровни организации вычислительного процесса - при создании проекта учитывается каждый уровень его организации: определение требований, описание акторной модели, программирование на определенном языке и запуск на вычислительной машине - несмотря на то, что уровни могут крайне резко отличаться друг от друга (например, требования никак не запустить на процессоре), тем не менее, это все связано.
- В каждом уровне можно попытаться охарактеризовать, что из себя представляет отдельный уровень:
 - o Языки - на нем пишется вычислительный процесс (он же и модель вычислений)
 - o Типы данных - то, с чем мы можем оперировать: целочисленные, с плавающей точкой и прочие
 - o Инстансы - это те операции, который нам представляет сам язык (минимальное наполнение, с которым можно работать)
 - o Паттерны - собирают все в кучу, позволяют решать определенные задачи
 - o *Элементы уровня организации* - это язык программирования -if, cycle, базовые элементы, которые предоставляются самой платформой (types - int, tlong, instances - стандартная библиотека), а поверх этого паттерны и бест-практисы, которые позволяют дизайнить программы и решать задачи эффективно
 - o Все уровни взаимосвязаны и работают вместе, нельзя выкинуть какой-либо из уровней

Что касается такого процесса как разделение на уровни (Dissaggregation), то можно сказать следующее: на начале эпохи компьютерных систем определились крупные компании (например, IBM), которые сами проектировали и создавали свои компьютеры с нуля, т.е. они развивались на рынке “в высоту”: они не переживали насчет обратной совместимости, налаживания связей с другими компаниями и прочее - потому что они все делали сами. Но со временем рынок изменился - компании начали расти “в ширину” и стали лидерами в определенной ограниченной области (Linux, MacOS, Windows - на уровне ОС; Intel, AMD, ARM - на уровне чипов). *(Закон развития систем Сидого? - про разнообразие на верхних уровнях и ограничения на нижних).*

Параллельно с разделением уровней идет тенденция смешения уровней в сложных случаях. Например, такой подход используется для повышения производительности кода, так как нужно учитывать несколько уровней - если посмотрим на оптимизированный код, то там будет мешанина (используется, например, в управляющих компьютерных системах).

Разделение на уровни (дизагрегация) разделение рынка, масштабируемость

- Группировка компаний/производителей по уровням организации вычислительной системы - независимость уровней, модульность, многообразие
- Уровни существуют независимо, имеют совместимость
- За счет масштаба каждый уровень получается качественнее, т.к. вкладывается больше ресурсов в его создание

Смешение уровней (интеграция) пишем на C, делаем АСМ вставку

- Для достижения отдельных высоких характеристик система смешивает уровни.
- Из-за этого растет связанность и сложность системы
- Позволяет это делать инструментарий, исключая универсальность выходного модуля - но его оптимизированность под конкретный случай

Обычно ради производительности и безопасности приходится смешивать уровни. Из-за этого ломается абстракция, растёт сложность системы и процессов разработки

Закон Пробстинга мы не трогаем программу и она ускоряется сама по себе просто за счёт улучшения работы компиляторов

Документирование не требуется в случае небольших проектов, которые задействуют один уровень (например, Веб-проект, потому что все на одном уровне OSI). Для представления цепочки выч. процесса используется граф актуализации, который представляет все это дело в виде ориентированного графа (может описать как во время дизайна, так и использования).

Документирование: было бы здорово документировать подобные системы в виде графов, то есть мы документируем не уровни, а цепочку преобразований. Плюсы: есть в системе много уровней и взаимосвязей, то мы можем легко отобразить это на одном листе и находить начало и конец этих цепочек.

При помощи документирования мы можем показать систему, как набор представлений, задействованных для работы системы

- Граф актуализации - способ визуализации многоуровневых система. Реализация - цепочка преобразований моделей выч. процесса через стадии жизненного цикла или вычислительной платформы (high - low level - isa - hw - physic)
- Ориентированный ациклический граф, вершины — модели, рёбра — трансляции/интерпретации.
- Назначение: анализ инструментальных цепочек.

Документирование. Граф актуализации

1. Цепочка преобразований моделей выч. процесса через:
 - стадии жизненного цикла (Design → Dev → Use),
 - вычислительные платформы (High- → Low-level Langs → ISA → HW → Physic).
2. Ориентированный ациклический граф, вершины — модели, рёбра — трансляции/интерпретации.
3. Назначение: анализ инструментальных цепочек.



№32 Особенности реализации структурного программирования в фон Неймановских процессорах. Работа с памятью, регистрами. Реализация процедур. Реентерабельность. Рекурсия. Реализация условного оператора и циклов.

Языки высокого уровня - это то, чем чаще всего пользуются разработчики программного обеспечения. Это связано со многими причинами (например, читабельность программ и большой предоставляемый функционал: процессор умеет делать набор относительно простых инструкций, а мы хотим, чтобы это было посложнее).

Какая была идея? А давайте отойдем от природы работы процессора, его последовательного исполнения с джампами, к структурным блокам. В каноническом виде у каждого этого блока есть свой вход и выход (правда, такое сейчас не так хорошо проявляется - отсутствует jturn, или как там его).

ЯП высокого уровня - работа не на уровне инструкций, а работа на уровне структурных блоков, не инструкций, машинный код сильно отличается от синтаксиса, с которым работает программист

В целом любой высокоуровневый ЯП это:

- Естественные элементы для архитектуры фон Неймана:
 - o Последовательный код
 - o Условные операторы
 - o while, циклы
 - o подпрограммы
- (этого и ниже уже нет в ФН):
 - o Работа с регистрами
 - o Математические выражения
- Процедуры, область видимости и автоматическая память
- Исключения, состояния и перезапуски
- Полиморфизм, замыкания

- *Инструкции и переходы заменены на структурные блоки и их последовательности.*
- *Код имеет четкую структуру (блоки кода собираются в группы инструкций, отвечая некоторым правилам). Один выход из блоков*
- *Такие элементы, как последовательный код, условный оператор, циклы и подпрограммы стали естественными благодаря архитектуре фон Неймана*

Полиморфизм в языках высокого уровня - это концепция, которая позволяет объектам разных типов иметь одинаковое поведение или интерфейс. Это означает, что разные объекты могут реагировать на одинаковые вызовы методов или операции, но с разным поведением в зависимости от их конкретного типа.

Распределение регистров: переходя к высокоуровневым языкам мы получаем длинные цепочки выражений с n переменными и для этого в идеале нам нужно n регистров (а в процессоре их по 15-30 штук), получается есть задача распределения всех этих переменных (ведь если регистров меньше чем переменных, то мы уже идем в память). Решается все это с помощью раскраски регистров (ну то есть алгоритм раскраски графов)

Вторая проблема это **процедуры и подпрограммы**, потому что процессор дает мало инструментов для подобной работы. Все переходы замедляют наш проц и тп.

Ну из решений делать:

- inline (просто вставить код нашей функции вместо вызова, но если вызовов много, то у нас уже дофига инструкций, не хватит кэша, производительность тоже падает).
- goto (Дейкстра говорит, что это плохо + из-за прерываний может все сломаться и в функцию попадаем только один раз).
- call + return (тут уже получше появляется **реентерабельность (но по дефолту ее нет)**, можем переиспользовать, выделять статически память, однако могут быть утечки данных и возможно кто-то перепишет адрес возврата), в нормальных процессорах для этого используется стек (как в бэвм), запикиваем все туда и тем самым сохраняем наши данные.

Проблемы: при вызове функций ломаются кэши (сразу сильно много новых данных), регистры перезаписываются и могут переписать что-то важное, ломается конвейер.

Подпрограммы. Процедуры

1. Память выделена статически.
2. Реализация:
 - Через inline.
 - Через goto.
 - Через call, return.
3. Зачем:
 - переиспользование машинного кода,
 - оптимизация работы кэша инструкций (код процедуры).
4. Нет реентерабельности.
5. Проблемы: кеширование, сброс регистров, переходы и конвейер.

```
int* x, int* y;
void* SWAP_RET;
int tmp;

swap:
    tmp = *x;
    *x = *y;
    *y = tmp;
    goto SWAP_RET;
```

```
int* x, int* y;
int tmp;

void swap() {
    tmp = *x;
    *x = *y; // прерывание: isr()
    *y = tmp;
}

int a, b;

void isr() {
    a = 1;
    b = 2;
    swap(&a, &b);
}
```

Реентерабельность - можем ли мы вызвать функцию несколько раз

Реентерабельность. Рекурсивный вызов

1. Реализация через call, return.
 - Статическое выделение памяти для каждого входа.
 - Автоматическая память, стек: push, pop (рекурсия).
2. Зачем:
 - переиспользование машинного кода (run-time),
 - рекурсивные алгоритмы.
3. Проблемы: автоматическая память, утечки данных, перезапись адреса возврата.

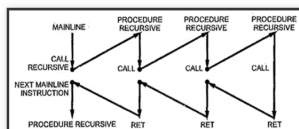
```
void fact(int n) {
    if (n == 0) return 1;
    return n * fact(n - 1);
}
```

```
struct swap {int* x; int* y, int tmp};

void swap(struct swap * base) {
    base->tmp = *(base->x);
    *(base->x) = *(base->y);
    *(base->y) = base->tmp;
} // напутал синтаксис?

struct swap enter1, enter2;
swap(&enter1);
swap(&enter2);

void swap(int* x, int* y) {
    int tmp;
    tmp = *x;
    *x = *y;
    *y = tmp;
}
```



Про реентерабельность тут стоит сказать про переиспользование подпрограмм (что динамическое выделение памяти лучше статического).

Про рекурсивные вызовы тут стоит сказать опять же про разницу в статическом и динамическом выделениях памяти: во втором случае мы можем спокойно вызывать из функции саму себя.

В плане реализации условного оператора - тут уже зависит от случая: как правило, это классический if, который либо пропускает часть инструкций, либо их выполняет, но существуют языки, в которых условные операторы могут быть разными (например, оператор when, unless и прочие), а также в некоторых ЯП можно реализовать свои условные операторы (в SmallTalk это делается через полиморфизм). К условным операторам можно отнести такую тему как ленивые вычисления (можно не считать полностью выражение, если оно уже заранее правда или ложь, либо применяется кеширование).

При выполнении математических операций, вызовов функций компилятор ставит перед собой задачу, типичную для низкоуровневых программистов - записать аргументы на физические регистры, но возникает вопрос, хватает ли нам их. Возникает еще один вопрос: а как согласовать использование регистров между вызовами процедур? Для такого придумали спецификации (Callee Saved и Caller Saved регистры). А также применяются алгоритмы на графах.

Следующая задача, которую стоит решить при разработке ЯП - это то, как мы работаем с подпрограммами. У любой подпрограммы должна быть своя выделенная память. Этот вопрос может решаться по-разному:

- 1. Статическое выделение памяти - при компиляции в коде создаются глобальные переменные, которые хранят параметры. Но проблема в повторном использовании - данные затираются.*
- 2. Если хотим переиспользовать подпрограмму, используется реализация через call/return, где память может выделяться либо статически (но тогда нужно статически выделять несколько блоков памяти, чтобы обеспечить сохранность данных), либо динамически (через стек; но тут тоже может быть проблема с утечкой памяти и перезаписью адреса возврата)*

Процедуры реализуются разными способами:

- inline*
- goto*
- call/return*

ФН нам не дает:

Распределение регистров (между командами):

- распределением регистров и выражениями - число регистров конечно, не все они одинаковы из-за разных задач. ЯП высокого уровня сами занимаются отображением значений на регистры (это становится проблемой компилятора или программиста низкого уровня).*

Решение проблемы - раскраска регистров (создание виртуальных регистров, которые будут отображаться на реальные), виртуальные регистры отображаются на физические, ищется НОД

они же (ЯП высокого уровня) привнесли

- подпрограммы, процедуры - нет поддержки подпрограмм и процедур из коробки. Для увеличения переиспользования кода и оптимизации работы кеша выносим процедуры в подпрограммы.
- области видимости, автоматическую память, исключения, полиморфизм

№33 Уровни параллелизма. Параллелизм уровня бит. Низкоуровневый параллелизм. Параллелизм уровня инструкций. Параллелизм уровня задач. Примеры и особенности.

Уровни параллелизма:

- Команд (задач) - про потоки, прерывания
- Инструкций - про последовательное выполнение нескольких инструкций на разных этапах их обработки процессором (конвейер)
- Битов - на уровне комбинационных схем - “ширина” комбинационных схем, шин данных и машинного слова (*про наличие нескольких дорожек на схеме*)

Параллелизм уровня битов (Bit-level Parallelism) достигается за счет увеличения “ширины” комбинационных схем, то есть за счет увеличения количества входных сигналов схемы.

Эта форма параллелизма основана на увеличении размера машинного слова. Увеличение размера машинного слова уменьшает количество операций, необходимых процессору для выполнения действий над переменными, чей размер превышает размер машинного слова (К примеру: на 8-битном процессоре нужно сложить два 16-битных целых числа. Для этого вначале нужно сложить нижние 8 бит чисел, затем верхние 8 бит и плюс учесть возможность переноса разряда от сложения нижних 8 бит. Итого 3 инструкции. С 16-битным процессором можно выполнить эту операцию одной инструкцией).

Низкоуровневый параллелизм - параллелизм на уровне инструкций (частично), скрытый от программиста компилятором или самим процессором. Не влияет на прикладное программирование (не знают друг о друге)

- Конвейеризация - разбиение выполнения инструкций на последовательность этапов
- Отделение ЦПУ от ввода/вывода (DMA)
- || узлы. Множественные функциональные узлы в ЦПУ - независимые функциональные блоки для арифметических и булевых операций, выполняемые одновременно (Суперскалярный процессор - процессор с несколькими АЛУ, которые работают параллельно (одновременно работаем с несколькими скалярными величинами))

Мы прячем от программиста это все дело

Параллелизм уровня инструкций - параллельное выполнение нескольких инструкций на одном вычислителе (разбиваем обработку инструкции на несколько этапов и выполняем параллельно для разных команд)

Суть в том, что мы представляем весь алгоритм в виде последовательно идущих команд. Из этого как бы следует что эти команды не могут быть выполнены параллельно. Но этой параллельности мы можем добиться по-другому:

- выполнение основной задачи команды (к примеру, деления) может требовать большого количества времени, а значит в процессоре будут присутствовать простаивающие элементы, которые можно загрузить другими задачами командами;
- Из этого следует решение - Разделить команду на составляющие и пустить так сказать по конвейеру - в каждый момент времени определенная часть инструкции будет выполняться только у одной команды.
- Кроме того, нужно помнить, что изменение порядка выполнения команд, зачастую, не изменяет их результата. Это и позволяет так делать

Конвейеризированное исполнение команд С точки зрения цифровой схемотехники, данный способ эксплуатирует возможность параллельной и независимой работы комбинационных схем разделенных регистрами, сдвиг конвейера реализуется по глобальному тактовому сигналу.

Современные процессоры имеют многоступенчатый конвейер команд. Каждой ступени конвейера соответствует определенное действие, выполняемое процессором в этой инструкции на этом этапе. Другими словами, процессор с N ступенями конвейера может иметь одновременно до N различных инструкций на разном уровне законченности. Классический пример процессора с конвейером - это RISC процессор с 5-ю ступенями: выборка инструкции из памяти(IF), декодирование инструкции(ID), выполнение инструкции(EX), доступ к памяти(MEM), запись результата(WB). Процессор Pentium 4 имеет 35-ти ступенчатый конвейер.

Параллелизм уровня задач - распределение задач между процессорами для их независимого выполнения (тут важно не путать с многозадачностью, когда много задач на 1 ядро, хотя они и лежат довольно близко). Реализуется путем выполнения на нескольких ядрах независимых задач. Зачастую этого мало, поэтому частью понятия является многозадачность - способ выполнения на одном вычислителе нескольких задач “параллельно” (на практике также последовательно, реализуется через разные виды многозадачности)

Распараллеливание на уровне задач нам демонстрирует операционная система, запуская на многоядерной машине программы на разных ядрах. Если первая программа показывает нам фильм, а вторая является файлообменным клиентом, то операционная система спокойно сможет организовать их параллельную работу.

канкаренси - одно ядро 2 задачи

многозадачность - два ядра две задачи

№34 Параллелизм уровня инструкций. Суперскалярные процессора. Особенности и принципы работы. Сравнение с VLIW и практика использования. Достоинства и недостатки. Барьеры памяти.

Параллелизм уровня инструкций - параллельное выполнение нескольких инструкций на одном вычислителе (разбиваем обработку инструкции на несколько этапов и выполняем параллельно для разных команд)

Например, у нас есть операция деления чисел с плавающей точкой: наш процессор/конвейер замрет на достаточное время - нам этого не надо. Для решения данной проблемы можно использовать:

- Суперскалярный процессор
- VLIW

Суперскалярный процессор включает себя отдельные блоки для специальных операций (например, для умножения и сложения). Диспетчер прозрачно для программиста определяет, куда пойдут инструкции и данные

Скалярная величина - величина, которая может быть представлена числом (целочисленным или с плавающей точкой).

Мы делаем независимые узлы, ставим инструкции в очереди, узлы их забирают и кладут результаты (reservation stations)

Суперскалярный процессор

- Идея - несколько параллельных алу, которые позволяют работать с несколькими суперскалярными величинами (величина, которую можно представить символом/числом)
- *VLIW - процессор, который обеспечивает параллелизм за счет создания больших инструкций, в которых содержится несколько действий сразу. Попадая в процессор они аппаратно разбиваются на несколько АЛУ.*
- Проблема - разные операции выполняются разное время, простаивание конвейера во время выполнения действий
- Решение - делаем независимые вычислительные узлы и процессор их параллелит через анализ потока инструкций
- Структура:
 - Извлекаем инструкцию в очередь команд
 - Декодируем и помещаем на станции резервирования
 - Станции резервирования переупорядочивают инструкции по мере доступности необходимых данных
 - Выполняют и кладут в буфер выполненных
 - Запись результата в память

Тут для структуры работы стоит добавить про буфер завершения: его задача восстановить линейный порядок инструкций, который был нарушен во время разделения по станциям.

- Достоинства

- o Рост производительности. Сглаживание длительности выполнения инструкций
 - o Повышение уровня нагрузки ресурсов
 - o Совместимость с существующим машинным кодом
 - o Компилятор может устранить значительное количество конфликтов за счет сортировки инструкций
- Недостатки
 - o Конфликты по данным оказывают значительное влияние на производительность и сложность процессора - как настроить компилятор, чтобы выстроил команды так, чтобы не было конфликтов
 - o Высокое энергопотребление
 - o Проблемы детерминированности работы многоядерных процессоров

Отличия VLIW от Суперскаляра - уровень, на котором мы передаем руководство Хардверу (у скаляра сразу, у влива после всех оптимизаций компилятора). Отличия VLIW от Суперскаляра - второе используется для более общих задач (условно), первое применяется широко во встроенных системах. (*структурированный поток (VLIW) vs. просто поток инструкций*)

Superscalar - лучше работает для систем общего назначения, потому что для нас как разработчиков процесс параллелизации работает в прозрачном режиме и подстраивается под ситуацию

VLIW - подходит для специальных применений, используется во встроенных системах. Так как можно написать хороший специализированный компилятор

Superscalar - лучше работает для систем общего назначения, потому что для нас как разработчиков процесс параллелизации работает в прозрачном режиме и подстраивается под ситуацию

VLIW - подходит для специальных применений, используется во встроенных системах. Так как можно написать хороший специализированный компилятор

Процессоры Pentium с микроархитектурой P5 стали первыми суперскалярными процессорами архитектуры x86.

Барьер памяти - инструкция для компилятора и CPU, что между барьером нельзя пересортировать инструкции. *В рантайме не нужны, но это в компайле (в отличие от суперскаляра?)*

Для решения детерминированности (однозначности при обращении к памяти) есть барьеры памяти - вид барьерной инструкции, которая приказывает компилятору и ЦП установить строгую последовательность между обращениями к памяти до и после барьера (гарантированно произведем все обращения к памяти до барьера перед первым обращением после барьера).

Какую проблему решает - процессор может менять порядок исполнения инструкций и испортить нам всю малину, вызвать неожиданное поведение. Барьеры памяти решают эти проблемы

№35 Параллелизм уровня инструкций. VLIW процессора. Особенности и принципы работы. Сравнение с суперскалярными и практика использования. Достоинства и недостатки. Барьеры памяти.

Параллелизм уровня инструкций - параллельное выполнение нескольких инструкций на одном вычислителе (разбиваем обработку инструкции на несколько этапов и выполняем параллельно для разных команд)

переносим параллелизм в СТ

Если RISC упростил процессор за счет упрощения набора инструкций, то VLIW это сделал благодаря реализации параллелизма инструкций в compile-time:

1. Объединим несколько инструкций в одну
2. Уберем из процессора механизмы перестановки инструкций
3. Компилятор имеет полный доступ к инструкциям → может “выжать” весь возможный параллелизм

VLIW - упрощение процессора за счет объединения нескольких независимых параллельных команд в одну связку. На этапе компиляции вписываем в эту инструкцию несколько параллельных действий (итог - разгрузка процессора от планировщика и перестановки инструкций)

Несколько инструкций → в одну. Уберем перестановки, компилятор выжмет весь возможный ||

- Достоинства
 - Упрощение процессора (выкидываем планировщик), снижение энергопотребления - процедуры и динамические библиотеки сложно анализировать. Конкретно место перехода к новому адресу
 - Упрощение декодера (уходит много схемотехники), частота повышается - каждый такт фетчим огромные инструкции и много данных для каждой из подкоманд
 - Компилятор имеет больше информации о коде и лучше параллелизует инструкции, выше потенциал для оптимизации
- Недостатки
 - Сложность написания таких компиляторов (из-за динамических библиотек, процедур из-за невозможности их оптимизации)
 - Высокая нагрузка на каналы данных и регистровые файлы (необходимо вынимать большое количество команд из длинных инструкций)
 - Конфликты конвейера приводят к простоям всех узлов - если данные недополучили, то придется приостанавливать весь конвейер
 - Низкая плотность кода (для простых команд полупустые сверхдлинные инструкции, которые нужно хранить и т.д.)
 - Ширина машинного слова влияет на микроархитектуру процессора (нельзя втиснуть больше или меньше команд - нарушение бинарной совместимости) - *Размер инструкции завязан на аппаратную часть в отличие от CISC. В cisc размер инструкции может быть любым (как я понимаю это в случае программного управления)*
 - *Не придумали пока таких компиляторов под VLIW (доходит даже до того, что все функции надо инлайнить) - мы создали специальный инструмент, которым мало, кто профессионально может пользоваться.*

Отличия VLIW от Суперскаляра - уровень, на котором мы передаем руководство Хардверу (у скаляра сразу, у влива после всех оптимизаций компилятора). Отличия VLIW от Суперскаляра - второе используется для более общих задач (условно), первое применяется широко во встроенных системах. (структурированный поток (VLIW) vs. просто поток инструкций)

В чистом виде архитектуру VLIW имеют процессоры TriMedia фирмы Philips и семейство DSP C6000 фирмы Texas Instruments. VLIW также получила хорошее распространение на рынке GPU, так, видеопроцессоры AMD/ATI Radeon начиная с R600 и до Northern Islands включительно имеют VLIW-архитектуру. (уже отошли)

Барьер памяти - инструкция для компилятора и CPU, что между барьером нельзя пересортировать инструкции. В рантайме не нужны, но это в компайле (в отличие от суперскаляра?)

Для решения детерминированности (однозначности при обращении к памяти) есть барьеры памяти - вид барьерной инструкции, которая приказывает компилятору и ЦП установить строгую последовательность между обращениями к памяти до и после барьера (гарантированно произведем все обращения к памяти до барьера перед первым обращением после барьера).

Какую проблему решает - процессор может менять порядок исполнения инструкций и испортить нам всю малину, вызвать неожиданное поведение. Барьеры памяти решают эти проблемы

№36 Классификация Флинна. Выделяемые классы и примеры машин этих классов. SIMT архитектура.

Это классификация параллельных процессоров. Классификация происходит по количеству потоков инструкций и количеству потоков данных. Если мы представим себе дефолтный ФН процессор это два потока - поток данных и поток инструкций.

У нас м.б. 1 или несколько потоков данных и 1 или несколько потоков инструкций. Их соотношения числа она и классифицирует.

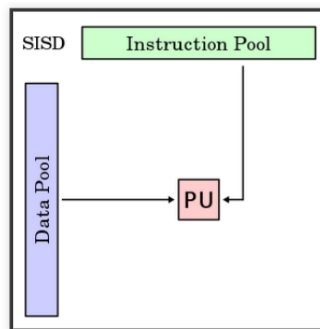
Достоинства:

- просто и понятно

Недостатки классификации:

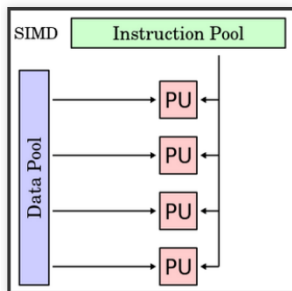
- плохо работает с не ФН (было создано до всего многообразия экзотик)
- Перегруженность класса (MIMD) (класс несколько процессов несколько потоков) - свалили все в одну кучу, внутри никакой четкой структуры (а-ля пусть у нас есть человек, все остальное - животные)

1. **SISD (Single instruction, single data)** 1 PU - process unit. Простой последовательный доступ, Абсолютное большинство рассмотренных ранее процессоров. Включая процессоры с низкоуровневым параллелизмом.



Пример: Это классический фон Неймановский процессор.

2. **SIMD (Single Instruction, Multiple Data)**. Мы увеличили кол-во PU - они работают в параллель



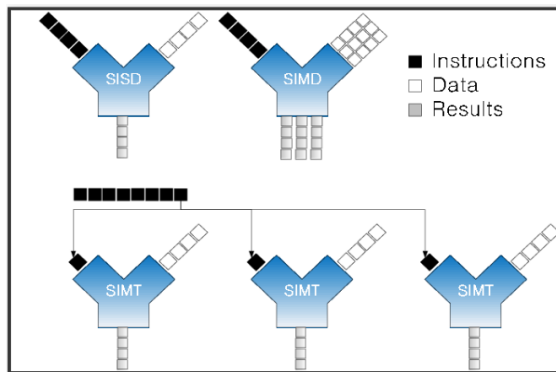
Использование: GPU, мат. задачи (расчет матриц), bitcoin block search (короче, много рядов данных, для которых нужно применить одну операцию)

Весь процессор работает в режиме lockstep (потому что один instruction pool) - одновременное выполнение всеми процессорами одной инструкции, поэтому не нужно писать много декодеров и управлять состоянием, т.к. Program Counter зафиксирован для всех процессоров

Назначение: однообразная обработка множества наборов данных. Ключевое ограничение: операции ветвления. Невозможно реализовать ветвление, так как оно будет зависеть от данных, а у нас все процессора должны обрабатывать ее одинаково

-: lockstep рассыпется, как только мы пойдем PU по разным адресам инструкций

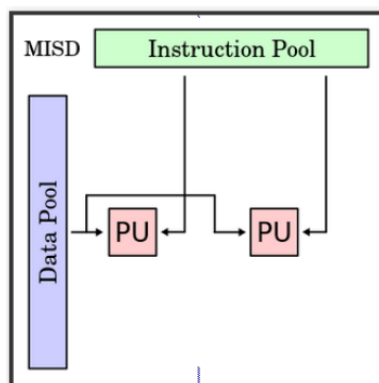
3. **SIMT (Single Instruction, Multiple Threads)**. Расширение предыдущей, не входит в классификацию, но обычно упоминается



Расширение SIMD, позволяющее работать с оператором ветвления. Очень часто используется в GPU/GPGPU из-за счет однотипных вычислений, не относящихся напрямую к графическому процессору (мат. моделирование)

У каждого проца личный PC (Program counter) и свое управление, в начале у нас тот же lockstep, но потом они могут разбежаться на условия, и потом нужно их собрать воедино (для синхронизации выходных данных). Включает механизмы синхронизации потоков между собой для достижения lockstep execution.

4. MISD (Multiple Instruction, Single Data). Очень специфический - созданный исключительно из-за классификации (искусственный).

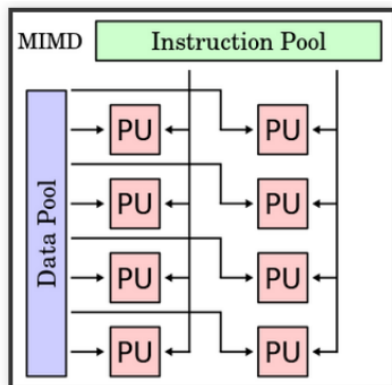


Суть - несколько потоков инструкций обрабатывают 1 поток данных

На практике — повышение надёжности работы систем. Защита от ошибок проектирования и алгоритмов. Параллельная разработка и эксплуатация решений задачи. *Параллельная разработка: Двоирование - два процессора выполняют один набор инструкций в режиме lockstep. Пока они работают идентично - все ок, иначе при разбиении выходных данных применяем аварийный протокол. Можно применить подход в управлении спутником или реактором - где нельзя ошибиться.*

Для избегания ошибок создаются несколько рабочих групп, они создают решение, у них уникальный код. Они ставят его работать в параллельно, и это позволяет нам защититься от ошибок программиста.

5. MIMD (Multiple Instruction, Multiple Data). Множество инструкций автономно выполняют различные инструкции над различными данными.



Примеры: современные микроядерные процессора - они работают относительно автономно и независимо со своими данными.

Проблема - все многопроцессорные архитектуры свалены в кучу этого типа классификации. На помощь пришла классификация Дункана.

№37 Классификация Дункана. Цели и задачи, классификация первого уровня. Синхронные архитектуры (векторные, SIMD, ассоциативные массивы, систолические). Принципы их работы и примеры.

Таксономия Дункана — это классификация компьютерных архитектур, предложенная Ральфом Дунканом в 1990 году.

Это классификация параллельных процессоров?

Не очень принята, потому что слишком перегружено, разнородно (разные уровни глубины), непонятно (без ее разбора)

Какие проблемы она решала?

- Как абстрагироваться от низкоуровневого параллелизма (из-за пограничных случаев классификации. Например, куда деть процессор vliw - по логике он SISD - один поток инстр, и один пул данных, но по организации он MIMD, тк. каждый PU имеет свою инструкцию внутри слота команды). Мы не рассматриваем низкоуровневый параллелизм, чтобы убрать это исключение - концентрируемся на систему команд и инструкции
- Переиспользуем таксономию Флинна
- Актуализировать и включить новинки на 1990 год - тогда эта классификация и вышла
- Навести порядок в MIMD.

+:

- относительная полнота - можно спокойно понять, что в ней происходит, как строятся вычислительные системы и как они могут функционировать

-:

- сложнее - как раз из-за полноты
- искусственность - выделяем класс не из-за фундаментального признака, а из-за явления, чтобы добавить исключение в классификацию

Классификация Дункана. Первый уровень

1. Синхронные архитектуры

Параллельные архитектуры с единым механизмом управления (глобальные часы, центральный блок управления и т.п.). Различаются механизмами синхронизации.

2. MIMD архитектуры

Параллельные архитектуры, которые могут исполнять независимые потоки инструкций. Потоки инструкций исполняются автономно, требуется динамическая синхронизация. Различаются структурой организации памяти.

3. MIMD парадигма (MIMD paradigm architectures)

Архитектура MIMD процессора определяется моделью вычислений (MoC). MoC определяет характер потоков данных, принципы взаимодействия и синхронизации. Различаются MoC.

В MIMD-архитектуре рассматривается конкретно аппаратная составляющая, а парадигма про то, как развивается и какова логика (модели вычислений).

Синхронные архитектуры - класс процессорных архитектур, обладающих единым/централизованным/синхронным управлением, определяющим поведение всего процессора в целом:

- Векторные архитектуры.
- SIMD архитектуры:
 - Процессорные массивы.
 - Ассоциативные массивы.
- Системные массивы.
- Совмещение памяти и вычислений

Векторная архитектура. Бывает тяжело и нужно выполнять простые арифметические операции. Почему бы не совершать эти операции пачкой (несколько раз подряд)? Основное преимущество - упрощение. Есть векторные регистры и вектор pipeline, в котором операции выполняются конвейерно. • *Векторные регистры + специальный векторный АЛУ*

1. Сокращение количества обрабатываемых инструкций.
2. Конвейерное исполнение векторных операций.
3. Множество функциональных узлов (суперскалярность).

Скалярная величина – величина, которая может быть представлена числом (целочисленным или с плавающей точкой). Вектор – величина, представленная последовательностью скалярных величин. Векторные операции — операции, выполняемые на последовательностях данных.

Пример: нам надо сложить все значения в векторе, мы говорим - у нас 10 значений, устанавливаем чтение векторных регистров, выполняем инструкции чтения векторных массивов и их сложения, после сохраняем в память. Это массивные инструкции, зато не требуется постоянная выборка инструкций и лишних декодирований, что оптимальнее.

SIMD-архитектуры как в классификации Флинна: Процессорные массивы (классический графический процессор). Выставляется одна инструкция и выполняется пачкой процессоров, у каждого из которых свой независимый поток данных. (CU генерит поток инструкций и тд)

Ассоциативные массивы. Позволяет сделать умную память. Благодаря этому позволяет проходить не по всей памяти. Строим вычислительные системы не вокруг процессора и комбинационных схем, а вокруг ассоциативной памяти. Работает по принципу Content Addressable Memory - поиск по памяти работает наоборот - есть массив данных, вы подаете ему на вход данные, которые не хотите найти и оно выдает их адрес. Зачем? Быстро понять, есть ли в массиве данных нужное. Сетевые устройства (есть ли какой-то адрес в таблице маршрутизации), кэш-память. Проходимся по строкам, если все значения в ячейках совпали – круто

Процессор, основанный на ассоциативной памяти (Content-addressable memory, CAM). Позволяет быстро искать информацию в массивах данных. Используется в сетевых устройствах.

Систолические массивы (типичный TPU). Хорошо работает при матричных вычислениях. Есть сеть процессоров, которые работают по глобальному тактовому сигналу. Сделал работу - передал дальше. и так по цепочке

Конвейерные мультипроцессоры:

1. данные ритмично поступают из памяти,
2. проходят через сеть процессоров,
3. и возвращаются в память.

Синхронизация — через глобальные часы, такт вычислений (не путать с тактовым сигналом).

Процессоры объединены локальными связями, и каждый цикл процессоры выполняют короткие неизменные операции.

Совмещение памяти и вычислений. Продолжаем идею ассоциативных массивов, но не для поиска, а хотим решать сложные арифметические операции. Загрузили в память параметры, а потом начали загружать входную матрицу - идет активация и вычисления, после которых идет обновление состояния массива в памяти. По сравнению с систолической матрицей тут простой интерфейс - просто работа с памятью.

Пример: перемножение матриц в нейронных вычислителях. $Out = In \times Parameters$

- параллелизм операций умножения (векторные операции);
- минимизация числа передач данных от процессора (вторая матрица загружается 1 раз);
- привычный интерфейс (работа с памятью).

Использование: мат. расчетов, это жесткие структуры, которые должны быть сбалансированы, сложно планировать

№38 Классификация Дункана. Цели и задачи, классификация первого уровня. MIMD архитектуры (распределённая и разделяемая память). Примеры и принципы их работы.

Таксономия Дункана — это классификация компьютерных архитектур, предложенная Ральфом Дунканом в 1990 году.

Это классификация параллельных процессоров?

Не очень принята, потому что слишком перегружено, разнородно (разные уровни глубины), непонятно (без ее разбора)

Какие проблемы она решала?

- Как абстрагироваться от низкоуровневого параллелизма (из-за пограничных случаев классификации. Например, куда деть процессор vliw - по логике он SISD - один поток инстр, и один пул данных, но по организации он MIMD, тк. каждый PU имеет свою инструкцию внутри слота команды). Мы не рассматриваем низкоуровневый параллелизм, чтобы убрать это исключение - концентрируемся на системе команд и инструкции
- Переиспользуем таксономию Флинна
- Актуализировать и включить новинки на 1990 год - тогда эта классификация и вышла
- Навести порядок в MIMD.

+:

- относительная полнота - можно спокойно понять, что в ней происходит, как строятся вычислительные системы и как они могут функционировать

-:

- сложнее - как раз из-за полноты
- искусственность - выделяем класс не из-за фундаментального признака, а из-за явления, чтобы добавить исключение в классификацию

Классификация Дункана. Первый уровень

1. Синхронные архитектуры

Параллельные архитектуры с единым механизмом управления (глобальные часы, центральный блок управления и т.п.). Различаются механизмами синхронизации.

2. MIMD архитектуры

Параллельные архитектуры, которые могут исполнять независимые потоки инструкций. Потоки инструкций исполняются автономно, требуется динамическая синхронизация. Различаются структурой организации памяти.

3. MIMD парадигма (MIMD paradigm architectures)

Архитектура MIMD процессора определяется моделью вычислений (MoC). MoC определяет характер потоков данных, принципы взаимодействия и синхронизации. Различаются MoC.

В MIMD-архитектуре рассматривается конкретно аппаратная составляющая, а парадигма про то, как развивается и какова логика (модели вычислений).

В MIMD-архитектуре ключевой вопрос - организация памяти:

- Разделяемая - единое пространство, на которое реагируют все процессора (аналогия - общая база данных)
- Распределенная - каждый работает со своими данными (аналогия - микросервисная архитектура)

Это про то, когда у нас нет единого тактового сигнала (как в синхронных архитектурах) и каждый процессор может жить своей жизнью.

1. Распределенная память у процессоров

Своя память у каждого процессора

Координация совместной работы через прямое взаимодействие процессоров. Данные/сигналы явно передаются между процессорами (message passing).

Мотивация: обеспечить масштабируемость многопроцессорной архитектуры и доступ к локальным данным процессора.

Пример: современные облака

Топологии (как организованы связи между):

- кольцо - если мало процов и редко нужна синхронизация. Как работает: передает по кольцу данные до тех пор, пока они не вернутся в нему
- ячеистая - в теме про стековую архитектуру был такой военный вариант системы. Как работает: каждый процессор - стековый асинхронный и матрицей взаимодействует с соседними. Реализуют сложные параллельные обработки
- дерево - центральный процессор спускает задачи на помощников. Хорошо для иерархических структур
- гиперкуб - маленький коммуникационный радиус (сколько пересылок нужно для доступа к соседу).
- реконфигурируемые - в процессе работы системы формируем нужную топологию.

1. Данные передаются по кольцу - рано или поздно данные дойдут
2. Ячеистая архитектура встречалась в Forth-процессоре, где процессора выстраивались в виде матрицы - легко реализуется
3. Дерево

4. Куб
 5. *Реконфигурируемые сети - сами выстраиваем трассы и потом их же используем*
2. Разделяемая память

Каждый процессор работает независимо - у него своя память (ни с кем он ее не делит). Но возникает вопрос по межпроцессорному взаимодействию (например, каналы).

1. Координация совместной работы через общую память.
2. Ключевая проблема: координация доступа, согласование кешей.
3. Примеры подходов:
 - (a) шинное соединение;
 - (b) перекрестная шина 2x2;
 - (c) сеть на кристалле, запрос маршрутизируется от Р к М.
4. Широко распространены, так как привычны и прозрачны для программистов.

Общение через:

- *Через шину (a)*
- *Через crossbar (b) - процессора могут параллельно обращаться к памяти, если нет конфликтов*
- *Через коммутацию (c)*

Пример: Наши компы и телефоны (широко распространены)

Здесь все проблемы многопоточки и синхронизации. Можно по-разному организовать шины: обычное (a), кроссбар(б), сеть на кристалле с маршрутизацией(с)

№39 Классификация Дункана. Цели и задачи, классификация первого уровня. MIMD парадигмы (MIMD/SIMD, потоки данных, редуционные, wavefront). Примеры и принципы их работы.

Таксономия Дункана — это классификация компьютерных архитектур, предложенная Ральфом Дунканом в 1990 году.

Это классификация параллельных процессоров?

Не очень принята, потому что слишком перегружено, разнородно (разные уровни глубины), непонятно (без ее разбора)

Какие проблемы она решала?

- Как абстрагироваться от низкоуровневого параллелизма (из-за пограничных случаев классификации. Например, куда деть процессор vliw - по логике он SISD -

один поток инстр, и один пул данных, но по организации он MIMD, тк. каждый PU имеет свою инструкцию внутри слота команды). Мы не рассматриваем низкоуровневый параллелизм, чтобы убрать это исключение - концентрируемся на систему команд и инструкции

- Переиспользуем таксономию Флинна
- Актуализировать и включить новинки на 1990 год - тогда эта классификация и вышла
- Навести порядок в MIMD.

+:

- относительная полнота - можно спокойно понять, что в ней происходит, как строятся вычислительные системы и как они могут функционировать

-:

- сложнее - как раз из-за полноты
- искусственность - выделяем класс не из-за фундаментального признака, а из-за явления, чтобы добавить исключение в классификацию

Классификация Дункана. Первый уровень

1. Синхронные архитектуры

Параллельные архитектуры с единым механизмом управления (глобальные часы, центральный блок управления и т.п.). Различаются механизмами синхронизации.

2. MIMD архитектуры

Параллельные архитектуры, которые могут исполнять независимые потоки инструкций. Потоки инструкций исполняются автономно, требуется динамическая синхронизация. Различаются структурой организации памяти.

3. MIMD парадигма (MIMD paradigm architectures)

Архитектура MIMD процессора определяется моделью вычислений (MoC). MoC определяет характер потоков данных, принципы взаимодействия и синхронизации. Различаются MoC.

В MIMD-архитектуре рассматривается конкретно аппаратная составляющая, а парадигма про то, как развивается и какова логика (модели вычислений).

MIMD-парадигма - имеется в виду парадигма программирования или точнее - модель вычислений (MoC)

Разбираем парадигмы

1. Гибридная MIMD/SIMD архитектура

Гибридные архитектуры, где управление MIMD (малые задачи) осуществляется в стиле SIMD (крупные задачи), т.е. в целом система является simd, но отдает задачи на исполнение mимd процессорам с локальной памятью

Отношение ведущий — ведомый (master — slave) может быть представлено деревом:

корень работает как SIMD и распределяет задания;

листья работают как MIMD с локальной памятью.

Хорошо работает динамически в runtime, похож чем-то на суперскаляр. Может выделить общий корень, но не синхронно, а просто он задает начало процессу.

Аналогия: fork/join.

Диспетчер виртуальной машины. Есть абстрактный управляющий контроллер, поручающий задания подпроцессорам и работающий как SIMD, и физический исполняющий, отчитывающийся начальнику и работающий по принципу MIMD с локальной памятью.

Каждый вычислительный блок сам знает, что делать, просто ждет данные. Диспетчер шины - управляющий блок, а внизу процессора

2. Потокосовая архитектура (dataflow)

Аналог как считали артиллерийские таблицы

Потоковые архитектуры. Возможность выполнения и исполнения инструкций определяется исключительно наличием входных аргументов для инструкций.

1. Идея: вычисления в виде ориентированного графа, где каждая вершина - какое-то действие. Выполняется до нахождения листа, из которого получаем результат. Порядок выполнения инструкций при этом непредсказуем. Отсутствует память и канал доступа как "бутылочное горлышко".

Потоковые машины статически формируют граф и по нему идут данные

Ждет, пока получит два нужных ему значения и идет выполнять свою работу, передает дальше

Отличия от систолической (сеть процессоров, глобальный тактовый сигнал) :

- о не требуется общего такта управления - взаимодействие самосинхронное, динамически устанавливается синхронизацию
- о не требуется однозначная завязка аппаратной структуры на алгоритм

Отличие от систолической в том, что мы не завязаны программно на тот алгоритм, который выполняем и не нужен общий такт управления, они динамически синхронизируются в рантайме.

+:

- о Высокий уровень параллелизма (каждый узел может работать в параллель)
- о Память локализована вокруг узлов (нет промежуточных сохранений)

Проблема - передача данных между узлами (как сделать это эффективно и оптимально)

Решение - реконфигурируемая топология под нужный алгоритм или кольцо (передаем токен с информацией в виде пары key, value. Сохраняет и сканирует и запускает функции, которые может (если к нему) и передает дальше по сети)

Проявляется в стримах джавы.

Устройство:

1. *Распределённая память. Топология кольцо.*
2. *По кольцу посылаются токены (метка + значение).*
3. *Получатели пересылают токен дальше и сохраняют значение, если являются адресатом.*
4. *Если задача разблокируется (все необходимые токены получены) — запускается работа.*
5. *Результат работы отправляется токеном по кольцу.*

3. Редукционная

Редукционные архитектуры (ориентированные на запрос). Инструкции разрешаются к выполнению, когда их результаты требуются в качестве операндов для уже разрешенных инструкций.

1. Базируется на Функциональном Программировании, что мы считаем не операции в узлах, а мат. выражения. Выполнение: выборка инструкций, замена инструкции на её результат в графе вычислений. Проблемы: ссылочная прозрачность, разворачивание.

Принципиальная разница с прошлым: в потоковой у нас статический граф, что отображается на вычислениях, а в редукционной у нас символьные выражения (решаемая задача), мы можем найти необходимое подвыражение и посмотреть, от чего оно зависит. Далее мы можем зависимости считать параллельно.

Потоковая архитектура заключалась в том, что данные продавливались от поступления данных до получения конечного результата. Редукционная архитектура делает то же самое, но эта архитектура направлена на разделяемую память (по сравнению с распределенной в потоковой).

Вычисления триггерят необходимость отдать результат на следующий уровень, в отличие от потоковой архитектуры, где вычисления идут от корня к листьям, здесь они идут от листьев к корню

Проявляется в функциональных языках с ленивыми вычислениями (например, хаскел).

Здесь необходимо сделать эффективно. Пример - одинаковые операции детектить на уровне аппаратуры (для избежания повторений) - что сложно

Реализация внутри - на уровне ЯП, виртуальных машин и рантаймов

Когда мы проводим вычисление контрол флоу нам не интересен Нам неважно где и что вычисляем и тут противопоставить можно потоковой: в ней есть абстрактный ход данных и его проталкивают вперед А редуцирующая такая, но шиворот навыворот Надо получить результат Например: регистр нужно записать сколько диодов зажечь Чтобы посчитать надо построить структуру данных и развернуть граф Поточковые машины статически формируют граф и по нему идут данные А редуцирующие запрашивают данные откуда они идут Процессор каждый раз видит, что ему надо результат Он смотрит операцию и проверяет есть ли аргументы Он вычисляет и удаляет вершину графа и потихонечку сворачивает Если в вершине а плюс б но они не вычислены Значит вычислитель выставляет запрос что ему надо и просит у узлов **Узлы работают независимо** Тем самым разворачивается в памяти этот граф вычислений Когда доходим до листиков, то сворачиваемся За счет того, что много наблюдателей за графом = параллелизм Если какие-то вычисления в реальности не нужны но они так написаны просто потому что В одной ветке ифа используется ф1, а в другой нет. Значит мы просто уберем его в иф **Лениво запрашиваются данные из-за этого оптимизация** Недостаток

- o Надо четко понимать чтобы не плодить вершины потому что когда прописываются вычисления (если аргумент макросу подать выражение себя то их станет двое это неприятно и тут то же самое)
- o Сложная структура памяти графа и надо эффективно ее поддерживать
- o Сегодня на уровне аппаратуры мало кто делает но на программном это довольно часто. ФП это программно строит (хаскель) Стримы ленивые коллекции это про то же Функциональные модели - модели на уровне символьных преобразований Левое равно правое вот это то же самое Мы смотрим на выражения

А редуцирующая это идеи засунули в железку

4. Wavefront

Гибрид систолической архитектуры (сеть процессоров, глобальный тактовый сигнал) с асинхронным потоковым исполнением. Блоки сами должны синхронизироваться между собой, но при этом у нас четкая структура данных и как только выполняется работа, мы прокидываем дальше имеющиеся результат

Очень похоже на систолическую архитектуру, но тут продавливание данных идет не по синхронному такту, а после того, как сам процессор что-то посчитал (у блоков есть свои входные/выходные буферы и они динамически синхронизируются).

1. Управление происходит динамически, за счёт распространения сигнала по сети.
2. Синхронизация и режим работы выстраиваются динамически.
3. Проще в программировании. Лучше масштабируется.
4. Риски переполнения буферов в Run-Time.

Очень тяжело выйти из ловушки Фон Неймана (популярность, обратная совместимость)

Но для специализированных задач такие процы - окэй

№40 CGRA процессора. Особенности и место в индустрии.

Пространственные (spatial) и временные (temporal) вычисления. Примеры и принципы их работы.

Классификация Дункана хороша, но не дает четкого пояснения того, как должны выглядеть сами процессора. По этой причине появилась:

CGRA (Coarse-Grained Reconfigurable Architecture)

Выводы из классификаций Флинна и Дункана:

1. Огромное разнообразие подходов к построению процессора.
2. Противоречие между пространственными и временными вычислениями (Spatial & Temporal Computation).
3. Противоречие между универсальностью и эффективностью.

Выделяется ниша:

- Domain-Specific Flexibility ("Предметно-специально гибких") имеется в виду, что выделяются устройства, которые гибко справляются в определенной области (например, шейдеры)
- Сочетающих пространство и время - ключевой вопрос, касающийся временных и пространственных вычислений (нужно уметь варьировать)
- Управляемых конфигурацией и данными (Configuration / Data-driven execution).

Заполняется крупно-гранулярно реконфигурируемыми архитектурами — **CGRA**.

CGRA архитектура определяет (снизу-вверх):

1. вычислительные блоки доступны и с какими интерфейсами (данные/управление);
2. коммуникацию между вычислительными блоками;
3. механика управления процессором;
4. механика планирования вычислительного процесса;
5. программы и программирование.

Мы можем совмещать temporal (временные) и spatial (пространственные) вычисления.

Temporal computation – схема последовательно выполняет требуемые задачи. Spatial computation – схема повторяет граф вычислений (параллелизм, потоки, конвейер)

Пространственные вычисления фокусируются на использовании параллельности и разделении задачи на множество небольших подзадач, которые могут выполняться одновременно. Они используют архитектуры, которые предоставляют большое количество вычислительных ресурсов, доступных одновременно, для одновременного выполнения различных частей задачи.

Принципы работы

1. **Параллельность.**
2. **Локальность данных.**
3. **Конвейеризация.**

Примеры пространственных вычислений

1. **Графические процессоры (GPU).**
2. CGRA

Временные вычисления

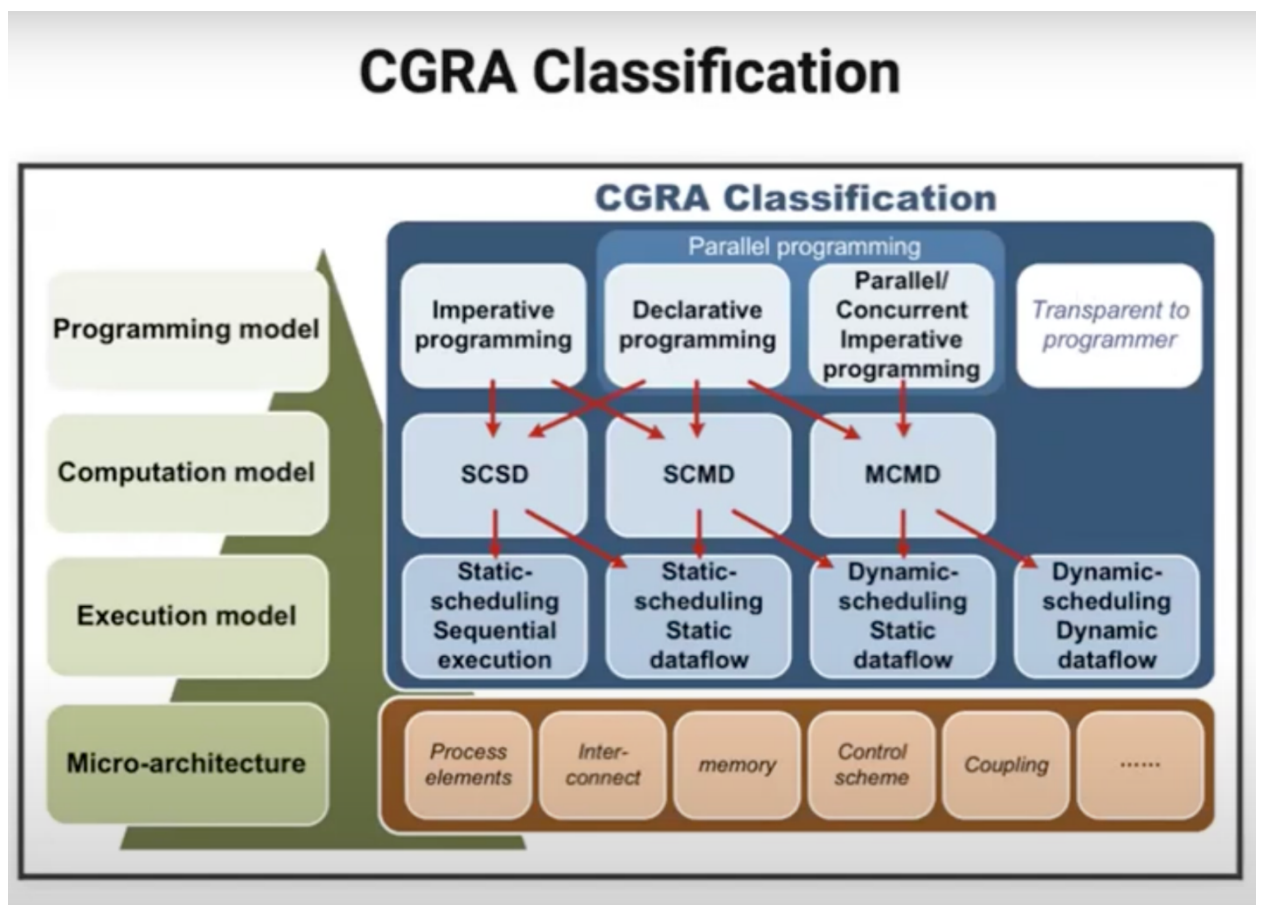
Временные вычисления основаны на последовательном выполнении задач, где одна задача выполняется в определенный момент времени, а другие задачи ждут своей очереди. Они используют архитектуры, которые оптимизируют использование времени выполнения для максимальной эффективности.

Принципы работы

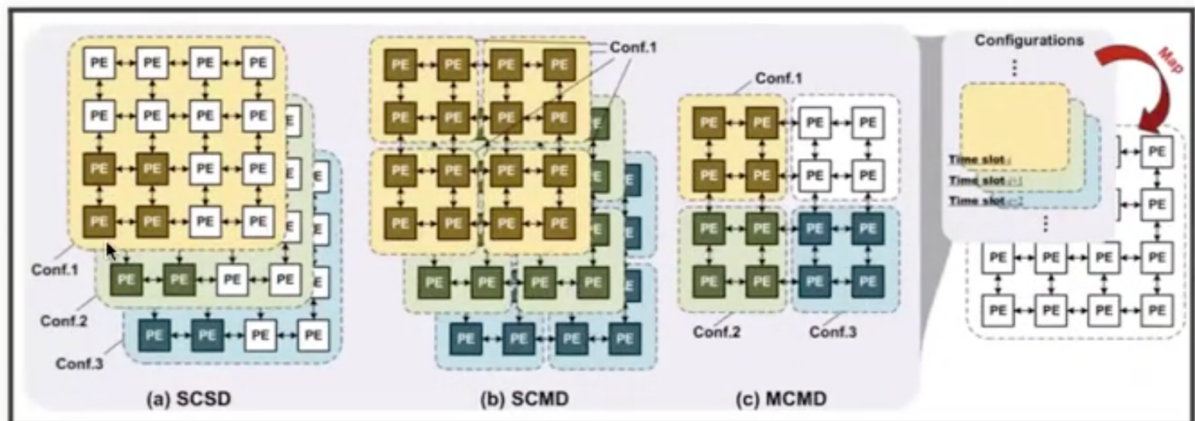
1. **Последовательность выполнения.**
2. **Управление потоками данных:**
 - Потоки данных управляются с помощью регистров и буферов, чтобы оптимизировать последовательное выполнение задач.

Примеры временных вычислений

1. **Центральные процессоры (CPU).**
2. **Цифровая обработка сигналов (DSP).**



CGRA. Computation model



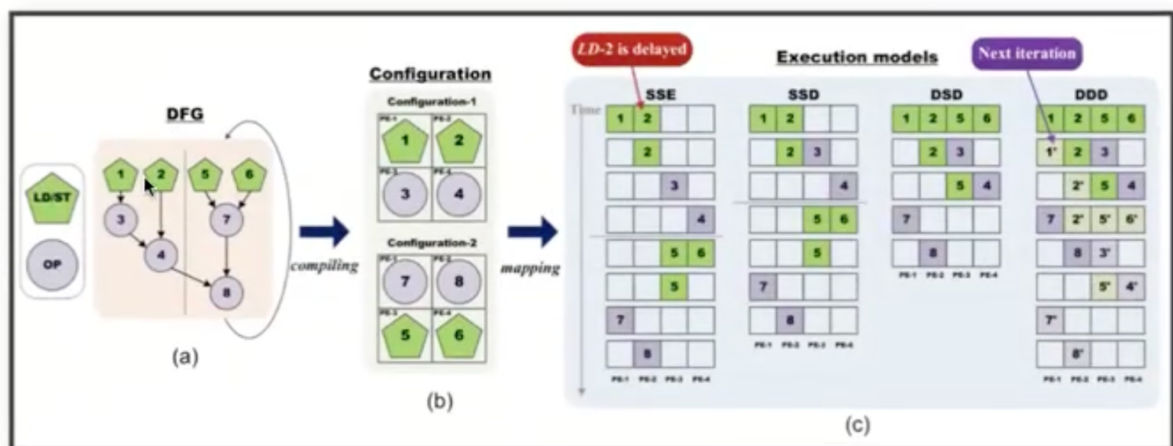
- **SCSD** — single configuration, single data
- **SCMD** — single configuration, multiple data
- **MCMD** — multiple configuration, multiple data

SCSD - записана только одна конфигурация и обрабатывается один поток данных

SCMD - записана только одна конфигурация и обрабатывается несколько потоков данных

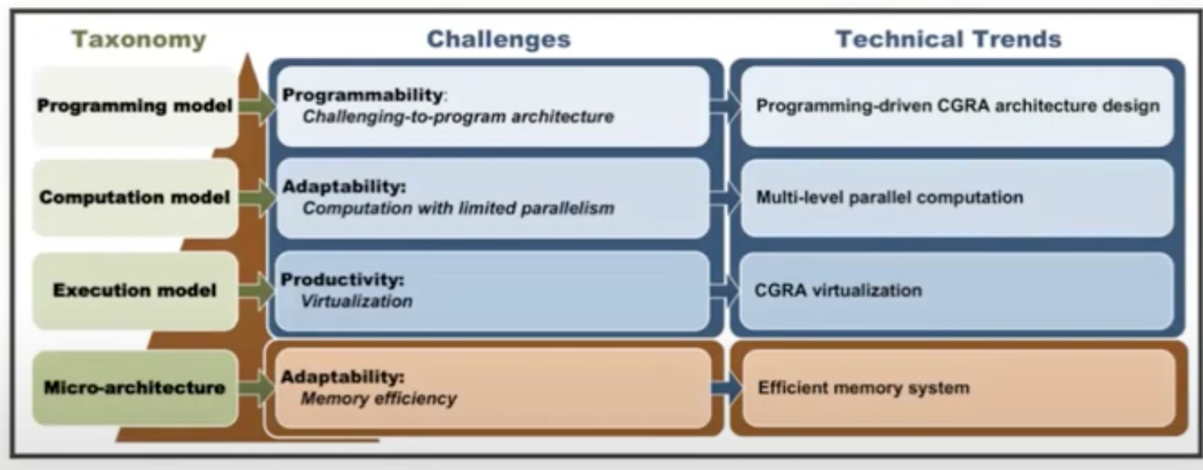
MCMD - записано несколько конфигураций и обрабатывается несколько потоков данных

CGRA. Execution Model



Зеленое - работа с памятью, серое - наша задача

CGRA: Challenges & Trends



1. Как писать программы? - Отталкиваться не от архитектуры, а от самого программирования
2. Адаптивность (как совмещать разные задачи) - умение совмещать несколько конфигураций
3. МоС - вопрос виртуализации
4. Эффективность памяти - увеличение за счет непосредственного взаимодействия процессоров

ДОП

Микропроцессор - цифровая схема, которая выполняет операции с внешним источником данных (обычно памятью или потоком данных).

Микроконтроллер - микросхема, предназначенная для управления электронными устройствами.

Типичный микроконтроллер сочетает функции процессора и периферийных устройств, содержит ОЗУ и (или) ПЗУ.

Оперативная память (память) обычно часть компьютерной памяти, в которой хранится выполняемый машинный код, а также входные, выходные и промежуточные данные.

Аппаратное обеспечение - электронные и механические части вычислительного устройства, входящие в состав системы или сети, исключая программное обеспечение и данные (информацию, которую вычислительная система хранит и обрабатывает). Аппаратное обеспечение включает: компьютеры и логические устройства, внешние устройства и диагностическую аппаратуру, энергетическое оборудование, батареи и аккумуляторы.

производство + дока = изделие

Особенности производства:

- Тенденции на интеграцию (уменьшить S, увеличить кол-во элементов), уменьшение затрат на единицу и усложнение цепочки,
- Нет права на ошибку - иначе проходим очень большой цикл заново

Монтаж (глобальный):

- Навесной (соединение на изолирующем шасси радиоэлементов проводами или выводами) Плюсы:
 - о Просто в производстве
 - о Просто в подготовке производства
 - о Гибкость реализации
 - о
 - эпоксидка = стабильность выше
 - о костыль для исправления других плат

Минусы:

- о Дорого в серии, сложно в автоматизации
 - о Не надежно
 - о Зависимость от конкретного исполнения
- **Монтаж на печатную плату** (печатная плата - пластина из диэлектрика, в объеме или на поверхности которой сформированы электропроводящие цепи электронной схемы). Бывают одно - двух-слойные и многослойные. Фиксирует компоненты на **жесткой раме**, что повышает надежность.

Виды монтажа компонентов:

- Выводной (в отверстие, пайка штыря в отверстия платы)
- Поверхностный (пайка на поверхностный контакт компонента)
- Встроенный (встраивание части компонентов в одном из слоев внутри печатной платы)

Плюсы

- Автоматизация процесса создания
- Высокая плотность размещения на плате
- Стоимость серии низкая за счет автоматизации
- Размеры (маленькие)

Минусы:

- Подготовка производства
- Удлинение производственной цепочки (много этапов подготовки и создания)
- Сложность исправления

Гибкость благодаря:

- Джамперы (ключики, замыкающие или размыкающие линии на плате)
- Перерезать дорожку скальпелем
- Навесная дорожка сверху

Прикол про два процессора и систематическую ошибку из-за длины шин

Интегральные схемы - это схемы изготовленные из кремниевом песке

Очень сложный этап производства, берем песок, из него выращиваем кристалл, нарезаем на ваферы, на них наносим вещество и потом через линзу пускаем свет, вырезаем мелкие интегральные схемки и корпусулируем их на одну плату. Это стоит очень дорого

dark silicon - часть чипа для отведения тепла

Производство: дофига логистики(склады, специалисты, магазины, гарантия ремонта, дистрибуция...)

Эксплуатация (проблемы):

- Амортизация. Вывод из строя
- Физический доступ (космос как?)
- среда
- долгосрочные эффекты ОЛОВЯННЫЕ НИТИ

Хорошо ли?

- повышается ответственность и безопасность
- **устаревание** - нужно переписать полностью, модульность, виртуализация или сделать интерфейс (SSD прикидывается магнитной лентой =))